

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

YOLOv9-Based Human Face Detection and Counting Under Human-Animal Faces, Complex Imaging Environments, and Image Qualities

SIVARANJINI PERIKAMANA NARAYANAN^{1,2}, (Student Member, IEEE), M. SABARIMALAI MANIKANDAN¹, (Senior Member, IEEE), and LINGA REDDY CENKERAMADDI², (Senior Member, IEEE)

¹Department of Electrical Engineering, Indian Institute of Technology Palakkad, Palakkad, Kerala 678623, India

²Department of Information and Communication Technology, University of Agder, NO-4879 Grimstad, Norway

Corresponding author: Linga Reddy Cenkeramaddi (e-mail: linga.cenkeramaddi@uia.no).

ABSTRACT Automatic human face detection and counting can play a vital role in the recognition and tracking of infant and adult faces in both outdoor and indoor human surveillance applications and facial-vital sign measurement. Despite the advancements in deep learning networks, accurate and reliable detection is still a challenging task in the presence of different kinds of objects, animal faces, and image characteristics. In this paper, we study the effectiveness of the YOLOv9-based face detection and counting method under skin color variations, different face sizes and number of faces, mixture of human and animal faces, image properties (brightness, contrast, illumination, glare), image qualities (image blurring, low-light facial images, lens flared images, grainy images taken at night, different types of noises), and pareidolia effects. The proposed method is trained and validated using the Wider Face database. In addition, we created image databases with different kinds of image qualities and image characteristics with the above-mentioned challenges. The YOLOv9-based face detection model achieves a precision of 86%, a recall of 62.8%, and a mean average precision of 70.8% at an inference time of 15.2 ms on the Wider Face database. Evaluation results demonstrate that the YOLOv9-based face counting outperforms most of the state-of-the-art face detection and people counting methods with a mean absolute error (MAE) of 3.36 and root mean square error (RMSE) of 22.38. The model was also deployed on the Raspberry Pi edge computing platform to study the real-time performance. The YOLOv9-based method achieves an MAE of 0.53-5.76 on the untrained infant database with Gaussian, salt and pepper, and speckle noises and an MAE of 0.43-2.87 on faces inside vehicles. The study further highlights the effectiveness of the YOLOv9 model in achieving promising face detection and counting results under a range of illumination and skin color variations. Evaluation results on a wide variety of animal faces and pareidolia-induced faces demonstrate more false positives due to the lack of contextual intelligence in the generation of deep-face models. Further, results show that the deep-face model detects artificial faces (statues, art, paintings, posters) if the model is deployed in uncontrolled face-based application environments. The performance of the model is degraded under different kinds of noise and blurred images. The results of this study highlight that performance can be improved by using noise-specific filtering techniques with optimal filtering parameters, but this requires the automatic identification of noise types and their corresponding parameters.

INDEX TERMS Face Detection and People Counting, YOLOv9, Crowd Counting, Object Detection Method, Image Denoising, Speckle Noise, Salt and Pepper Noise, Gaussian Noise, Motion Blur, and Filtering

I. INTRODUCTION

AUTOMATIC human face detection and counting method is highly demanded for automatic recognition

and tracking of infant and adult faces in both outdoor and indoor human surveillance applications, including biometric authentication and security, traffic management,

pedestrian monitoring, human emotional recognition, expressive face analysis, human-machine interactions, vital sign measurement, sentiment analysis, affective computing interfaces, human presence detection and counting in the presence of other objects and pets in the home, and crowd management [1]–[4]. Despite the advancements in deep learning networks, accurate and reliable detection is still a challenging task under different face skin colors, illumination changes, poor image resolution, and noisy images, which is highly demanded to improve the accuracy and reliability of the facial features-based health, emotional, security and defense applications, including video surveillance, public security, access control, urban planning, traffic management and forecasting, human-computer interaction, media, and crowd management [5]–[8]. In security and surveillance systems in public places such as airports, railway stations, stadiums, and shopping malls, accurate counting of the number of individuals in the monitored areas can help in assessing crowd density, identifying potential security threats, and managing emergency responses effectively [9], [10]. In retail and commercial environments, human counting may help in understanding customer behavior and optimizing staff allocation, improving customer service, and making informed decisions about store layout and marketing strategies [8]. Human counting data also plays a crucial role in urban planning, traffic management, urban security, crime management, and optimizing public transportation systems. The ability to accurately detect and count faces in a scene can remarkably enhance the functionality and efficiency of systems in these areas. As manual counting becomes tedious for larger crowds, an automatic human counting system can help to analyze and improve the efficiency of crowd monitoring systems [11].

Face detection is vital for many applications such as face recognition [29], [30], face biometrics [4], [31], facial expression identification [3], [32], human emotion recognition [1], [33], and contactless vital sign measurement [34]. In addition, face detection has been used for human counting for many decades. In past studies, many face detection methods were studied by using the Viola-Jones Detector, Histogram of Oriented Gradients (HOG), Multi-task Cascaded Convolutional Networks (MTCNN), and deep learning methods [9], [35], [36]. However, one of the main challenges in real-time face detection is the variation in pose, scale, lighting, and skin color, which can considerably affect the performance of face detection algorithms [2], [7], [37]. Hence, this paper proposes a face detection and people counting model using the YOLOv9 object detection method, which is capable of handling face skin color variations effectively.

A. EXISTING RELEVANT METHODS AND LIMITATIONS

Many human counting methods were presented in the past using regression-based, crowd density-based, and person detection-based algorithms [6], [7], [9], [11], [35], [36]. In this section, we briefly summarize some of the

relevant methods and their features. Liu et al. (2019) [6] proposed a crowd counting method using the deep structured scale integration network (DSSINet) by incorporating scale variation of people in crowd density maps. Ranjan et al. (2018) [11] presented an iterative counting convolutional neural network (ic-CNN) for crowd density estimation and crowd counting. However, the regression-based methods fail to localize people and cannot be used for any further analysis other than counting [7].

Sam et al. (2021) [7] developed an LSC-CNN model using point head annotations for counting and localizing the heads in a crowd. Neethu et al. (2016) [35] proposed a face detection-based people counting system by combining face detection techniques such as Haar classifier, Haar classifier for profile face, and normalized pixel difference (NPD) to achieve an accuracy of 90.20%. Object detection algorithms such as region-based convolutional neural network (R-CNN), faster R-CNN, you only look once (YOLO), and single shot detector (SSD) were also used for face detection and crowd counting. Atrash et al. (2021) [36] trained the Faster R-CNN and SSD object detection models for face detection and people counting using the Wider Face dataset. The models gave an accuracy of 50% with mean absolute errors of 7.5 and 8.6 for faster RCNN and SSD, respectively. Vasantha et al. (2023) [9] presented a comparative study of different face detection models such as MTCNN, YuNet, HOG-SVM, and YOLOv8 for people counting applications. Each method demonstrated efficiency in particular scenarios, with the YOLOv8 object detection model providing the most superior people counting performance regardless of the size or viewpoint. Ha et al. (2021) [38] proposed a face and head detection pair mechanism to improve the performance of people counting under various postures or occlusions. The method was validated using the Hollywood Heads, Wider Face, and Casablanca datasets to achieve average precisions of 79.45%, 69.23%, and 89.34%, respectively. Table 1 summarizes the performance of some of the existing YOLO-based face detection methods. Jaiswal et al. (2024) [39] analyzed the performance of YOLOv5, YOLOv7, and YOLOv9 for face and person detection applications. The study showed that YOLOv9 can achieve very good accuracy compared to its previous versions, particularly in complex settings, although it required higher computational resources, making them ideal for high-performance systems. However, crowd counting using object detection methods may fail to accurately count extremely dense crowds [11].

Although traditional face detection methods are effective in controlled environments, these methods often fail under diverse real-world conditions such as variations in lighting and skin color [27]. Additionally, the noises that are introduced during image acquisition further affect the performance of the methods [40]. In most of the existing face detection methods, the performance of deep learning methods is not well-studied under changes in illumination, noise effects, and image blur conditions, which can cause severe performance degradation [30]. In some studies, the

TABLE 1. Summary of existing YOLO-based face detection methods

Reference	YOLO Version	Database	Performance
Garg et al. (2018) [12]	YOLO	FDDB	IoU ACC: 92.2%
Silva et al. (2019) [13]	YOLO-FD	WIDER FACE, UCCS, UFDD	WF: AP: 0.908 (Easy), 0.894 (Medium), 0.809 (Hard) UFDD: AP: 0.688
Li et al. (2020) [14]	YOLOv3	WIDER FACE, CelebA, FDDB	FDDB: ACC: 93.9%
Tuli et al. (2021) [15]	Modified YOLOv3	WIDER FACE	mAP: 93.8% (Easy), 92.4% (Medium), 86.5% (Hard)
Chen et al. (2021) [16]	YOLOv3	WIDER FACE, FDDB	WF: DR: 0.899 (Easy), 0.872 (Medium), 0.693 (Hard)
Aung et al. (2021) [17]	YOLOv2 + VGG16	FDDB	AP: 95%; ACC: 0.93%
Castellano et al. (2021) [18]	YOLOv5	WIDER FACE, MIVIA Age	P: 94.6%, R: 63.1%, AP: 67.6%
Abbasi et al. (2021) [19]	YOLOv4	—	P: 92%; R: 97%; mAP0.5: 97%
Gombe et al. (2021) [20]	YOLOv3-Tiny	WIDER FACE, FDDB	WF: Avg IoU: 0.38; mAP: 0.34; P: 0.52; R: 0.36; F1: 0.42 FDDB: Avg IoU: 0.38; mAP: 0.44; P: 0.55; R: 0.54; F1: 0.54
Moreno et al. (2021) [21]	YOLO-Face (YOLOv3)	WIDER FACE, FDDB, CelebA, Honda/UCSD	FDDB: P-99.8%; R-72.9%; ACC: 72.8% CelebA: P-99.9%; R-99.6%; ACC-99.6% WF: AP: 95.8% (Easy), 94.2% (Medium), 87.4% (Hard)
Yu et al. (2022) [22]	YOLOv5	WIDER FACE	ACC: 98.7% (Easy), 97.2% (Medium), 87.7% (Hard)
Pebrianto et al. (2022) [23]	YOLOv3-Tiny	Google Open Images	P: 70%, R: 69%, mAP: 66%
Khalili et al. (2022) [24]	WBF ensemble YOLOv1-v4	WIDER FACE	mAP: 0.69 (Easy), 0.59 (Medium), 0.61 (Hard)
Khalili et al. (2022) [24]	NMW ensemble YOLOv1-v4	WIDER FACE	mAP: 0.68 (Easy), 0.58 (Medium), 0.60 (Hard)
Gao et al. (2022) [25]	YOLOv3 Tiny	WIDER FACE	mAP: 95.26% (Easy), 89.2% (Medium), 77.9% (Hard)
Qi et al. (2023) [26]	YOLOv5	WIDER FACE	mAP: 96.67% (Easy), 95.08% (Medium), 86.55% (Hard)
Chan et al. (2024) [27]	YOLOv7	WIDER FACE, FDDB	mAP: 96.11% (Easy), 95.12% (Medium), 88.14% (Hard)
Amoudi et al. (2024) [28]	YOLOv8n	WIDER FACE	mAP50: 0.676, mAP50-95: 0.37

FDDB: Face Detection Data Set and Benchmark; FD: Face Detection; UCCS: UnConstrained College Students Dataset; UFDD: Unconstrained Face Detection Dataset; IoU: Intersection over Union; ACC: Accuracy; WF: WIDER FACE; AP: Average Precision; DR: Detection Recall; R: Recall; P: Precision; mAP: Mean Average Precision; Avg: Average; F1: F1-Score;

robustness of face detection or face recognition methods is evaluated by using noisy images, or by synthetically adding different amounts and types of noise and blurring effects to the noise-free face images [30], [32], [41]. Chen (2019) [30] studied the effects of speckle noise, Gaussian white noise, Poisson noise, and salt and pepper noise on the existing face recognition methods. Kumar et al. (2020) [32] used different levels of salt and pepper and Gaussian noises to evaluate the performance of their facial expression recognition model. In [41], Zhou et al. (2018) discussed the challenges of using face detection methods trained and tested on good-quality images for practical applications, including surveillance systems. The authors studied the degradation in the performance of existing face detection methods by testing them on low-quality images with different levels of contrast, noise, and blur. The study demonstrated that the face detectors are not robust for low-quality images. In certain studies, the methods were trained using face image data augmented with different types of noise and blurring effects [29], [31]. Zulfikar et al. (2019) [29] developed a face recognition system using a face image dataset augmented with five illumination levels and five different noise types, such as Gaussian noise, Poisson noise, speckle noise, local variance noise, and salt and pepper noise. Yu et al. (2018) [31] developed a CNN-based face biometric quality assessment method to improve the reliability of face recognition methods by using a database augmented with five types of common image distortions in video surveillance, such as blurring, low-resolution, salt and pepper noise, additive white Gaussian noise, and Poisson noise.

The emergence of deep learning and convolutional neural networks (CNNs) has revolutionized face detection, with models such as Faster R-CNN, SSD, and the YOLO family [9]. Many state-of-the-art deep learning models, such as faster R-CNN and SSD, provide high detection accuracy but have higher computational complexity and cost. This makes them less suitable for deployment on resource-constrained devices or in scenarios requiring real-time processing. Among these, YOLO has gained prominence for its superior real-time object detection ability with high accuracy [27]. The latest version of YOLO, YOLOv9, incorporates architectural improvements to strengthen its detection capabilities while building upon the merits of its predecessors [42]. Although data augmentation can be adapted with the standard facial image databases (Wider Face database) by including geometric transformations, image properties/attributes, contrast, brightness, blurring, noises, etc. to improve the accuracy and robustness of the model, ensuring the addition of all possible variations of image qualities or image geometric variations will still be a challenging task in the training process. However, one can argue that the data augmentation process can include all possible image qualities/attributes and geometrical variations. In such training cases, achieving accurate, robust, and discriminative features would be another challenging task under severe noises and blurring effects, in addition to other issues of overfitting and speed of convergence (as it leads to more outliers in data). Therefore, by incorporating image quality assessment, noise identification, and noise-specific denoising techniques into the models that are created with

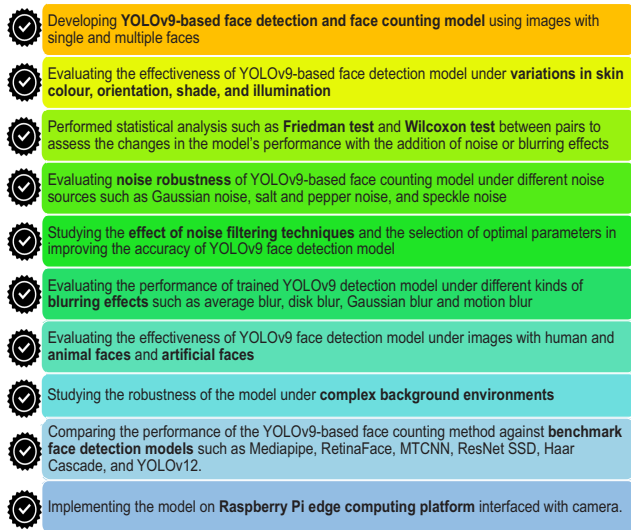


FIGURE 1. Major contributions of this paper.

and without data augmentation techniques, we can ensure the reliability of the model.

In this paper, we present a YOLOv9-based face detection and counting method and study the performance of face detection under skin color variations, different face sizes and number of faces, image properties (brightness, contrast, illumination, glare), image qualities (image blurring, low-light facial images, lens flared images, grainy images taken at night, different types of noises), and pareidolia effects. This paper further presents the performance of face counting in the presence of artificial faces (wall-painted faces, crafted faces, statue faces), and face pareidolia (the misperception of illusory faces formed by objects in the image), leading to more false positives caused by ambiguous visual facial patterns mimicking facial features, which are not addressed in past studies. We also created a large, wide variety of test datasets for this study, in addition to the Wider Face dataset, with practical applications of indoor and outdoor scenarios. Fig. 1 illustrates the primary contributions of this paper in comparison to existing methods. These contributions include the demonstration of the failure cases and robustness of the YOLOv9-based method in face detection and counting under real-world application scenarios, as well as the importance of image quality assessment and noise-specific image denoising techniques in reducing the false positives (false alarms) of the method, which were not addressed in past studies. We further highlight the importance of postprocessing in ensuring the detection of real human faces in indoor and/or outdoor visual images because deep learning networks, including YOLO architectures, are not able to distinguish real faces in the presence of artificial faces and face pareidolia. Extensive evaluation results on a wide variety of real-world images demonstrate that the above-mentioned key challenges need to be addressed to improve the accuracy and reliability of deep learning-based face detection and

counting methods for face recognition and drone and/or robot-based crowd surveillance. The major contributions of this paper are shown in Fig. 1 and are summarized below.

- 1) An accurate and lightweight face detection and people counting method is presented using the YOLOv9 object detection algorithm.
- 2) Image properties such as the skin color of the subjects, illumination, resolution, and background texture are studied to investigate their effect on face detection using the YOLOv9 model.
- 3) The developed model is tested using untrained images with different backgrounds and application scenarios, such as inside vehicles or in images containing combinations of both humans and animals, or artificial faces. Further, the trained model is evaluated using infant faces to study generalization.
- 4) Statistical tests, such as the Friedman test and the Wilcoxon test, are performed to study whether the addition of noise or blurring effects to the same images has caused significant changes to the model's performance.
- 5) The impact of different types and levels of noises on the face detection method is studied by adding different amounts of Gaussian, speckle, and salt and pepper noise to the images. Further, the impact of blurring effects is also examined.
- 6) The effect of type and parameters of filters for specific noise types is presented to emphasize the need for noise estimation and noise-specific denoising techniques to enhance face detection and people counting performance.
- 7) The performance of the YOLOv9-based face detection and face counting model is compared against a YOLOv12-based face detection model trained using the same WiderFace dataset with 5-fold cross validation and against benchmark face detection models such as MTCNN, Haar Cascade, RetinaFace, Mediapipe, and ResNet SSD, to demonstrate the superior performance of the model for dense crowds.
- 8) The YOLOv9-based face detection and face counting model is implemented on the Raspberry Pi edge computing platform to measure the latency of the model by interfacing with an RGB camera.

The YOLOv9 face detection model is developed using the Wider Face database, which has been extensively used in past studies as it shows diversity in human poses, ranges, and skin color. We further evaluate and test the reliability of the model using real-world images.

The rest of the paper is organized as follows. Section II describes the YOLOv9 architecture used for face detection. The evaluation results of the study are presented in Section III for face detection and people counting, along with the noise robustness study, blurring, and denoising effects. Finally, the conclusions are summarised in Section IV.

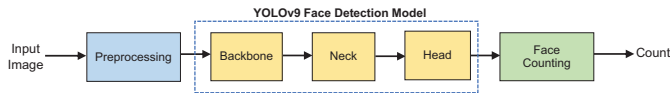


FIGURE 2. Illustration of the YOLOv9-based face detection and counting pipeline. The input image undergoes preprocessing, which includes resizing, normalization, and augmentation to ensure consistency and robustness. The processed image is passed through the YOLOv9 model, comprising three primary components: the backbone, which extracts low and high-level visual features; the neck, which fuses multi-scale features for improved detection of faces of varying sizes; and the head, which generates bounding box coordinates, objectness scores, and class probabilities. The detected faces are subsequently passed to the face counting module, which aggregates the final count. This modular flow enables efficient end-to-end face detection and people counting, optimized for real-time applications.

II. MATERIALS AND METHODS

The generalized block diagram of the YOLOv9-based people counting method is shown in Fig. 2. The method consists of major steps of preprocessing of the input image, YOLOv9-based face detection, and face bounding box-based people counting. This section provides an overview of the You Only Look Once (YOLO) object detection algorithm and the YOLOv9 architecture, which is used for face detection and people counting.

A. YOU ONLY LOOK ONCE (YOLO)

Object detection is a computer vision task that involves the identification and localization of objects within an image or a video frame. In contrast to image classification problems, which involve predicting the class label of an entire image, object detection provides the precise bounding boxes around each detected object along with their corresponding class labels. Real-time object detection plays a major role in various applications and fields, including robotics, autonomous vehicles, video surveillance, medical imaging, agriculture, security, and augmented reality [43].

Mainly, two types of convolutional neural networks (CNN) are used for object detection: one-stage and two-stage [43]. The two-stage detectors, including R-CNN, faster R-CNN, region-based fully convolutional networks (R-FCN), and mask R-CNN, provide rough region proposals or suggestions in images and then use a head network to classify and regress these proposals [44]. Region proposals are generated using methods such as selective search, edge boxes, or R-CNN series, which create candidate areas of different scales and aspect ratios for potential objects [43]. Although the two-stage detection algorithms provide superior accuracy and localization, they have complex architecture and require more time and computational resources [44]. On the other hand, single-stage detectors can directly detect the object locations and categories in a single step, skipping the region proposal stage [44]. The CNN-based backbone network employs detection heads to detect the category and bounding box of each item after extracting the features from the images [43]. Additional layers are added to detect objects of different sizes by combining features of multiple scales [43]. Although one-stage detection algorithms have lower accuracy

and limited localization precision in the case of small objects or cluttered scenes, they are faster than two-stage detectors, making them suitable for real-time tasks with constrained computational resources [44].

Among different single-stage object detection algorithms such as YOLO, SSD, CornerNet, and M2Det, the YOLO framework achieved huge popularity due to its higher speed, accuracy, and real-time end-to-end approach for object detection [43], [44]. YOLO uses reference bounding boxes called anchor boxes, with predefined sizes and aspect ratios, to accurately detect objects of different sizes and aspect ratios within an image [45]. The various stages of the YOLO object detection algorithm are as follows [45]:

- 1) **Input:** Images or individual frames of videos are given as the input.
- 2) **Grid Division:** The input image is divided into a grid of cells whose grid size is determined by the size of the image and the size of the final convolution feature map of the network. Each cell detects the objects in that particular cell.
- 3) **Feature Extraction:** Each cell is passed to a pre-trained CNN network to extract features such as contours, colors, and textures.
- 4) **Objectness Score:** The Logical regression function calculates the probability of the presence of an object and assigns an “objectness score” to each cell.
- 5) **Class Probability:** If an object is predicted in a cell, the softmax function calculates the conditional probability of the object for each of the possible classes to assign a class to the object with the corresponding probability.
- 6) **Bounding Box:** The approximate center of the object is calculated with reference to the cell edges to find the dimensions of the box that surrounds the object. An enclosing bounding box shows the size and location of the detected object. The width and height of the bounding box are relative to the size of the grid cell to adapt to the changes in the size of objects in the image.
- 7) **Non-Maximum suppression (NMS):** The confidence score indicates the confidence of the model that the object belongs to the predicted class. Redundant or overlapping bounding boxes having low confidence scores are eliminated using non-maxima suppression to improve detection accuracy.
- 8) **Output:** Finally, the YOLO algorithm gives a collection of bounding boxes as the final output, with each box containing a class and confidence score.

B. YOLOV9 ARCHITECTURE

The YOLO object detection algorithm has undergone multiple iterations over the years to overcome the limitations of its previous versions and enhance performance and real-time object detection with improved class predictions, deeper architectures, and more detection layers [43], [45]. YOLOv9 is a popular version of the YOLO family that was introduced in February 2024 [42]. The auxiliary

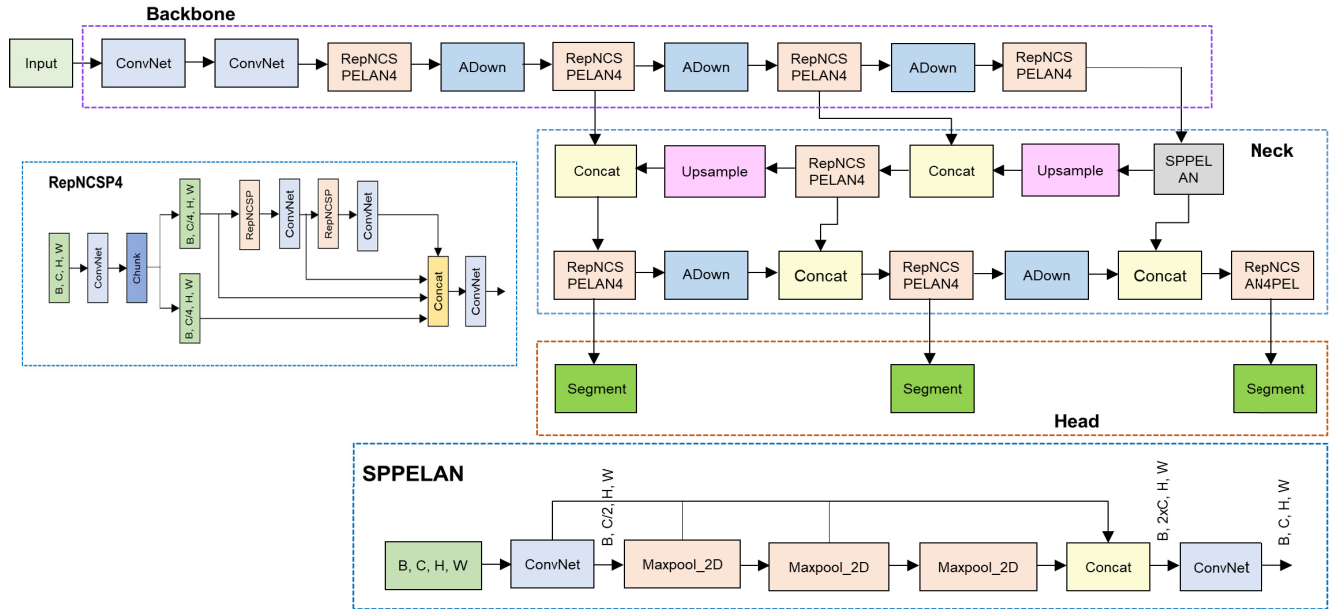


FIGURE 3. Overview of the YOLOv9-based network architecture used for face detection, highlighting the flow and function of each major component [46]. The Backbone module extracts multi-scale feature representations from the input image using a series of ConvNet layers and repeated RepNCSPELAN4 and ADown blocks. These layers gradually reduce the spatial resolution while increasing feature abstraction. The RepNCSPELAN4 sub-block within the backbone enhances feature reuse and spatial awareness by combining parallel convolutions with concatenation operations. The Neck module is responsible for aggregating and refining features across multiple scales. It uses concatenation, upsampling, and downsampling operations, along with SPPELAN, to improve the receptive field and feature diversity. The SPPELAN block itself applies multiple 2D max-pooling operations in parallel to extract multi-scale context before merging the outputs through concatenation and convolution. The Head module predicts segmentation maps for different levels of the feature pyramid. It applies convolutional operations to the output from the neck to generate fine-grained, spatially accurate segmentation outputs at multiple scales. This modular architecture enables the model to preserve spatial detail while enhancing contextual understanding, which is critical for accurate and robust face detection in varied environments.

supervision framework of YOLOv9 helps to maintain speed and lightweight, along with high accuracy [42].

In this study, we adopt the YOLOv9 architecture for face detection and counting due to its efficiency, robustness, and high detection accuracy. The architecture is built upon a modular structure composed of three major components: the Backbone, the Neck, and the Head [42], as illustrated in Fig. 3. The Backbone is responsible for extracting hierarchical features from the input image using stacked convolutional blocks, downsampling operations (ADown), and advanced feature aggregators. It incorporates the RepNCSPELAN4 block, a lightweight module based on the Generalized Efficient Layer Aggregation Network (GELAN), which enhances representational power through repeated and parallel convolution paths [42], [47], [48]. This improves feature reuse while maintaining low computational cost.

The Neck fuses multi-scale features using a combination of upsampling, concatenation, and additional RepNCS blocks. It also includes the SPPELAN module, which performs parallel spatial pyramid pooling to enrich the receptive field. These operations enable the model to detect faces at different sizes and scales while preserving contextual information. The Head generates the final detection outputs. It processes the aggregated features to produce bounding boxes, class probabilities, and objectness scores. Segment layers are used in multiple branches to improve spatial precision at different

feature map resolutions.

One of the key innovations in YOLOv9 is the use of Programmable Gradient Information (PGI), which incorporates a reversible auxiliary branch and multi-level auxiliary features to alleviate information bottlenecks during training. This enables better gradient flow and improved learning efficiency. Additionally, by eliminating depth-wise convolutions in favor of standard convolutional layers, YOLOv9 achieves a better trade-off between model capacity and hardware efficiency. Compared to its predecessor YOLOv8, YOLOv9 reduces parameter count by 16% and computation by 27%, while achieving a 1.7% increase in detection performance [42]. These architectural refinements make YOLOv9 well-suited for real-time face detection tasks in complex or degraded environments.

Unlike regression or crowd density-based methods, in this study, people counting is achieved by counting the face bounding boxes predicted by the YOLOv9 face detection model in the image. The performance of the face detection model affects the people counting accuracy. The YOLOv9 face detection model is trained with images containing variations in pose, size, and skin color to improve the performance of the model.

Face Detection as an Object Detection Problem Let $x \in \mathbb{R}^{H \times W \times 3}$ be an input RGB image. The goal of the object

detector is to learn a function $f_{\theta}(x) \rightarrow \hat{y}$, where $\hat{y}$ is a set of predictions for bounding boxes, objectness scores, and class probabilities:

$$\hat{y} = \{(b_i, c_i, s_i)\}_{i=1}^N,$$

where $b_i \in \mathbb{R}^4$ represents the bounding box coordinates, $c_i \in \{0, 1\}$ denotes the class label, and $s_i \in [0, 1]$ is the predicted confidence score.

The YOLOv9 model learns to approximate a highly non-linear mapping from the input image domain $\mathbb{R}^{H \times W \times 3}$ to a structured output space comprising bounding box coordinates, class probabilities, and objectness scores. This mapping is parametric in nature, governed by a fixed set of learnable weights θ , which are optimized via gradient descent on a composite loss function $\mathcal{L}_{\text{total}}$. The effectiveness of this approximation is inherently dependent on the diversity, quality, and representativeness of the training data. In particular, its accuracy degrades when evaluated on inputs that deviate significantly from the training distribution, which is a common challenge in real-world face detection scenarios.

In this study, we focus on examining the scope and limitations of this parametric approximation under conditions of visual degradation. Two key assumptions underlying the learned mapping are explicitly investigated:

- **Distributional generalization:** The model is trained on the Wider Face dataset, and thus its performance may deteriorate on out-of-distribution (OOD) inputs such as noisy, blurred, occluded, or non-human faces.
- **Parametric sensitivity:** Detection performance is influenced by the choice of hyperparameters (e.g., IoU thresholds, loss weights), which modulate the model's responsiveness to input perturbations and prediction uncertainty.

The primary objective of this work is to develop and rigorously evaluate a YOLOv9-based face detection and counting framework that remains robust across varying levels of image degradation. We assess the model using the standard Wider Face dataset, under different image degradations (blur effects and noises) and real-world test cases. Statistical tests, including the Friedman and Wilcoxon signed-rank tests, are used to determine the significance of observed performance differences across distortion levels.

III. RESULTS AND DISCUSSIONS

In this section, the database and performance metrics used to evaluate the performance of the face detection and people counting methods are presented, along with an assessment of the method's accuracy, processing speed, model size, noise robustness, and generalizability.

A. DATABASE DESCRIPTION

The YOLOv9 face detection model was trained and validated using the Wider Face database¹ [49]. The database contains

¹The Wider Face database is available at <http://shuoyang1213.me/WIDERFACE/>

32203 images with 393703 face-bounding box labels. The images show a high degree of variability in pose, scale, and occlusion, which increases the reliability of the face detection model. We used 12880 images from the Wider Face database to train the YOLOv9 face detection model. The testing database consisted of 3226 images. The images were resized to the dimension of 640×640 pixels and trained for 100 epochs with a batch size of 8.

The performance of the people counting method is evaluated using standard performance evaluation metrics such as the mean absolute error (MAE), root mean square error (RMSE), and standard deviation (SD) [6]. For the given reference count C_r and the estimated count C_e , the MAE is computed as

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |C_r(i) - C_e(i)|, \quad (1)$$

where N is the number of test images. RMSE is computed as

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (C_r(i) - C_e(i))^2}. \quad (2)$$

The standard deviation is computed as

$$\text{SD} = \sqrt{\frac{1}{N} \sum_{i=1}^N (C_r(i) - C_e(i))^2 - \left(\frac{1}{N} \sum_{i=1}^N (C_r(i) - C_e(i))\right)^2} \quad (3)$$

The loss of an object detection method is an important performance metric that measures the deviation of the detected outputs of the model from the actual location and label of the object in an image [50]. To improve the model's performance, the loss has to be minimized as much as possible during the training process [50]. The loss function in YOLO networks is defined by considering its object and background confidences and intersection over union (IoU) scores [45]. The loss function is defined as the weighted sum of the object confidence, background confidence, and IoU calculated over all the cells and scales of the grid [45]. The major loss functions explored in YOLO architectures include the box loss, objectness (Obj) loss, intersection over union (IoU) loss, classification (cls) loss, and distributional focal (dfl) loss [50]. YOLOv9 utilizes the box loss, dfl loss, and cls loss as [42]:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{dfl}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}, \quad (4)$$

where $\mathcal{L}_{\text{box}}$ is the regression loss for bounding box prediction, $\mathcal{L}_{\text{dfl}}$ is the distribution focal loss to improve box quality, $\mathcal{L}_{\text{cls}}$ is the classification loss for face/non-face. The YOLOv9 face detection performance is evaluated using metrics such as precision (P), mean absolute precision (mAP), and recall (R) [51]. The YOLOv9 detector models a parametric mapping $f_{\theta} : \mathbb{R}^{H \times W \times 3} \rightarrow \mathbb{R}^{N \times 6}$, where each output vector corresponds to bounding box coordinates, objectness score, and class probability. This mapping is learned by minimizing a loss function composed of a localization loss $\mathcal{L}_{\text{box}}$, an objectness loss $\mathcal{L}_{\text{obj}}$, and a classification loss $\mathcal{L}_{\text{cls}}$. While this

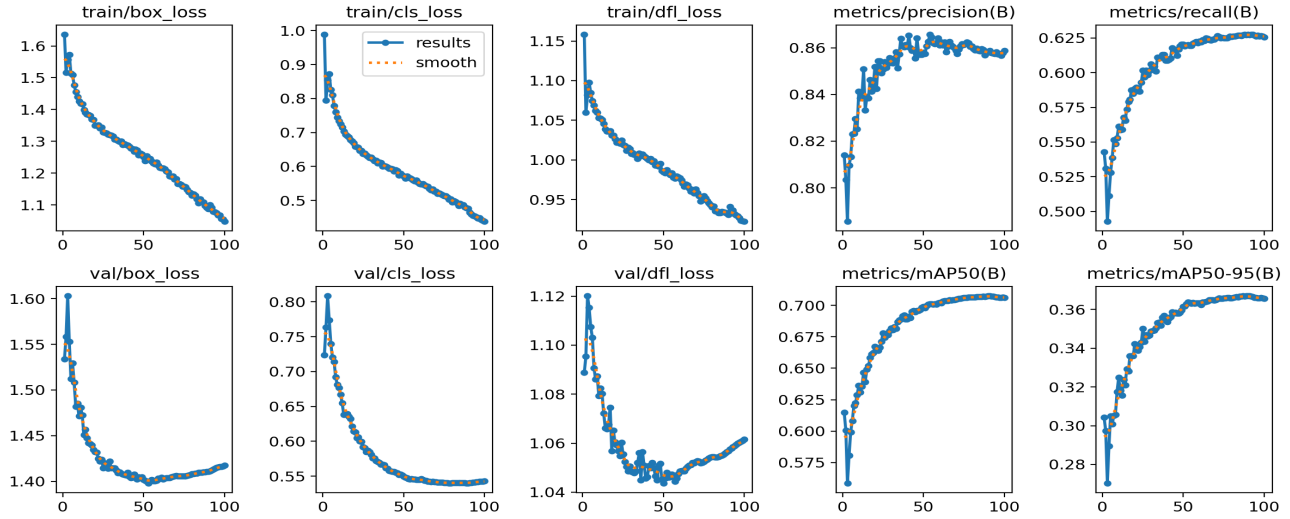


FIGURE 4. Training and validation losses and performance metrics of the YOLOv9 face detection model across epochs. This figure illustrates the evolution of different training and validation loss components—box loss, classification (cls) loss, and distribution focal loss (DFL)—as well as key performance metrics (precision, recall, mAP50, and mAP50-95) over 100 training epochs. A consistent and steady decrease in all loss values indicates effective convergence of the model. Notably, the validation losses plateau slightly earlier than the training losses, suggesting good generalization without overfitting. Precision and recall curves demonstrate significant early gains before stabilizing, implying that the model rapidly learns to distinguish face regions accurately. The mAP50 and mAP50-95 metrics exhibit continuous improvement, confirming progressive enhancement in detection accuracy.

formulation is effective under diverse training data, it assumes sufficient training coverage of all distortions. As shown in our noise robustness analysis, this approximation degrades under out-of-distribution distortions, revealing a limitation of purely parametric approaches. Fig. 4 illustrates the training and validation loss and performance metric curves for different training epochs of the YOLOv9 face detection model. Fig. 5 shows the per-image loss distribution of the YOLOv9-based face detection model for the test images of the Wider Face database. The curve is heavily right-skewed (i.e., most images have low loss).

B. PERFORMANCE EVALUATION ON THE WIDER FACE DATABASE

Based on the face bounding box annotations of the Wider Face database, the YOLOv9 face detection model achieved a precision of 86% and a recall rate of 62.8%. The YOLOv9 face detection model achieved a mean average precision of 70.8% for an IoU threshold of 0.5, which is 4.73% higher than its predecessor YOLOv8 (mAP=67.6%) [28]. In addition to standard performance metrics such as precision and recall, the resource efficiency of the model is an important performance indicator for deploying the model in real-time applications. The YOLOv9 face detection model has a memory requirement of 49.2 MB and an inference time of 15.2 ms (≈ 65.79 FPS).

Fig. 6 illustrates the performance of the face detection and people counting method using some of the images from the Wider Face database. Fig. 7 shows the error distribution of the YOLOv9-based face detection and people counting model for the Wider Face database. The histogram shows that most prediction errors cluster tightly around a small negative

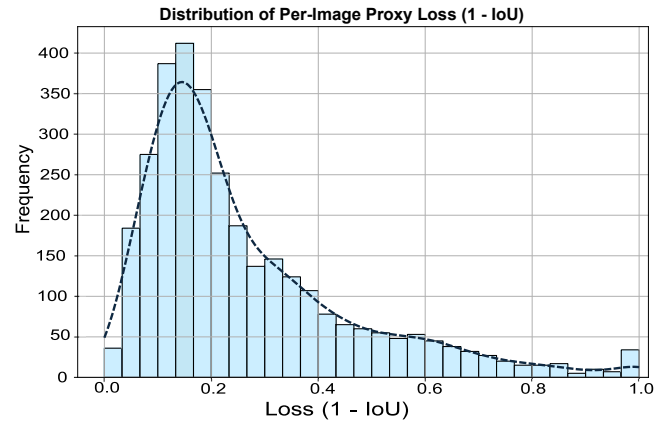


FIGURE 5. Distribution of per-image proxy loss for the YOLOv9-based face detection model on the Wider Face dataset. The histogram depicts the frequency distribution of per-image proxy loss, serving as an indicator of prediction accuracy across individual test images. The distribution is heavily skewed toward lower loss values, peaking near 0.15, which implies that a large number of predictions have high overlap with the ground truth (i.e., high IoU). The long tail toward higher loss values suggests the presence of some harder samples or outliers that the model struggles to localize accurately. This imbalance highlights the robustness of the model for the majority of instances, while also pointing to potential areas for improvement.

value, with a mean error of -2.82. The 95% confidence interval ranges from -3.58 to -2.05. The narrow spread indicates a high degree of prediction stability. However, the negative bias may point to a systematic shift in bounding box estimation, which could require recalibration of model outputs for further improvement. Table 2 summarizes the performance of the YOLOv9-based people counting model in terms of benchmark metrics and compares the performance with some of the existing people counting methods. The



FIGURE 6. Illustrates the detection performance of the YOLOv9-based face detection and people counting method on images with varied crowd settings from the Wider Face database [49], including (left) a Caucasian group with dark attire and (right) a large, diverse crowd with significant skin tone variation. Results confirm robust detection across different crowd densities, poses, and lighting conditions.

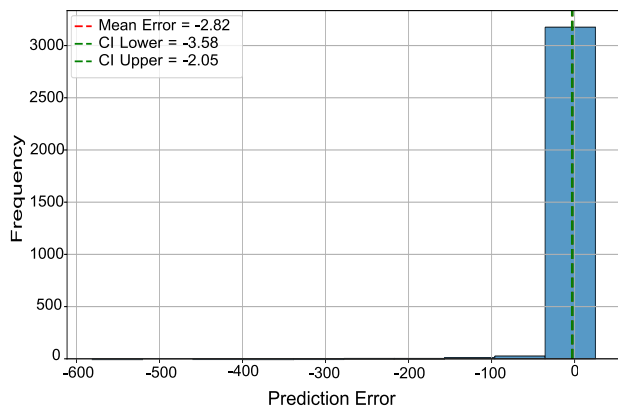


FIGURE 7. Error distribution of the YOLOv9-based face detection model on the Wider Face database. The histogram illustrates that the majority of prediction errors are clustered near zero, with a mean error of -2.82. The 95% confidence interval ranges from -3.58 to -2.05, indicating low variability in the prediction error. The tight concentration suggests that the YOLOv9 model consistently produces accurate bounding box predictions with a slight tendency toward underestimation. The absence of significant outliers further confirms the model's robustness and reliability in face localization across the dataset.

YOLOv9-based people counting method achieved an MAE $\pm$ SD of 3.36 ± 22.20 and an RMSE of 22.38 faces on the Wider Face database. The method outperforms most of the people counting models trained using the Wider Face database and other popular face detection databases such as ShanghaiTech, UCF-QNRF, and UCF_CC_50. The YOLOv9 object detection method surpasses the Faster RCNN and SSD object detection methods with an MAE improvement of 55.2% and 60.93%, respectively, on the Wider Face database. The model was also evaluated using the ShanghaiTech A and B datasets, which were widely used for developing people counting methods in past studies [53]. The ShanghaiTech datasets have very dense crowds in which the faces are not always visible. The model achieved an MAE of 298.05 and 81.49 on the ShanghaiTech A and ShanghaiTech B datasets, respectively.

TABLE 2. Performance comparison of different people counting models

Method	Model	Dataset	MAE $\pm$ SD	RMSE
Liu et al. (2019) [6]	DSSINet	UCF-QNRF	99.1	159.2
Liu et al. (2019) [6]	DSSINet	UCF_CC_50	216.9	302.4
Liu et al. (2019) [6]	DSSINet	ShanghaiTechA	60.63	96.04
Liu et al. (2019) [6]	DSSINet	ShanghaiTechB	6.85	10.34
Wang et al. (2020) [5]	DM-Count	UCF-QNRF	85.6	148.3
Wang et al. (2020) [5]	DM-Count	UCF_CC_50	211.0	291.5
Wang et al. (2020) [5]	DM-Count	ShanghaiTechA	59.7	95.7
Wang et al. (2020) [5]	DM-Count	ShanghaiTechB	7.4	11.8
Sam et al. (2021) [7]	LSC-CNN	WIDER FACE	40.5	—
Sam et al. (2021) [7]	LSC-CNN	UCF_CC_50	225.6	302.7
Sam et al. (2021) [7]	LSC-CNN	ShanghaiTechA	66.4	117
Sam et al. (2021) [7]	LSC-CNN	ShanghaiTechB	8.1	12.7
Sam et al. (2021) [7]	LSC-CNN	UCF-QNRF	120.5	218.2
Atrash et al. (2021) [36]	F-RCNN	WIDER FACE	7.5	—
Atrash et al. (2021) [36]	SSD	WIDER FACE	8.6	—
Wang et al. (2021) [37]	Crowd-SDNet	WIDER FACE	2.2	—
Wang et al. (2021) [37]	Crowd-SDNet	ShanghaiTechA	65.1	104.4
Wang et al. (2021) [37]	Crowd-SDNet	ShanghaiTechB	7.8	12.6
Our Method	YOLOv9	WIDER FACE	3.36 ± 22.20	22.38
		ShanghaiTechA	298.05 ± 350.78	460.31
		ShanghaiTechB	81.49 ± 84.09	117.09
		Own database	0.03 ± 0.17	0.18

C. PERFORMANCE STUDY USING OWN DATABASE

To study the impact of image properties on the performance of the YOLOv9 face detection and people counting model, we further evaluated the model on an unseen database consisting of 32 handpicked images showing varying skin tones. The face counts in these images varied from 2 to 60 faces per image. The performance of the face detection and people counting method on these images is summarized in Table 3 along with the description of the images. The skin color is represented using numbers from I to VI, ranging from pale white to dark brown/black, based on the Fitzpatrick skin type color representation [52]. Among these images, the YOLOv9-based face detection and people counting method attained an MAE of 0.03 and RMSE of 0.18, which indicates that the model is robust to skin color, illumination, background texture, and cloth color variations.

TABLE 3. Performance evaluation of the YOLOv9-based face detection and people counting method based on image properties. Results demonstrate that the model showed excellent performance under diverse image properties, such as skin color, illumination, resolution, background texture, and cloth types.

Image	Actual Count	Count by Algorithm	Image Properties				
			Skin Colour	Illumination	Resolution	Background Texture	Cloth Type
Image01	15	15	II, III, VI	Good	640x512	Colourful	Multicolour
Image02	11	11	I, II, III, IV	Good	720x480	Plane	Uniform
Image03	2	2	II, III	Dusky	1800x1800	Nature	Single colour
Image04	7	7	II, V	Good	640x427	Colourful	Multicolour
Image05	43	43	III, IV, V	Dim	708x475	Plane	Uniform
Image06	12	12	I, II, III, IV, V, VI	Good	432x117	White	Black
Image07	5	5	I, II, IV, VI	Good	275x183	Plane	Black
Image08	9	10	I, II, III, IV, V, VI	Good	275x183	Plane	Uniform
Image09	6	6	I, II, IV	Bright	275x183	Plane	Multicolour
Image10	5	5	I, V, VI	Good	275x183	White	Multicolour
Image11	28	28	I, II, III, IV, V, VI	Good	2000x1211	Plane	Multicolour
Image12	26	26	III, IV, V, VI	Good	283x178	Colourful	Multicolour
Image13	6	6	V, VI	Good	275x183	Plane	Multicolour
Image14	6	6	II, III, V, VI	Good	276x183	Nature	Multicolour
Image15	5	5	III, VI	Good	204x247	Plane	Uniform
Image16	5	5	III, V, VI	Good	310x163	Buildings	Multicolour
Image17	3	3	II, III	Good	300x168	Colourful	Uniform
Image18	3	3	VI	Good	275x183	Blurred	Uniform
Image19	4	4	VI	Good	299x168	Nature	Multicolour
Image20	3	3	VI	Dim	275x183	Nature	Uniform
Image21	6	6	I, II, III	Bright	1200x900	Nature	Multicolour
Image22	60	60	I, II, III, IV, V, VI	Good	1427x746	Plane	Single color
Image23	7	7	III	Good	175x200	Plane	Multicolour
Image24	5	5	VI	Bright	598x604	Plane	Multicolour
Image25	12	12	III, IV, V	Bright	1276x606	Nature	Multicolour
Image26	7	7	Black and White	Good	913x504	Plane	—
Image27	11	11	III, IV	Good	838x472	Plane	Multicolour
Image28	47	47	I, II, III	Good	5151x3434	Buildings	Uniform
Image29	7	7	III, IV, V, VI	Good	1200x514	Nature	Multicolour
Image30	14	14	I, II, III, IV, V	Good	1200x514	Plane	Multicolour
Image31	15	15	IV, V	Good	640x675	Colourful	Uniform
Image32	15	15	V, VI	Good	1500x675	Colourful	Uniform

Skin color based on Fitzpatrick skin type color representation [52]

D. PERFORMANCE UNDER ILLUMINATION CONDITIONS

The performance of the YOLOv9-based face detection method was studied under different illumination conditions to find its effectiveness. The impact of illumination was studied by using a set of images recorded under different natural lighting conditions ² as shown in Fig. 8. As the face detection algorithm gives good performance for brightly illuminated scenes, in this study, we mainly focused on finding the effectiveness of YOLOv9-based face detection in less illuminated scenes, where the faces are only partially visible. The images were extracted from the different frames of a video recorded while traveling in a moving bus during night lighting. The video was recorded at a frame rate of 30fps with face movements. We extracted one frame from every second of the video to apply the face detection model. Fig. 8 illustrates that the YOLOv9-based face detector gives exceptional face detection performance even in the case of poorly lit scenes and partly visible faces. The major observations from this study (in Fig. 8) are as follows:

- The YOLOv9-based face detection method gives good performance in image shades and different angles of illumination.
- The performance of the face detection method is not affected by the orientation of the face in the image. This is illustrated in Figs. 8 (d2), (d3), and (a4).
- Although the method gives good performance in varying illumination conditions, there is a chance of false positives for irregular illumination conditions, as shown in Fig. 8 (e5) where a region in the image was wrongly predicted as a face.
- In very dark scenes, the size of the bounding box doesn't completely cover the face of the image. For instance, in Figs. 8 (e4), (g4), (a5), and (b5), there is a significant difference in the size of the face bounding box, and only a part of the face is covered in Fig. 8 (b5).
- In an extremely motion-blurred case in Fig. 8 (b1), the face was not detected by the YOLOv9-based face detection method.

In addition, the YOLOv9-based face detection method considerably improved the face detection performance in dark scenes compared to traditional face detection methods such as

²The database used for the illumination study is available at <https://drive.google.com/drive/folders/1ZxkMFsRVbfRblbeDpqlCwG8A1uBUK2t?usp=sharing>

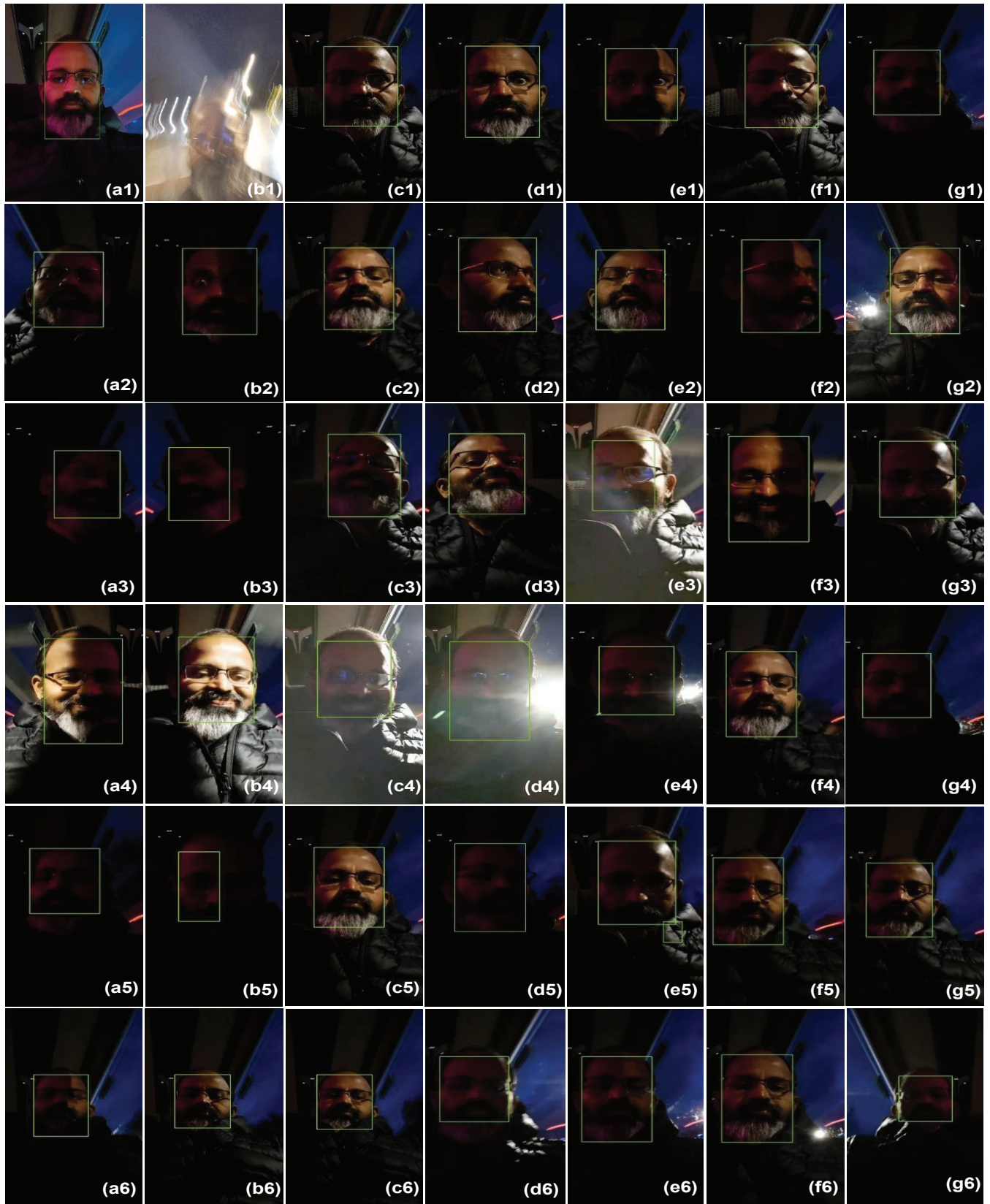


FIGURE 8. Detection performance under low-light, real-world conditions: Illustrates the performance of the YOLOv9-based face detection method on real images extracted from a video taken in a moving bus at night with different illumination. YOLOv9 detects faces across a wide range of lighting and occlusion conditions, showing resilience with minimal false positives and few missed detections, even under motion blur and backlighting.

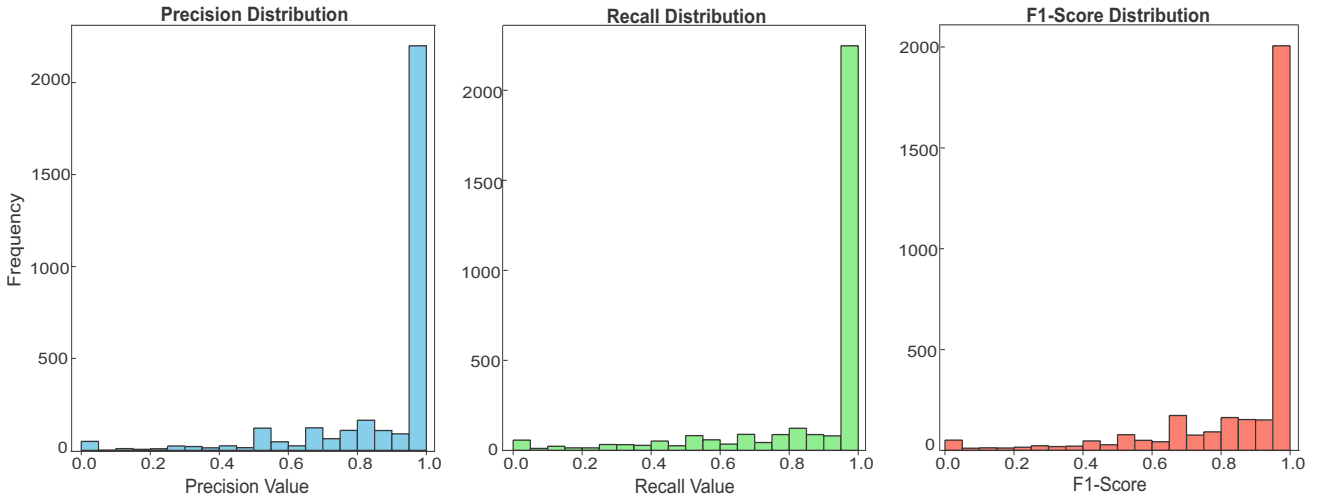


FIGURE 9. Distributions of precision, recall, and F1-score for the YOLOv9-based face detection model on the Wider Face database. The distribution of precision, recall, and F1-scores does not follow a uniform distribution. All three performance metrics show highly skewed distributions, with a sharp peak near 1.0, indicating that the model is highly accurate in detecting faces. Similarly, the F1-score distribution confirms strong overall performance with limited trade-off between precision and recall. However, the presence of a tail toward lower values reveals that performance significantly drops on a subset of more challenging cases.

MTCNN (27 correct detections out of 42 for MTCNN [54]; 40 correct predictions out of 42 for YOLOv9), which clearly showcases the effectiveness of the model.

E. STATISTICAL EVALUATION OF DETECTION ROBUSTNESS UNDER NOISE AND BLUR EFFECTS

We evaluate the robustness of a YOLOv9-based face detection and people counting method trained on the Wider Face dataset under varying levels of image distortions, such as image blur (average, disk, Gaussian, and motion blur effects with various kernel sizes and/or standard deviations) and image noises (Gaussian, salt and pepper, and speckle noise with varying noise density or noise variance). For each image, the detection performance is assessed using standard performance metrics, including precision, recall, F1-score, and mAP. The same set of images is evaluated under different degradation conditions. Identifying significant performance drops under different conditions helps in quantifying the robustness of face detection systems for model architecture tuning and deployment in adverse conditions.

1) Friedman Test: Non-Parametric Repeated Measures Analysis

A one-way repeated measures ANOVA is a statistical test used to compare the means of three or more related groups when the same subjects are measured under different conditions or at different time points [55]. It is an extension of the paired t-test for more than two related groups [55]. The repeated-measures ANOVA test can only be performed if the data are normally distributed. However, the precision obtained from the test images of the Wider Face dataset does not follow uniform distribution (as shown in Fig. 9). Hence, we use the Friedman test for statistical significance analysis.

The Friedman test is a non-parametric statistical test used

to detect differences in treatments across multiple conditions when the same subjects (here, images) are measured under each condition [55]. It serves as a non-parametric alternative to the repeated-measures ANOVA when data normality cannot be assumed. In our case, each image is a subject, and each distortion level (e.g., original, low noise, medium noise, high noise) is a condition.

Let N be the number of images, k be the number of conditions, and R_{ij} be the precision of i^{th} image under the j^{th} condition. The Friedman statistic χ_F^2 is computed as [55]:

$$\chi_F^2 = \left[\frac{12}{Nk(k+1)} \sum_{j=1}^k (R_j^2) \right] - 3N(k+1). \quad (5)$$

The degrees of freedom (df) is given by

$$df = k - 1. \quad (6)$$

The p-value is the probability that a Chi-squared distributed variable with df degrees of freedom takes on a value as extreme or more extreme than the computed statistic, which is given as [55]:

$$p = P(\chi^2 \geq \chi_F^2 | df). \quad (7)$$

The test evaluates the null hypothesis:

H_0 : The distribution of detection scores (precision) across different conditions is the same.

We set the level of risk α (or the level of significance) associated with the null hypothesis at 0.05, stating that there is a 95% chance that any observed statistical difference will be real and not due to chance. If χ_F^2 is large and the corresponding p-value is small (< 0.05), the null hypothesis is rejected, indicating a statistically significant difference in detection performance across conditions.

TABLE 4. Statistical analysis for different blurring and noise effects on the Wider Face database. The Friedman test reveals a statistically significant difference between conditions under varying noise and blurring effects.

Recording Condition		Friedman test statistic	p-Value	Post-hoc Test (Wilcoxon between pairs)
Noises	Gaussian Noise	1590.2913	0	All p-values < 0.05
	Salt and Pepper Noise	466.2128	0	All p-values < 0.05
	Speckle Noise	753.5258	0	All p-values < 0.05
	Average Blur	3378.8315	0	Original-Blur (k=10) : p-value = 0.2065 Blur (k=5)-Blur (k=10) : p-value = 0.7479
	Disk Blur	2281.9149	0	All p-values < 0.05
Variable kernel sizes, Uniform variance	Gaussian Blur ($\sigma = 1$)	515.7224	0	Blur (k=10)-Blur (k=20): p-value = 0.9719 Blur (k=10)-Blur (k=30): p-value = 0.9719 Blur (k=20)-Blur (k=30): p-value = 1.0000
	Gaussian Blur ($\sigma = 2$)	210.3481	0	Blur (k=10)-Blur (k=20): p-value = 0.6297 Blur (k=10)-Blur (k=30): p-value = 0.6292 Blur (k=20)-Blur (k=30): p-value = 0.9719
	Gaussian Blur ($\sigma = 4$)	17.8207	0.0005	Original-Blur (k=20): p-value = 0.0943 Blur (k=20)-Blur (k=30): p-value = 0.5488
	Gaussian Blur ($\sigma = 8$)	1106.3091	0	Original-Blur (k=10): p-value = 0.0699
Variable variance, Uniform kernel size	Gaussian Blur (kernel = 10)	118.7444	0	Original-Blur ($\sigma=8$): p-value = 0.2708 Blur ($\sigma=4$)-Blur ($\sigma=8$): p-value = 0.0933
	Gaussian Blur (kernel = 20)	474.9369	0	All p-values < 0.05
	Gaussian Blur (kernel = 30)	1497.8656	0	All p-values < 0.05
Variable angle, Uniform kernel size	Motion Blur (kernel = 10)	69.7866	0	Blur ($\theta=0$)-Blur ($\theta=15$): p-value = 0.3799 Blur ($\theta=0$)-Blur ($\theta=30$): p-value = 0.1622 Blur ($\theta=45$)-Blur ($\theta=60$): p-value = 0.9079
	Motion Blur (kernel = 20)	260.8565	0	Blur ($\theta=15$)-Blur ($\theta=30$): p-value = 0.1014 Blur ($\theta=30$)-Blur ($\theta=45$): p-value = 0.0546 Blur ($\theta=30$)-Blur ($\theta=60$): p-value = 0.5447 Blur ($\theta=45$)-Blur ($\theta=60$): p-value = 0.2742
	Motion Blur (kernel = 30)	2504.3545	0	Blur ($\theta=30$)-Blur ($\theta=60$): p-value = 0.7973
Variable kernel sizes, Uniform angle	Motion Blur ($\theta = 0^\circ$)	537.9317	0	All p-values < 0.05
	Motion Blur ($\theta = 15^\circ$)	797.1972	0	All p-values < 0.05
	Motion Blur ($\theta = 30^\circ$)	1468.4988	0	All p-values < 0.05
	Motion Blur ($\theta = 45^\circ$)	1692.8539	0	All p-values < 0.05
	Motion Blur ($\theta = 60^\circ$)	1503.7361	0	All p-values < 0.05

2) Post-hoc Pairwise Comparisons using Wilcoxon Signed-Rank Test

While the Friedman test tells us that some differences exist, it doesn't tell us which specific pairs of conditions differ [55]. For this, we use the Wilcoxon signed-rank test, which is a non-parametric test for paired samples [55].

Each comparison tests the null hypothesis: H_0 : The median difference in detection score (precision) between Condition A and Condition B is zero.

Given paired observations (x_i, y_i) , we compute the differences $(d_i = x_i - y_i)$ and rank the absolute differences $|d_i|$, excluding zeros. The Wilcoxon statistic is the sum of the ranks of the positive or negative differences. A small p-value indicates a significant difference between the two conditions.

To rigorously assess the impact of noise and blur on face detection performance, we employed the Friedman test followed by Wilcoxon signed-rank tests for post-hoc pairwise comparisons. The Friedman test was conducted on the 3221 images of the Wider Face database under different types of noises and blurring effects at various levels. For example, a single test was performed to evaluate the statistical significance across the precision of detections from original images and images corrupted with Gaussian noise at noise variance levels of 0.01, 0.05, and 0.1. Similar tests were

conducted for different noises and blurring effects. The statistical analysis for different blurring and noise effects on the Wider Face database is presented in Table 4. The Friedman test revealed statistically significant differences across different conditions. The Friedman test revealed a statistically significant difference in precision scores across varying distortion levels (eg: $\chi^2(3) = 1590.29$, $p < 0.001$ for different Gaussian noise levels and original images, $k = 4$, $df = 3$). All the p-values being ≈ 0 strongly indicate that the model is sensitive to image degradation, impacting detection accuracy. This confirms that even small amounts of noise significantly impact the model's performance, and the effect increases with distortion intensity.

To further explore how many differences exist and which conditions are different, we perform the post-hoc test. Post-hoc analysis confirmed significant performance degradation with increasing noise severity, with all pairwise Wilcoxon tests yielding < 0.05 . However, certain blur level experiments showed higher p-values, especially for uniform variance variable kernel size Gaussian blur tests and uniform kernel size variable angle motion blur tests. These findings underscore the sensitivity of the YOLOv9-based model to image distortions, highlighting the need for robustness-aware training or preprocessing strategies.

F. BLURRING EFFECTS ON YOLOV9-BASED FACE DETECTION MODEL

In this subsection, we study the performance of the YOLOv9-based face detection model, which is created by using the Wider Face database under different blurring effects. In real-life scenarios, the quality of the images obtained from sensors is degraded due to different blurring and noise effects. Blurring refers to the reduction of sharpness or clarity in an image. Blurring often occurs due to camera shake, the motion of the subject or the camera, shake due to internal vibrations, a dirty lens, or non-focus [56]. When face detection is used in mobile surveillance systems, particularly in crowd monitoring and tracking, blurred images can often be encountered due to the movement of objects (humans, animals, etc.) and/or movement of the camera (in drone/robot-based crowd monitoring). Hence, the study of the effect of blurring on the performance of the face detection and counting methods is necessary in all the major application scenarios. In this study, we analyze the impact of different blurring effects, such as average blur, circular average blur (disk blur), Gaussian blur, and motion blur, on face detection and people counting performance [56], [57]. The mathematical expressions for these blurring effects are given in Annexure A of the Supplementary Material.

1) Average Blur

The average blur technique smooths the image by averaging the pixel values within a defined kernel (neighborhood) [57]. The blur is uniform, making it look “boxy”. The average blur kernel is also called the box filter [57].

2) Disk Blur

As a camera projects a 3D scene to a 2D imaging plane, some areas of a scene are in focus while other areas go out of focus [56]. If the camera's aperture is circular, the image of any point source appears as a small disk. The degree of defocus depends on the focal length, aperture number of the lens, and the distance between the object and the camera [56]. Disk blur uses a circular averaging filter (pillbox) within the square kernel. The kernel size is characterized by a size of $2r + 1$, where r is the radius of the disk.

3) Gaussian Blur

The Gaussian blur or atmospheric turbulence blur is introduced by atmospheric turbulence, such as a change in temperature, exposure time, or wind speed [56]. The Gaussian blur is a rotationally symmetric lowpass filter of size h with standard deviation σ [31]. Pixels closer to the center of the kernel have a greater influence on the final value.

4) Motion Blur

The motion blur characterises the linear motion of a camera [56]. The motion blur is characterized by the length of the motion and the angle of motion, referring to translation and rotational motion.

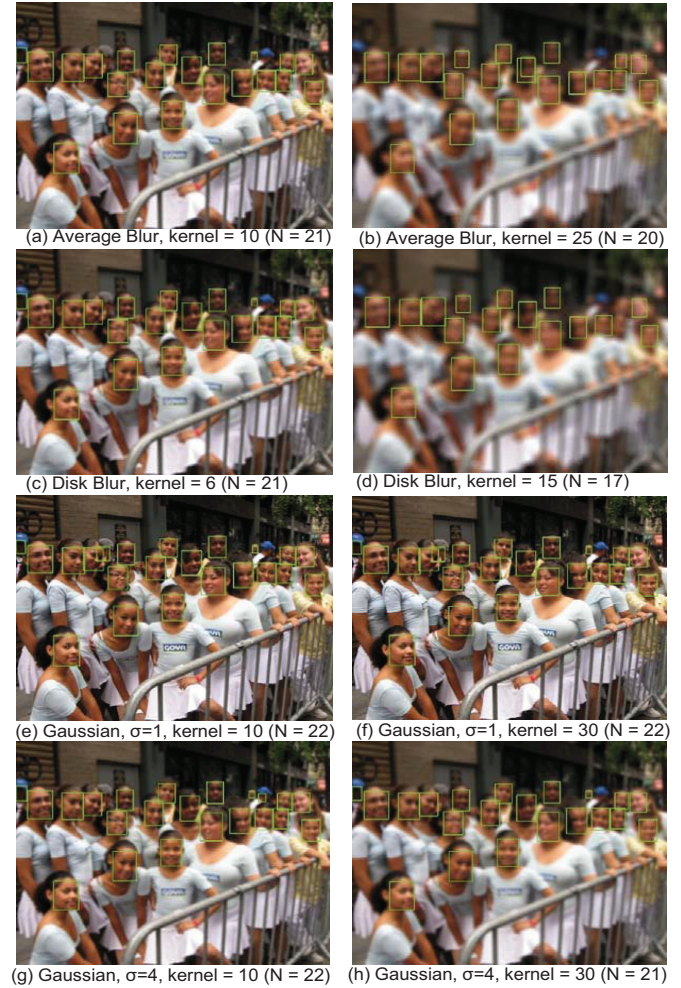


FIGURE 10. Illustrates the performance of the YOLOv9-based face detection and people counting method under different types of blurring effects: (a) Image after applying average blur with a kernel size of 10×10 (Faces detected: 21); (b) Image after applying average blur with a kernel size of 25×25 (Faces detected: 20); (c) Image after applying disk blur with kernel radius of 6 (Faces detected: 21); (d) Image after applying disk blur with kernel radius of 15 (Faces detected: 17); (e) Image after applying Gaussian blur with $\sigma = 1$ and a kernel size of 10×10 (Faces detected: 22); (f) Image after applying Gaussian blur with $\sigma = 1$ and a kernel size of 30×30 (Faces detected: 22); (g) Image after applying Gaussian blur with $\sigma = 4$ and a kernel size of 10×10 (Faces detected: 22); (h) Image after applying Gaussian blur with $\sigma = 4$ and a kernel size of 30×30 (Faces detected: 21). Original image is taken from the Wider Face database [49]. The figure shows that larger kernel sizes and higher σ values reduce face detection. However, different blurring effects show varying sensitivity. The figure implies that YOLOv9 handles mild to moderate blur well, but its accuracy declines with excessive smoothing, highlighting the importance of image sharpness for optimal detection.

We used *MATLAB fspecial* predefined kernels to generate blurred versions of the images of the Wider Face database. Fig. 10 and Fig. 11 illustrate the performance of the YOLOv9-based face detection and people counting methods for images with different levels of blurring effects. The performance of the method on different blur effects on the Wider Face database is summarized in Table 5 and specifically for various motion blur effects in Table 6 in terms of precision, recall, mAP, MAE, RMSE, and SD. The study

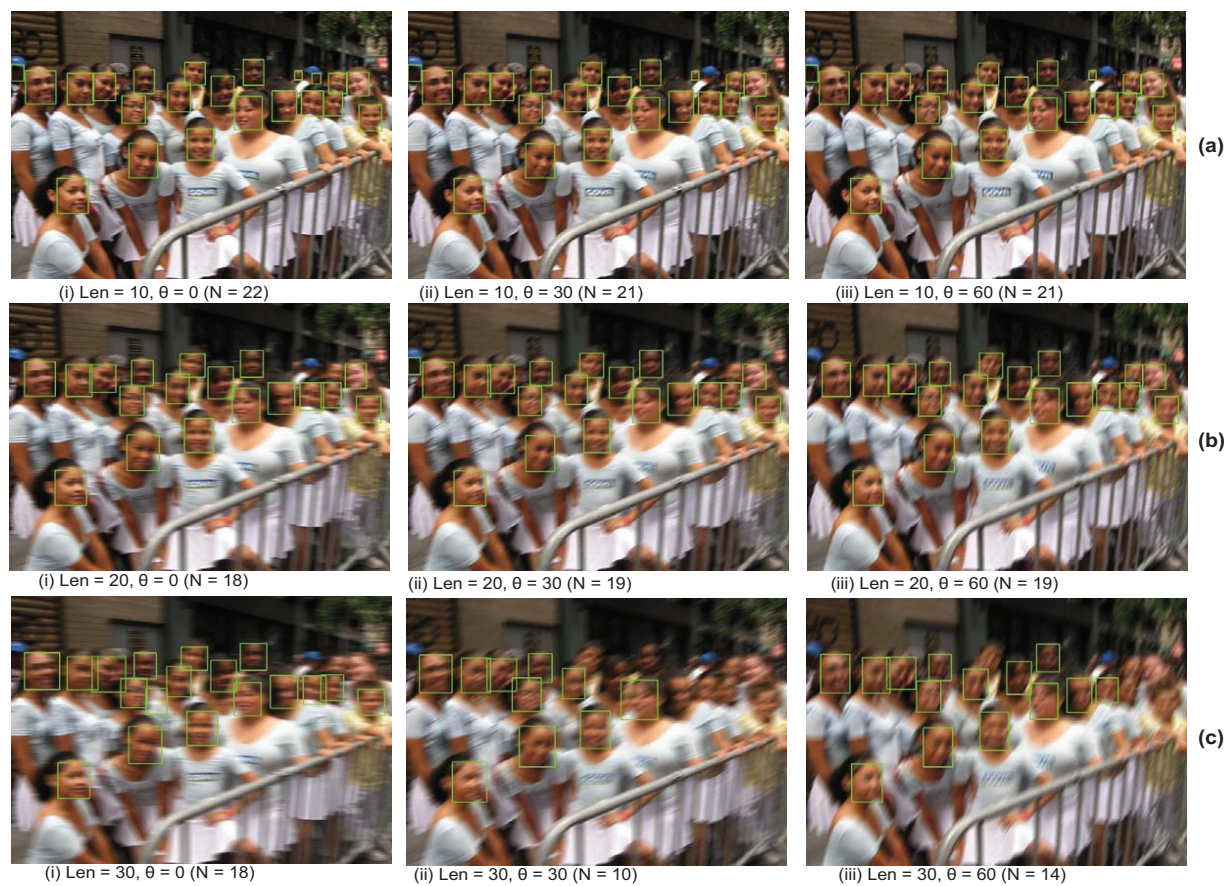


FIGURE 11. Illustrates the performance of the YOLOv9-based face detection and people counting method under different levels of motion blurring effects on an image from the Wider Face database [49]. The angle of camera motion and linear motion of the camera are indicated in each image with the number of faces detected in parentheses. We observe that the performance remains stable for short motion blur, drops gradually for moderate motion blurs, and is significantly impaired for severe motion blurs. YOLOv9's performance under camera motion is sensitive to blur intensity. The face detection degrades sharply with increasing blur lengths, regardless of direction.

reveals that different blurring effects significantly affect the performance of the YOLOv9-based face detection and people counting method. As the kernel size increases for each type of blurring effect, the performance deteriorates. The kernel size of the Gaussian blur has little impact on face detection and people counting performance compared to the standard deviation σ . In the case of motion blur, the angle of motion has a lesser effect on the performance compared to the linear motion of the camera. Fig. 12 shows the per-image loss distribution of the YOLOv9-based face detection model for various blur effects on the Wider Face database. With the addition of different blur effects, the loss distribution has shifted its shape with a skew towards the right as compared to the distribution of the original images in Fig. 5, showing that the error has increased.

G. NOISE-ROBUSTNESS STUDY OF YOLOV9 FACE DETECTION

In past studies, the performance of image processing and computer vision techniques was evaluated with different kinds of noisy images, such as images corrupted with salt and pepper or random noise, Poisson noise, Gaussian noise, and

speckle noise [29]–[32], [40], [41]. Similarly, in past studies, various kinds of image denoising/enhancement methods were presented for improving the performance of image classification and object detection approaches. In general, the accuracy and robustness of deep learning-based face detection and recognition methods should be evaluated using common additive and multiplicative noise sources. In this study, we evaluate the robustness of the YOLOv9-based people counting method in terms of different levels of common noise sources such as salt and pepper noise, Gaussian noise, and speckle noise [56], [57]. The mathematical expressions for the different types of noise are presented in Annexure B of the Supplementary Material.

1) Gaussian Noise

Additive white Gaussian noise (AWGN), commonly referred to as Gaussian noise, is a type of statistical noise characterized by a probability density function (PDF) that follows the normal distribution (Gaussian distribution) [56]. When applied to color images, Gaussian noise affects the RGB channels independently and adds a normally distributed random value to each pixel. Gaussian noise typically arises

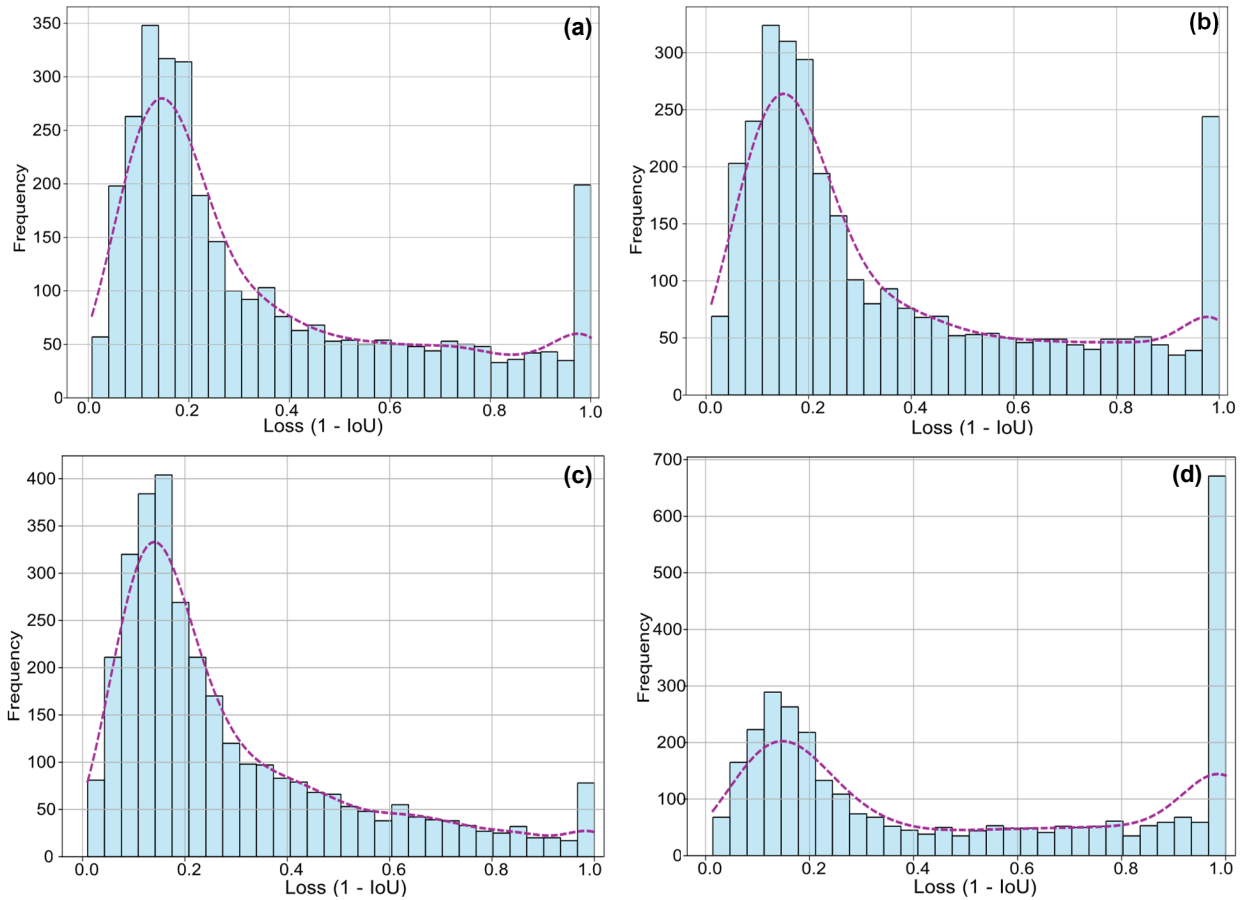


FIGURE 12. Per image loss distribution of the YOLOv9-based face detection model under various blur conditions applied on the Wider Face database: (a) Average blur of kernel size=10; (b) Disk blur of kernel size=6; (c) Gaussian blur of $\sigma = 2$ and kernel size=10; (d) Motion blur of length=20 and $\theta = 30^\circ$. In subplots (a–c), the loss distributions are skewed left, with a majority of images exhibiting low loss and a gradual tail toward higher loss, suggesting moderate resilience to average, disk, and Gaussian blurs. However, subplot (d) shows a significant increase in high-loss instances and a secondary peak near 1.0, indicating that motion blur severely degrades model performance. These trends imply that YOLOv9 is more robust to isotropic blur types than to directional motion blur.

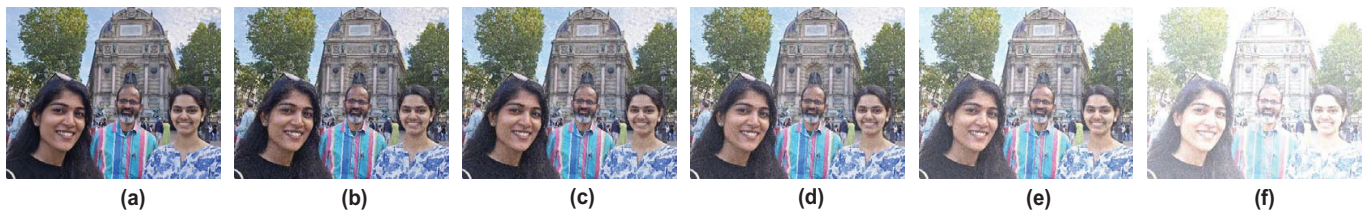


FIGURE 13. Effect of Gaussian noise: (a) Face image; (b)–(f) Image corrupted with Gaussian noise with variances (b) $\sigma = 0.02$; (c) $\sigma = 0.05$; (d) $\sigma = 0.1$; (e) $\sigma = 0.2$; (f) $\sigma = 0.5$. As the noise variance increases from 0.02 to 0.5, facial features become increasingly corrupted. Extreme Gaussian noise can destroy discriminative features critical for face detection, necessitating noise removal strategies in preprocessing.

due to sensor noise in the imaging system, electronic circuit noise, shot noise, amplifier noise, and photographic film grain noise [32], [56], [57]. Fig. 13 illustrates the effect of different levels of Gaussian noise (zero-mean, non-zero variance noise) on an image.

2) Salt and Pepper Noise

Salt and pepper noise, also referred to as impulse noise, is a random occurrence of black and white pixels in the image [56]. Salt and pepper noise is usually caused by sharp,

sudden disturbances in the image. The impulse noise is a very heavy-tailed noise [56]. In a color image, it appears as random white or black pixels, distributed sparsely throughout the image, affecting only a few pixels, similar to the effect of sprinkling white and black dots (salt and pepper) on the image [31], [56]. The noise ratio determines the number of pixels affected by this noise [31].

Salt and pepper noise is generally caused by errors in data transmission arising from faulty pixels in camera sensors, image transmission over noisy digital links, ADC errors or

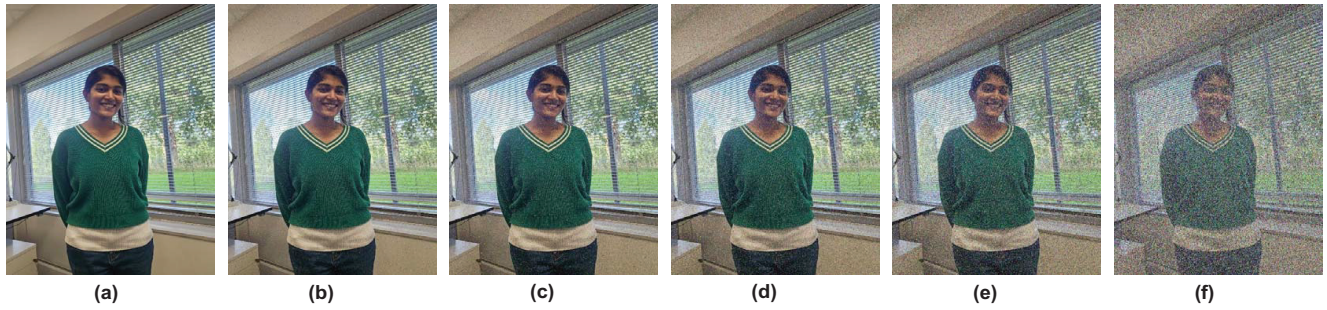


FIGURE 14. Effect of salt and pepper noise: (a) Face image; (b)-(f) Image corrupted with salt and pepper noise with noise densities (b) $d = 0.02$; (c) $d = 0.05$; (d) $d = 0.1$; (e) $d = 0.2$; (f) $d = 0.5$. As the noise density increases from 0.02 to 0.5, facial features become increasingly corrupted. Fine facial details are lost rapidly beyond $d = 0.2$, indicating potential for detection failure. Extreme Gaussian noise can destroy discriminative features critical for face detection, necessitating noise removal strategies in preprocessing.

TABLE 5. Performance of YOLOv9-based face detection and people counting model on different blurring effects on the Wider Face database. Results demonstrate a degradation in the model's performance with increased blurring while maintaining performance at smaller blurring levels.

Blur Type	Kernel Size	Face Detection				Counting	
		Precision	Recall	mAP50	mAP50-95	MAE $\pm$ SD	RMSE
Average	—	0.860	0.628	0.708	0.368	3.36 $\pm$ 22.20	22.38
	5	0.843	0.576	0.665	0.347	4.86 $\pm$ 28.09	28.44
	10	0.810	0.414	0.515	0.275	7.56 $\pm$ 37.34	38.05
	15	0.807	0.283	0.397	0.203	9.17 $\pm$ 39.37	40.40
	20	0.836	0.194	0.338	0.171	10.29 $\pm$ 39.81	41.10
	25	0.334	0.204	0.304	0.151	11.05 $\pm$ 39.94	41.43
Disk	30	0.388	0.148	0.289	0.144	11.46 $\pm$ 39.96	41.56
	3	0.843	0.563	0.653	0.342	5.11 $\pm$ 29.18	29.57
	6	0.805	0.386	0.485	0.254	7.92 $\pm$ 38.13	38.91
	9	0.819	0.256	0.378	0.196	9.56 $\pm$ 39.59	40.70
	12	0.306	0.252	0.325	0.166	10.66 $\pm$ 39.89	41.27
Gaussian $\sigma = 1$	15	0.380	0.174	0.303	0.153	11.31 $\pm$ 39.96	41.52
	10	0.869	0.633	0.721	0.397	3.89 $\pm$ 24.16	24.40
	20	0.869	0.633	0.721	0.397	3.89 $\pm$ 24.16	24.40
	30	0.869	0.633	0.721	0.397	3.89 $\pm$ 24.16	24.40
Gaussian $\sigma = 2$	10	0.846	0.557	0.649	0.357	5.54 $\pm$ 30.86	31.30
	20	0.842	0.556	0.648	0.356	5.58 $\pm$ 30.88	31.33
	30	0.842	0.556	0.648	0.356	5.58 $\pm$ 30.88	31.32
Gaussian $\sigma = 4$	10	0.819	0.450	0.548	0.297	7.14 $\pm$ 36.21	36.86
	20	0.812	0.354	0.464	0.253	8.40 $\pm$ 36.21	39.61
	30	0.815	0.353	0.463	0.253	8.43 $\pm$ 38.75	39.62
Gaussian $\sigma = 8$	10	0.814	0.422	0.523	0.280	7.48 $\pm$ 37.23	37.93
	20	0.844	0.211	0.359	0.191	10.10 $\pm$ 39.77	41.01
	30	0.398	0.206	0.335	0.185	11.06 $\pm$ 39.94	41.42

ADC saturation, and failure in memory cells, which results in black pixels in white regions and white pixels in black regions [56], [57]. Fig. 14 shows the effect of different levels of salt and pepper noise on an image.

3) Speckle Noise

Speckle noise is a type of multiplicative noise, i.e., the noise is proportional to the local pixel value [56]. Coherent imaging modalities, such as active radars and ultrasonic imaging, create images by combining echoes from the heterogeneities of objects and are affected by speckle noise. [56]. The interference of acoustic echoes with random phases and amplitudes creates a complex pattern known as speckle noise,

TABLE 6. Performance of YOLOv9-based face detection and people counting model on motion blur effects on the Wider Face database. Results demonstrate a degradation in the model's performance with increased blurring levels while maintaining the performance at smaller blurring levels.

len	θ	Face Detection				Counting	
		Precision	Recall	mAP50	mAP50-95	MAE $\pm$ SD	RMSE
—	0	0.860	0.628	0.708	0.368	3.36 $\pm$ 22.20	22.38
	0	0.830	0.504	0.596	0.315	6.13 $\pm$ 31.78	32.32
	15	0.829	0.500	0.590	0.313	6.12 $\pm$ 31.58	32.12
	30	0.826	0.477	0.569	0.305	6.63 $\pm$ 33.85	34.45
	45	0.829	0.461	0.560	0.300	6.95 $\pm$ 34.94	35.58
	60	0.827	0.458	0.560	0.296	7.11 $\pm$ 35.24	35.92
20	0	0.818	0.289	0.389	0.206	9.10 $\pm$ 39.11	40.13
	15	0.814	0.269	0.368	0.197	9.28 $\pm$ 39.41	40.46
	30	0.844	0.227	0.343	0.189	9.84 $\pm$ 39.72	40.90
	45	0.856	0.206	0.340	0.190	10.09 $\pm$ 39.80	41.03
	60	0.853	0.218	0.356	0.194	9.95 $\pm$ 39.73	40.93
30	0	0.823	0.170	0.289	0.149	10.46 $\pm$ 39.90	41.23
	15	0.829	0.141	0.268	0.141	10.77 $\pm$ 39.94	41.35
	30	0.304	0.141	0.250	0.139	11.24 $\pm$ 40.00	41.53
	45	0.307	0.132	0.244	0.138	11.35 $\pm$ 39.97	41.54
	60	0.310	0.150	0.255	0.139	11.24 $\pm$ 39.95	41.48

which ranges from zero to a maximum based on whether the interference is destructive or constructive. Medical ultrasound imaging is prone to speckle noise, which lowers contrast and picture quality, thereby reducing its diagnostic value. In digital images, speckle noise is produced by a broken image sensor, lack of lighting or overheating of the device during image capture, or interference of the transmission channel [56]. Fig. 15 shows the effect of different levels of speckle noise on an image from the Wider Face database.

Fig. 16 shows the per-image loss distribution of the YOLOv9-based face detection model for various noise on the Wider Face database. With the addition of noise, the loss distribution has shifted its shape with a skew towards the right as compared to the distribution of the original images in Fig. 5, showing that the error has increased. Fig. 17 illustrates the performance of the YOLOv9-based face detection and people counting method under the three different noises for two noise levels, 0.1 and 0.2. The figure depicts the loss of important information and edges of the image as more noise is added,

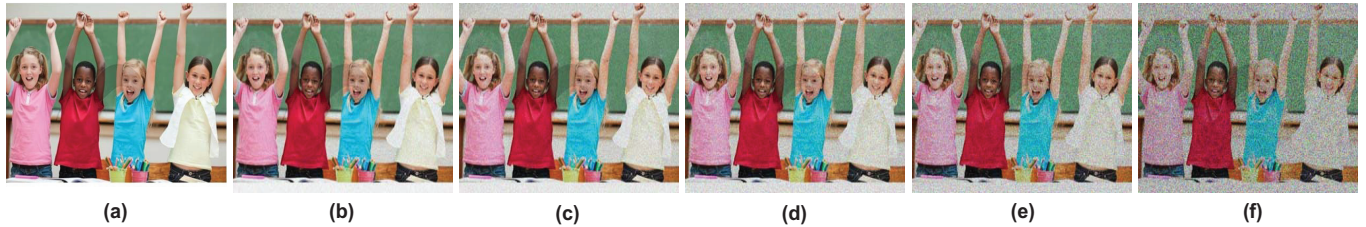


FIGURE 15. Effect of speckle noise: (a) Face image from Wider Face database [49]; (b)-(f) Image corrupted with speckle noise with variances (b) $\sigma = 0.02$; (c) $\sigma = 0.05$; (d) $\sigma = 0.1$; (e) $\sigma = 0.2$; (f) $\sigma = 0.5$. Although mild speckle noise ($\sigma \leq 0.1$) has limited visual impact, high variance ($\sigma = 0.5$) significantly distorts facial regions. YOLOv9 is robust to low-to-moderate speckle noise but may struggle under extreme levels. This implies that speckle noise is less critical than Gaussian or salt and pepper noise.

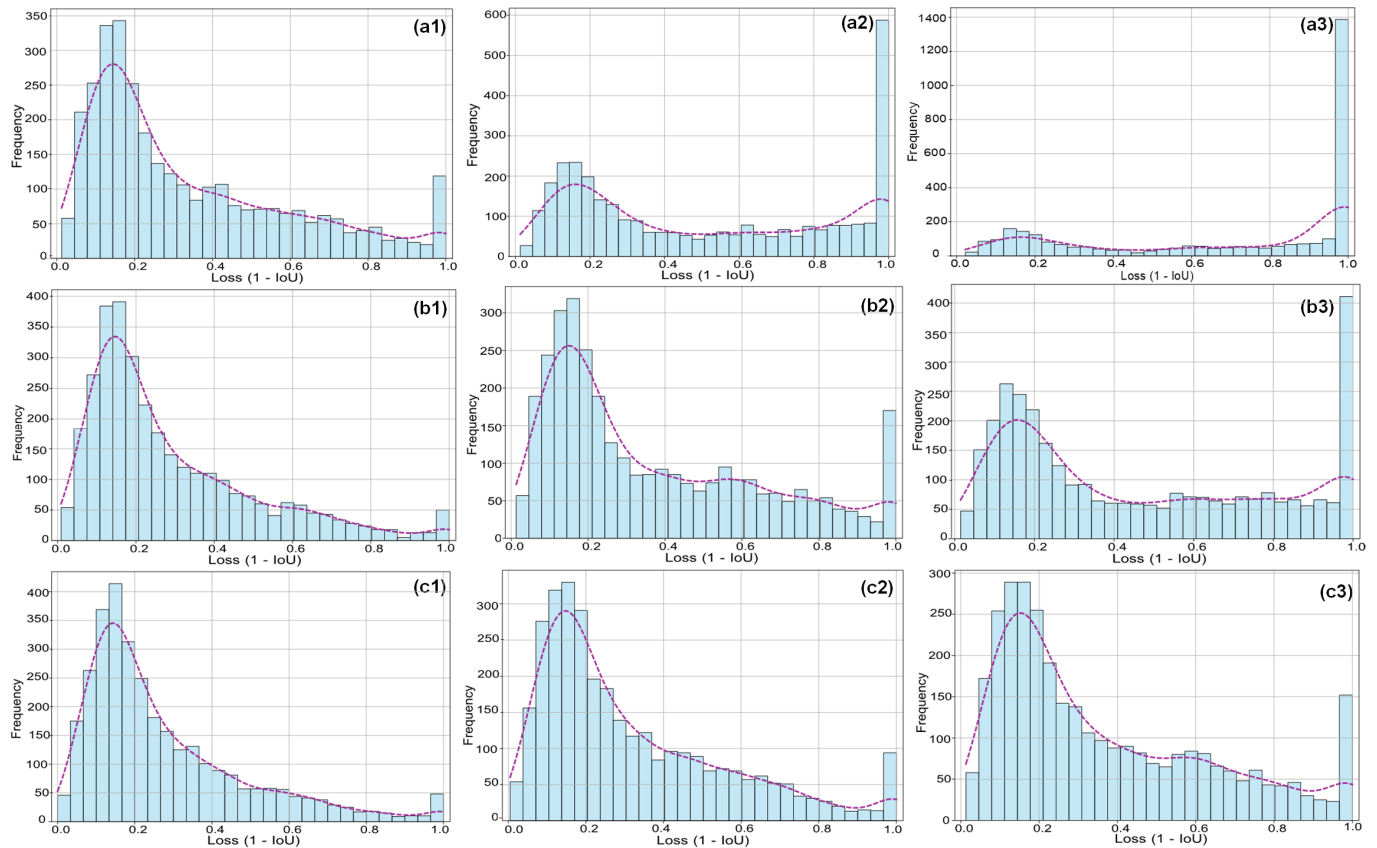


FIGURE 16. Histogram-based analysis of per-image loss distribution (1 - IoU) for the YOLOv9 face detection model under varying types and intensities of image noise on the Wider Face dataset. (a1–a3) show the impact of Gaussian noise with variances 0.01, 0.05, and 0.1 respectively; (b1–b3) represent salt and pepper noise with densities 0.01, 0.05, and 0.1 respectively; (c1–c3) show speckle noise with variances 0.01, 0.05, and 0.1 respectively. A consistent trend across all noise types shows that increasing noise severity shifts the loss distribution toward higher values and increases the frequency of near-maximum loss, especially for higher Gaussian and salt-and-pepper noise levels.

which affects the performance of the face detection method. To further study the robustness of the model, we added 3 different levels of salt and pepper, Gaussian, and speckle noises to the Wider Face database and further evaluated the face detection and counting methods for specific noises. The performance of the face detection and people counting methods for different noise levels is summarized in Table 7 and Table 8, respectively. The key observations from the results are as follows:

- **Observation 01:** The YOLOv9-based face detection and people counting method is not robust to very

high levels of noise. The mAP of the face detection method is reduced from 0.708 in noise-free conditions to 0.595, 0.679, and 0.698 when the noise variance level is 0.01 for salt and pepper noise, Gaussian noise, and speckle noise, respectively. In the case of people counting, the MAE values are increased from 3.36 (with MAE reference obtained for noise-free images) to 4.93, 3.65, and 3.45 when the noise variance level is 0.01 for Gaussian noise, salt and pepper noise, and speckle noise, respectively. Further, the results showed that the performance of the YOLOv9-based model is degraded

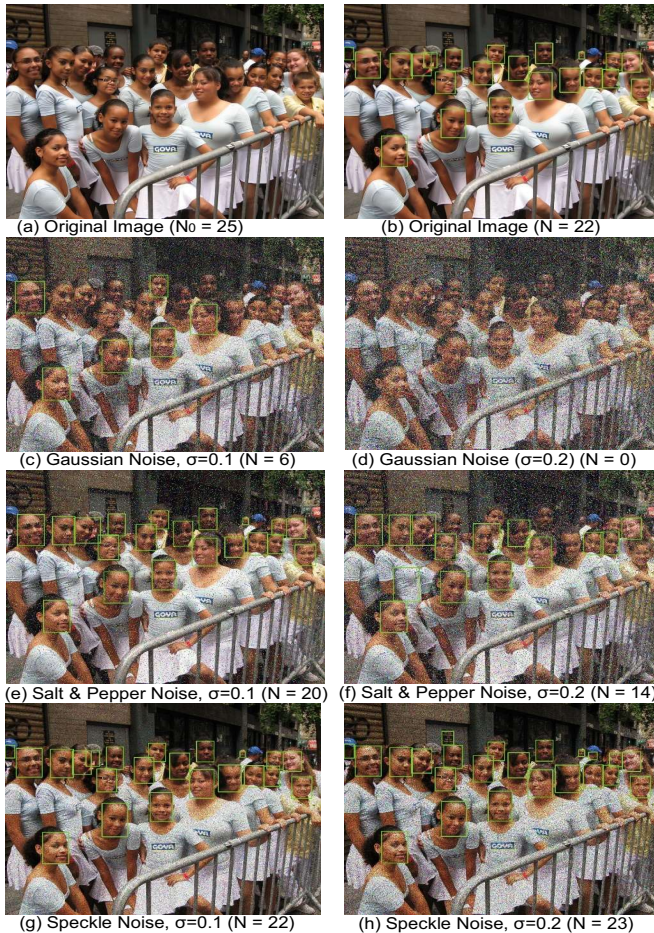


FIGURE 17. Illustrates the performance of the YOLOv9-based face detection and people counting method under different types of noises: (a) Original Image with 25 faces, taken from the Wider Face database [49]; (b) Face detection by the YOLOv9 model to detect 22 faces from the original image; (c) Image after the addition of Gaussian noise of variance of 0.1 (Faces detected: 6); (d) Image after the addition of Gaussian noise of variance of 0.2 (No face detected); (e) Image after the addition of salt and pepper noise of noise density of 0.1 (Faces detected: 20); (f) Image after the addition of salt and pepper noise of noise density of 0.2 (Faces detected: 14); (g) Image after the addition of speckle noise of variance of 0.1 (Faces detected: 22); (h) Image after the addition of speckle noise of variance of 0.2 (Faces detected: 23). The trend suggests that YOLOv9 is highly sensitive to Gaussian noise and moderately robust to impulse-based noise like salt and pepper. These findings imply that for practical deployment in noisy environments, noise-reduction preprocessing strategies may be necessary.

drastically when the noise level is increased, as shown in Table 8. This highlights the importance of image denoising to improve the robustness of the face detection method when the model is not created by using images corrupted with various kinds of real-world noises, which are unavoidable.

- **Observation 02:** From the results as shown in Fig. 17, it can be observed that the performance of the YOLOv9-based face counting method was improved for certain images under low noise levels unlike the usual performance degradation that can be seen in most of the methods (performance decreases when noise level

TABLE 7. Performance of YOLOv9-based face detection model on different noise levels of Wider Face database. Results demonstrate a degradation in the model's performance with increasing noise levels, while maintaining the performance at smaller noise levels.

Noise Type	Noise Level	Precision	Recall	mAP50	mAP50-95
Gaussian	0	0.860	0.628	0.708	0.368
	0.01	0.805	0.516	0.595	0.297
	0.05	0.679	0.288	0.336	0.165
	0.1	0.663	0.131	0.150	0.075
Salt and Pepper	0.01	0.850	0.597	0.679	0.348
	0.05	0.804	0.484	0.563	0.281
	0.1	0.748	0.363	0.422	0.208
Speckle	0.01	0.856	0.614	0.698	0.358
	0.05	0.814	0.561	0.636	0.317
	0.1	0.784	0.498	0.571	0.277

TABLE 8. Performance of YOLOv9-based people counting method on different noise levels of Wider Face database. Results demonstrate a degradation in the model's performance with increasing noise levels while maintaining the performance at smaller noise levels.

Noise Type	Noise Level	MAE $\pm$ SD	RMSE
Gaussian	—	3.36 $\pm$ 22.20	22.38
	0.01	4.93 $\pm$ 24.94	25.32
	0.05	8.86 $\pm$ 35.64	36.61
	0.1	10.91 $\pm$ 39.64	41.08
Salt and Pepper	0.01	3.65 $\pm$ 22.78	23.00
	0.05	5.54 $\pm$ 26.35	26.87
	0.1	7.69 $\pm$ 31.85	32.70
Speckle	0.01	3.45 $\pm$ 22.35	22.54
	0.05	4.17 $\pm$ 23.38	23.65
	0.1	5.24 $\pm$ 25.67	26.06

increases). Further, evaluation results demonstrate that better convergence and feature extraction optimization can be achieved with low levels of noise. Our evaluation results showed that the estimation of noise level for a specific type of noise in an image can improve accuracy even without applying the denoising process for the images with lower noise levels, in addition to the fast convergence of the model. However, the performance of the model is degraded at higher levels of noise. Further, evaluation results demonstrate that noise-specific denoising with suitable tunable parameters can improve the performance of the model if an automatic image quality assessment and image noise identification are incorporated as a preprocessing step.

- **Observation 03:** Although the model was created with the standard Wider Face database, real-world images may be corrupted with different kinds of noises. Therefore, the robustness of the YOLOv9-based model needs to be investigated for each type of noise. From the evaluation results as summarized in Table 7, it can be noticed that the performance degradation was not uniform for all the types of noises. For the Gaussian noise, a large MAE variation is observed when the noise variance increases, whereas a small MAE variation is observed for the speckle noise for the same order noise variance. The effect of noise types (Gaussian, speckle,



FIGURE 18. Illustrates the effect of Gaussian filtering for filtering images corrupted with Gaussian noise. (a1) Image corrupted with Gaussian noise of variance $\sigma = 0.1$; (a2) Image in (a1) after application of Gaussian filter with variance $\sigma = 1$; (a3) Image in (a1) after application of Gaussian filter with variance $\sigma = 5$; (b1) Image corrupted with Gaussian noise of variance $\sigma = 0.2$; (b2) Image in (b1) after application of Gaussian filter with variance $\sigma = 1$; (b3) Image in (b1) after application of Gaussian filter with variance $\sigma = 5$. Results show progressive noise reduction with increasing filter variance. Optimal filter parameters play a major role in preserving the image content.

and salt and pepper) in face boundary detection is shown in Fig. 17 for two different noise levels (in terms of variance and density). The results demonstrate that the performance of the deep model highly depends on the type of noise and its level.

Although the model's performance is degraded under different kinds of noises and levels of noise, this study further investigates the performance of the YOLOv9-based face detection model by combining it with traditional denoising techniques that are discussed in the following subsection.

H. EFFECT OF DENOISING ON FACE DETECTION

The performance of object detection methods is significantly affected by the presence of noise and artifacts, which is unavoidable in the different stages of image acquisition due to imaging systems, motion artifacts, etc. In past studies, noise and artifacts were removed by various filtering and image enhancement techniques. Image enhancement refers to the process of smoothing the irregularities or noise that has corrupted the image, with minimal modification to the original image information [56]. While most image denoising and enhancement techniques are specific to the type of noise, image details, and application, many deep learning-based image enhancement techniques were proposed that are capable of enhancing some of the artifacts, such as illumination variations and motion blur [58], [59]. In this paper, we study the effect of the Gaussian and median filters to improve the performance of face detection and counting methods under the presence of various noises.

1) Gaussian Filter

A Gaussian filter is a linear filter used to blur images and reduce noise, without ringing and noise leakage artifacts [56]. It works by convolving the image with a Gaussian function,



FIGURE 19. Illustrates the effect of median filtering for filtering images corrupted with salt and pepper noise. (a1) Image corrupted with salt and pepper noise of noise density $d = 0.1$; (a2) Image in (a1) after application of median filter of kernel size 3×3 ; (a3) Image in (a1) after application of median filter with kernel size 11×11 ; (b1) Image corrupted with salt and pepper noise of noise density $d = 0.5$; (b2) Image in (b1) after application of median filter with kernel size 3×3 ; (b3) Image in (b1) after application of median filter with kernel size 11×11 . Results show that median filtering effectively reduces salt and pepper noise. Optimal filter parameters play a major role in preserving the image content.

which has a bell-shaped curve described by the impulse response ($h(m, n)$) [56]

$$h(m, n) = \frac{1}{2\pi\sigma^2} e^{-\frac{m^2+n^2}{2\sigma^2}}, \quad (8)$$

where σ is the standard deviation of the Gaussian distribution, which determines the spread and thus the amount of blurring. In color images, the Gaussian filter is applied independently to each of the RGB channels. The filter smooths the image by averaging pixel values with nearby pixels, weighted by the Gaussian function, resulting in a reduction of high-frequency noise and minor details. Fig. 18 illustrates the effect of the Gaussian filter for filtering the images corrupted with Gaussian noise. Fig. 18 shows the same face image with two levels of Gaussian noise filtered with two different Gaussian filters. In Fig. 18 (a) with a Gaussian noise variance of 0.1, the Gaussian filter with variance 1 was able to remove the noise component from the image. On the other hand, in Fig. 18 (b) with a Gaussian noise variance of 0.2, only a part of the noise was filtered by the Gaussian filter of variance of 1. Although the application of the filter blurred the image, the Gaussian filter of variance 5 in Fig. 18 (b3) was able to better filter out the noise components compared to (b2). In the case of (a3), the application of the filter with variance 5 blurred the image and cleared important information compared to (a2), which is unnecessary. This highlights the importance of tuning the optimal filter parameters based on noise estimation to achieve

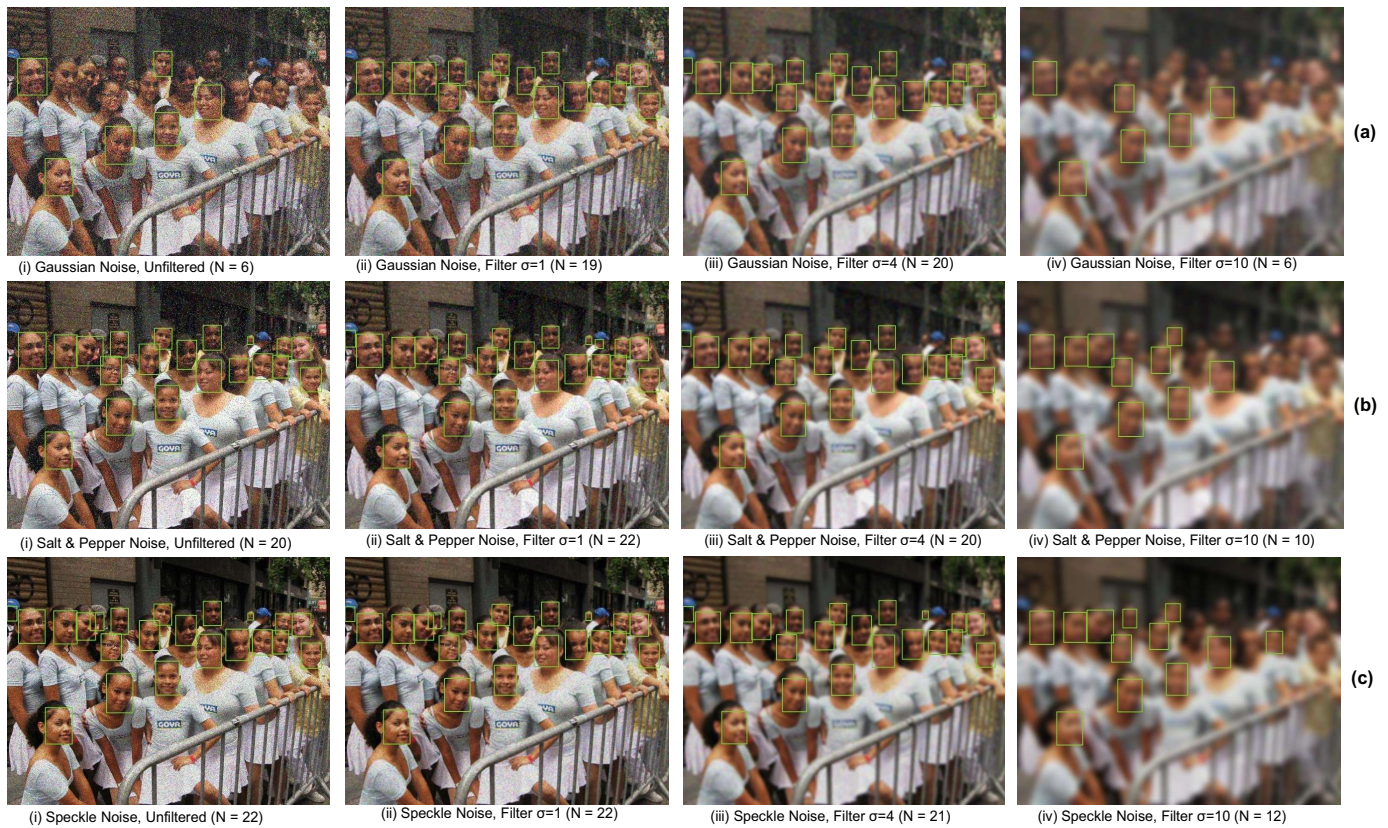


FIGURE 20. Illustrates the effect of different levels of Gaussian filtering ($\sigma = 1, 4, 10$) to improve the face detection performance and reduce the effect of (a) Gaussian noise of variance of 0.1; (b) Salt and pepper noise of noise density of 0.1; and (c) Speckle noise of variance 0.1. The number of faces detected in each case is given in parentheses. The original image is taken from the Wider Face database [49]. Results show that moderate filtering significantly enhances performance under Gaussian and salt and pepper noise. Filtering is a viable denoising method to enhance YOLOv9 face detection, but the proper choice of filter parameters is important to avoid oversmoothing.

a better denoising effect, which is a highly challenging task in most scenarios.

2) Median Filter

A median filter is a non-linear filter used primarily to eliminate salt and pepper noise [56]. It works by sliding a window over the image and swapping out the window's center pixel for the median value of the pixels within the window. The median filter effectively preserves the edges in the image while eliminating noise because the median value is less sensitive to outliers compared to the mean [56]. This makes it suitable for enhancing images corrupted with impulse noise while maintaining structural details.

Fig. 19 illustrates the effect of the median filter for denoising images corrupted with salt and pepper noise. As in the case of the Gaussian filter, the selection of optimal kernel size for the median filter is essential to obtain the best denoising performance. Fig. 19 (a1) shows an image with salt and pepper noise of noise density 0.1. For this noise level, the $3 \times 3 \times 3$ median kernel gives the best denoising compared to the $11 \times 11 \times 3$ median kernel, which blurs the image. However, in Fig. 19 (b), the salt and pepper noise of noise density 0.5 is better filtered by the $11 \times 11 \times 3$ median kernel

compared to the $3 \times 3 \times 3$ median kernel, underlining the need for noise estimation before denoising.

Fig. 20 illustrates the effect of different levels of Gaussian filter on a test image from the Wider Face database, corrupted with a 0.1 variance level of Gaussian noise, 0.1 noise density of salt and pepper noise, and 0.1 variance level of speckle noise. From Fig. 20, the visual appearance of the image and performance of the face detection method improved with low levels of Gaussian filtering. However, with higher filtering kernels, the image is blurred, and many faces are undetected. Interestingly, in the case of Gaussian noise with $\sigma = 10$, the image characteristics are lost due to the high variance of the smoothing filter. In spite of the excess smoothing, the YOLOv9-based face detection method was able to detect 6 faces, which is even difficult for the naked human eyes, highlighting the effectiveness and efficiency of the method. Fig. 21 compares the effect of the same type of Gaussian and Median filters to improve the face detection performance of the image corrupted by salt and pepper noise, Gaussian noise, and speckle noise. The figure illustrates that the face detection performance of the model was improved with the application of both types of filters. However, the performance and image aesthetics were much better with the application of



FIGURE 21. Compares the effect of Gaussian filter ($\sigma = 2$) and median filter (kernel size: 3-by-3-by-3) to improve the face detection performance and reduce the effect of (a) Gaussian noise of variance of 0.1; (b) Salt and pepper noise of noise density of 0.1; and (c) Speckle noise of variance 0.1. The number of faces detected in each case is given in parentheses. The original image is taken from the Wider Face database [49]. The results reveal that both filtering methods significantly improve detection accuracy compared to unfiltered noisy images. Median filtering performs better for salt and pepper noise, while Gaussian filtering is more effective for Gaussian noise. Both methods show comparable improvements for speckle noise, demonstrating the effectiveness of noise-specific filtering in enhancing face detection robustness.

the median filter for salt and pepper noise, and the Gaussian filter for Gaussian and speckle noise, which suggests the need for noise-specific denoising approaches for improving the performance of the face detection method.

The variations of face count with respect to different variances of the Gaussian filter are presented in Figs. 22 for 3 different images and five different noise levels of salt and pepper, Gaussian, and speckle noises. The face counts for the same 3 images are presented in Figs. 23 for the five different noise levels of salt and pepper, Gaussian, and speckle noises, with and without the Median filter. From the plots, the key observations are as follows.

- **Observation 01:** As observed from Fig. 22, the face count obtained from the YOLOv9-based face detection and counting model follows a bell curve with the increasing Gaussian filter parameters. The people counting performance is improved with the increasing variance of the Gaussian filter, reaches a maximum, and starts decreasing.
- **Observation 02:** A small level of noise ($\sigma = 0.01$) is shown to improve the performance of the face detection

method in most cases, as shown in Fig. 22.

- **Observation 03:** Although higher levels of smoothing invariably reduce the performance, results as shown in Fig. 22 suggest that the filter variance that gives the best performance is different for each noise level. It is also different for each image.
- **Observation 04:** The performance of the YOLOv9-based face counting model is mainly affected by the Gaussian noise as compared to the salt and pepper and speckle noises. In fact, even at higher speckle noise levels, the performance of the face detection method is almost unchanged, as observed from Fig. 22 (a3), (b3), and (c3).
- **Observation 05:** As shown in Figs. 23, the application of the median filter improved the face counting performance for all noise levels and all noise types for the 3 images. Especially in the case of salt and pepper noise, the median filter significantly boosts the performance, which can be observed by comparing Fig. 22 and Fig. 23.

Based on these observations, we conclude that the performance of the face detection model is mainly affected by

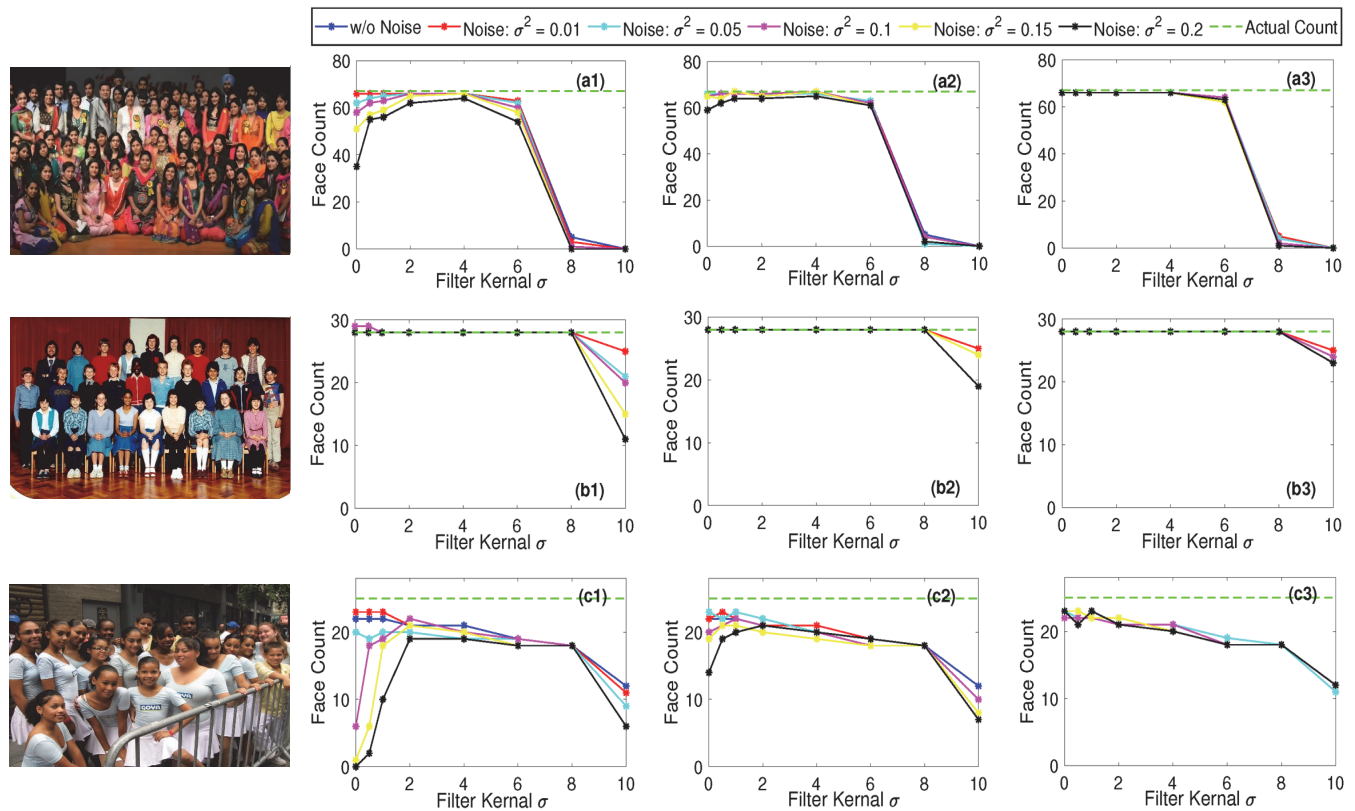


FIGURE 22. Illustrates the impact of Gaussian filtering (with varying kernel σ) on face count under noise levels such as 0, 0.01, 0.05, 0.1, 0.15, and 0.2 for 3 different images. Each row consists of the original image and plots for (1) Gaussian noise, (2) salt and pepper noise, and (3) speckle noise, respectively. Across all conditions, face detection performance improves with low σ values ($\sigma \leq 2$), but declines sharply at high σ ($\sigma \geq 8$), where over-smoothing leads to feature loss. The plots reveal a sweet spot around $\sigma = 2-4$, particularly under moderate noise, where detection remains close to the actual face count. This indicates that moderate Gaussian filtering can effectively suppress noise without removing essential facial features, enhancing model robustness across varying noise levels.

Image Sources: (a) <https://www.youtube.com/watch?v=cbRQgGB0i0A> (b) https://www.livingarchive.org.uk/content/catalogue_item/holne-chase-school-collection/holne-chase-school-photographs/middle-school-class-photograph-1991 (c) Wider Face Database [49].

the Gaussian noise and salt and pepper noise. Although both the Gaussian and median filters significantly improve the face detection performance for all types of noise, the image quality and performance are significantly boosted by the Gaussian filter in the case of Gaussian noise and the median filter in the case of salt and pepper noise, as seen in Fig. 22 and Fig. 23. In addition, the choice of a proper filter variance also significantly affects the performance. Although this is the general trend for all images, it is also affected by the type and properties of the image under consideration. For example, in the case of the 3 images under consideration in Fig. 22, image (b) seems to have little effect of varying noise levels and filter kernels, and gave consistently good performance. On further investigation, we found that image (b) has better resolution compared to the other 2 images (Resolutions 1280×720 , 2000×1211 , and 1024×768 respectively). With decreasing resolutions, the people counting performance of image 2 approaches the bell curve. The results suggest the importance of noise-specific denoising or image enhancement techniques with optimum filter parameters to give the best performance for each image. This involves finding the type and magnitude of noise present in each image before processing and applying

the optimum denoising filter based on the properties of the image. Thus, the assessment of the image quality is most significant for improving the performance of the face/object detection model.

In addition to the noise robustness study, we also evaluate the YOLOv9 face detection and people counting model for more generalized applications in open outdoor or uncontrolled environments in the following subsections.

I. PERFORMANCE OF YOLOV9 FACE COUNTING UNDER MIX OF HUMAN FACES AND ANIMAL FACES

The face detection system can often encounter animals, including pets, in generalized face detection and human counting scenarios, such as crowd counting and surveillance in public places, including parks and zoos. In such cases, the face detection method should be capable of differentiating human faces from a mix of human faces and live animal faces. Some animals, in particular, such as dogs and monkeys, may have extremely similar facial features to humans. The facial similarity of humans with chimpanzees and monkeys can be partially explained by the common ape ancestor that humans shared with monkeys and chimpanzees in the evolutionary

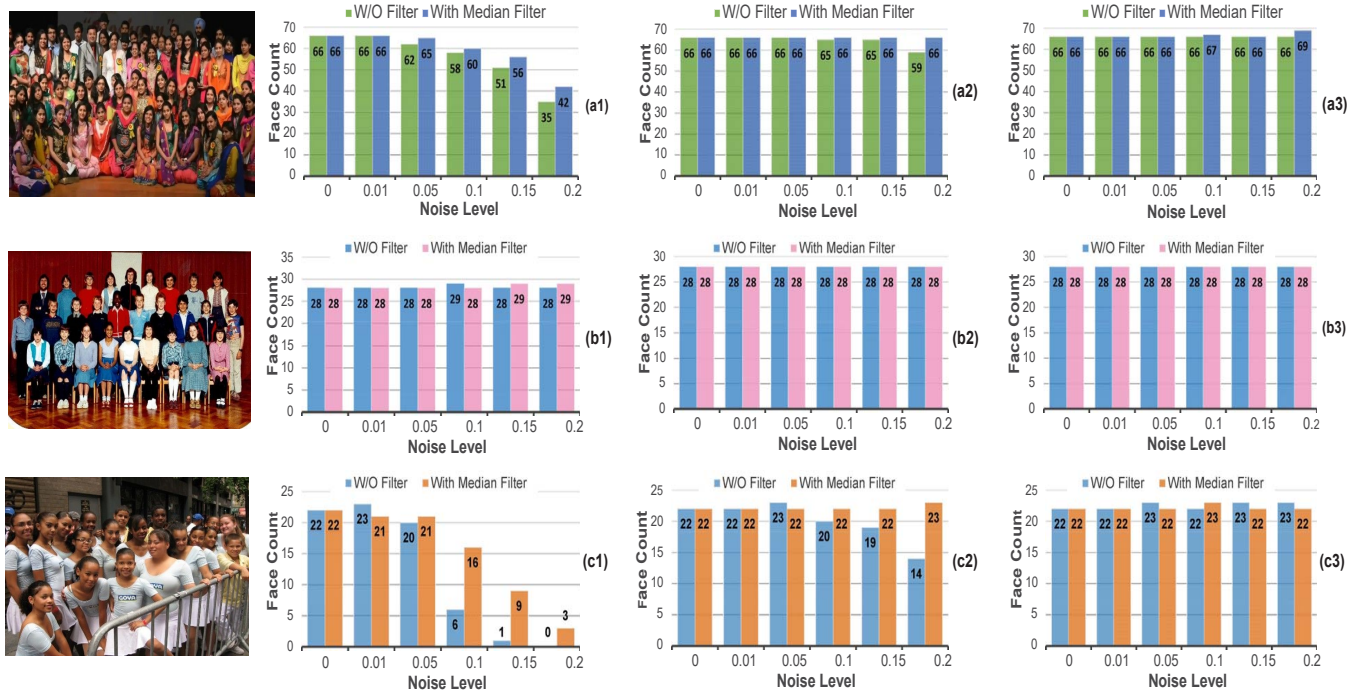


FIGURE 23. Illustrates the effect of median filter to improve the face detection performance for noise levels such as 0, 0.01, 0.05, 0.1, 0.15, and 0.2 for 3 different images. Each row consists of the original image and plots for (1) Gaussian noise, (2) salt and pepper noise, and (3) speckle noise, respectively. Median filtering significantly improves face count under salt and pepper and speckle noise, especially in the third image (c1–c3), where performance drops dramatically without filtering. In contrast, its effect is limited for Gaussian noise. These trends suggest that median filtering is highly effective for mitigating impulsive and granular noise types, offering a simple yet powerful preprocessing step for robust face detection in noisy real-world conditions. Image Sources: (a) <https://www.youtube.com/watch?v=cBRQgGB0i0A> (b) https://www.livingarchive.org.uk/content/catalogue_item/holne-chase-school-collection/holne-chase-school-photographs/middle-school-class-photograph-1991 (c) Wider Face Database [49].

TABLE 9. Performance of YOLOv9-based people counting method on different noise levels under a mix of human faces and animal faces. Number of images = 100; Reference count = 482. Results demonstrate an increased number of false positives due to animal faces, which decreases with increasing noise levels.

Noise Type	Noise Level	Count by Algorithm	MAE ± SD	RMSE
—	—	506	0.44 ± 0.86	0.97
Gaussian	0.01	495	0.67 ± 0.94	1.15
	0.05	342	2.16 ± 4.15	4.68
	0.1	180	3.24 ± 5.64	6.51
Salt and Pepper	0.01	500	0.38 ± 0.76	0.85
	0.05	468	0.68 ± 1.40	1.56
	0.1	359	1.71 ± 3.49	3.88
Speckle	0.01	496	0.46 ± 0.87	0.98
	0.05	499	0.67 ± 0.99	1.20
	0.1	495	1.07 ± 1.59	1.92

chain, millions of years ago, according to the Darwinian theory. Most of the existing face detection databases, such as the Wider Face database that we used for training our model in this study, seem to have overlooked this similarity and do not contain such a mix of human and animal faces for training and testing purposes. In this paper, we evaluate the performance of the YOLOv9 face detection model trained on the Wider Face database by testing on animal faces and a mix of human and animal faces.

Fig. 24 illustrates the performance of the YOLOv9-based face detection and people counting method on images with

a mix of human and animal faces. In Figs. 24 (a) - (e), the YOLOv9-based face detection and counting method successfully detects human faces from among the dog, squirrel, bull, and monkey faces. However, in Figs. 24 (f) - (j), there are many false positives due to the misclassification of animal faces as human faces by the face detection model. In Figs. 24 (f) and (j), certain facial features of dogs, such as the placement of hair and ears, resemble human features, which potentially led to the misclassification. In Figs. 24 (h) - (i), the facial features of monkeys, including the face shape, color, eyes, and jawline, have very sharp similarities with the human face. Fig. 24 (g) shows a group of human beings with animal masks, in which one of the faces is detected by the algorithm. Interestingly, the detection of masked human faces will be beneficial for identifying presentation attacks using fake faces in biometric systems.

The robustness of the YOLOv9-based face detection and people counting method was further evaluated on a curated set of 100 images containing both human and animal faces. These images contained between 1 and 45 human faces per image, with a total reference count of 482 human faces. As summarized in Tables 9 and 10, the model detected 506 faces in total, yielding a mean absolute error (MAE) of 0.44 and a root mean square error (RMSE) of 0.97 faces per image. The overcount is primarily attributed to false positives caused by the misclassification of animal faces as human faces.



FIGURE 24. Qualitative evaluation of YOLOv9 face detection on mixed human-animal imagery, revealing misclassification trends. The model frequently generates false positives by detecting animal faces (dogs, monkeys, etc.) as human faces. These results highlight the model's lack of discrimination between human and non-human facial patterns, especially in visually similar contexts, and underscore the need for more robust training to handle such out-of-distribution (OOD) data.

Image Sources: (a) <https://www.factualamerica.com/documentary-lab/8-documentaries-on-the-science-of-human-animal-interactions>

(b) <https://www.pinterest.com/pin/835065955886770061/> (c) https://commons.wikimedia.org/wiki/File:Avaniyapuram_Jallikattu_08.jpg

(d) <https://happytailsvacaville.com/about-us/team/>

(e) <https://www.scmp.com/news/asia/south-asia/article/2173290/monkey-kills-12-day-old-baby-boy-after-snatching-breastfeeding>

(f) <https://www.lifestyleasia.com/bk/style/fashion/balenciagas-i-love-pets-collection/> (g) <https://www.filmweb.no/streamingguide/film/76483> (h)

https://www.freepik.com/premium-ai-image/man-poses-with-several-monkeys-room-with-one-them-wearing-shirt-that-says-monkey_73588009.htm

(i) https://www.freepik.com/premium-ai-image/group-monkeys-with-their-mouths-open-mouths-wide-open-all-them-looking-camera_145093580.htm

(j) https://pngtree.com/freebackground/a-group-of-beagle-puppies-surround-a-man-photo_7505630.html

Model performance was observed to degrade with increasing levels of both blurring and noise. Notably, for a fixed standard deviation in Gaussian blur, increasing the kernel size resulted in minimal change in detection performance, suggesting that the spatial spread of the blur had less impact than the intensity of the distortion. In the case of noise, low levels of salt-and-pepper noise (e.g., noise level = 0.01) slightly improved the face counting accuracy, possibly by acting as a mild regularizer. In contrast, higher levels of Gaussian noise significantly deteriorated model performance across all metrics. Speckle noise introduced a unique pattern: although the total number of faces detected remained nearly constant (between 495 and 499), MAE and RMSE values increased sharply with noise level, indicating an increase in false positives rather than detection loss.

These findings indicate that the YOLOv9 detector is sensitive to high-frequency distortions, particularly when evaluating scenes containing non-human facial structures. The consistent misclassification of animal faces, especially

those with visual similarities to human facial features (e.g., monkeys or dogs), highlights a limitation in the model's discriminative capacity. This underscores the importance of including negative training samples (e.g., animal faces) to improve model robustness and reduce false positives. However, attempting to suppress animal face detection entirely may risk unintended consequences, such as reduced sensitivity to certain human facial variations that resemble animal traits (e.g., eye spacing, jaw contours, or ear positioning). Therefore, future work should explore domain-aware fine-tuning strategies or hierarchical classifiers that distinguish human from non-human faces without compromising true positive rates. This analysis provides critical insight into the failure modes of deep learning-based face detection systems when applied in unconstrained, mixed-species environments, which is increasingly relevant in surveillance, wildlife monitoring, and safety applications.



FIGURE 25. Assessment of YOLOv9's generalization performance on unseen infant faces across diverse visual conditions. The model demonstrates reliable face detection on infants in various postures, skin tones, and environments, including colored, grayscale, and artificially generated contexts. Despite being untrained on infant-specific data, YOLOv9 maintains robust detection, indicating strong generalization capability to unfamiliar age demographics.
Image Sources: (a) <https://www.shutterstock.com/image-photo/group-smiling-babies-standing-huddle-on-319530215>
(b) <https://www.alamy.com/stock-photo-8-babies-lying-down-portrait-29433783.html?imageid=81F66679-DB21-42BB-8308-1423BD2CC630&p=52031&pn=1&searchId=072C23C019A0922bc58ff2faad23067b&searchtype=0>
(c) https://www.freepik.com/premium-ai-image/photos-babies-lined-up-smiling-wearing-hooded-clothes-natural-colors_56400054.htm
(d) <https://easy-peasy.ai/ai-image-generator/images/kindergarten-first-day-emotions-crying-laughing-fearful>
(e) <https://www.theguardian.com/social-care-network/2015/jan/19/adoption-fostering-hard-to-place-children>
(f) <https://www.fieldfisher.com/en/injury-claims/insights/why-are-black-babies-dying-more-stillbirth-rate>

J. PERFORMANCE OF YOLOV9 FACE COUNTING METHOD FOR INFANT FACES

Infant face detection plays a vital role in contactless health monitoring systems deployed in neonatal intensive care units (NICUs), smart cradles, and home-based infant health monitoring platforms. However, the Wider Face database used to train the YOLOv9 face detection model predominantly consists of adult faces, with minimal representation of infant or child faces. To evaluate the generalizability of the model, we tested its performance on a dedicated set of infant face images.

Fig. 25 illustrates qualitative results of the YOLOv9 model on infant faces under various conditions. The model demonstrates successful detection across diverse facial orientations (Fig. 25a), poses (Fig. 25e), facial expressions (Fig. 25d), and skin tone variations (Fig. 25c). However, detection fails in cases where infants lie in a prone position with faces partially occluded, as shown in Fig. 25f. Such scenarios are common due to frequent movement and non-frontal postures of infants. This indicates the need for infant-specific training data encompassing a wide range of poses and occlusion patterns to ensure reliable performance in real-world deployments. Despite the model being trained exclusively on adult faces, these results show that YOLOv9

exhibits a reasonable degree of generalization to infant face detection. Nevertheless, fine-tuning on age-diverse or infant-specific datasets could improve both accuracy and robustness.

To quantitatively evaluate face counting performance on infants, we tested the model on 113 images containing 697 infant faces. The results, summarized in Tables 11 and 12, show that the model successfully detected 682 faces, achieving a mean absolute error (MAE) of 0.26 and a root mean square error (RMSE) of 1.07 faces per image. As observed in prior evaluations, the performance declines with increasing blur and noise. These results underscore the importance of distortion-aware model evaluation and suggest that robust face detection in infants may require domain-specific augmentation or retraining.

K. PERFORMANCE OF YOLOV9 FACE COUNTING UNDER HUMAN FACES INSIDE VEHICLES

The YOLOv9-based face detection and people counting method has practical applications in vehicular and roadside environments for tasks such as driver health monitoring, drowsiness detection, security verification, enforcement of road regulations, and surveillance in public transportation systems. A critical challenge in such settings involves

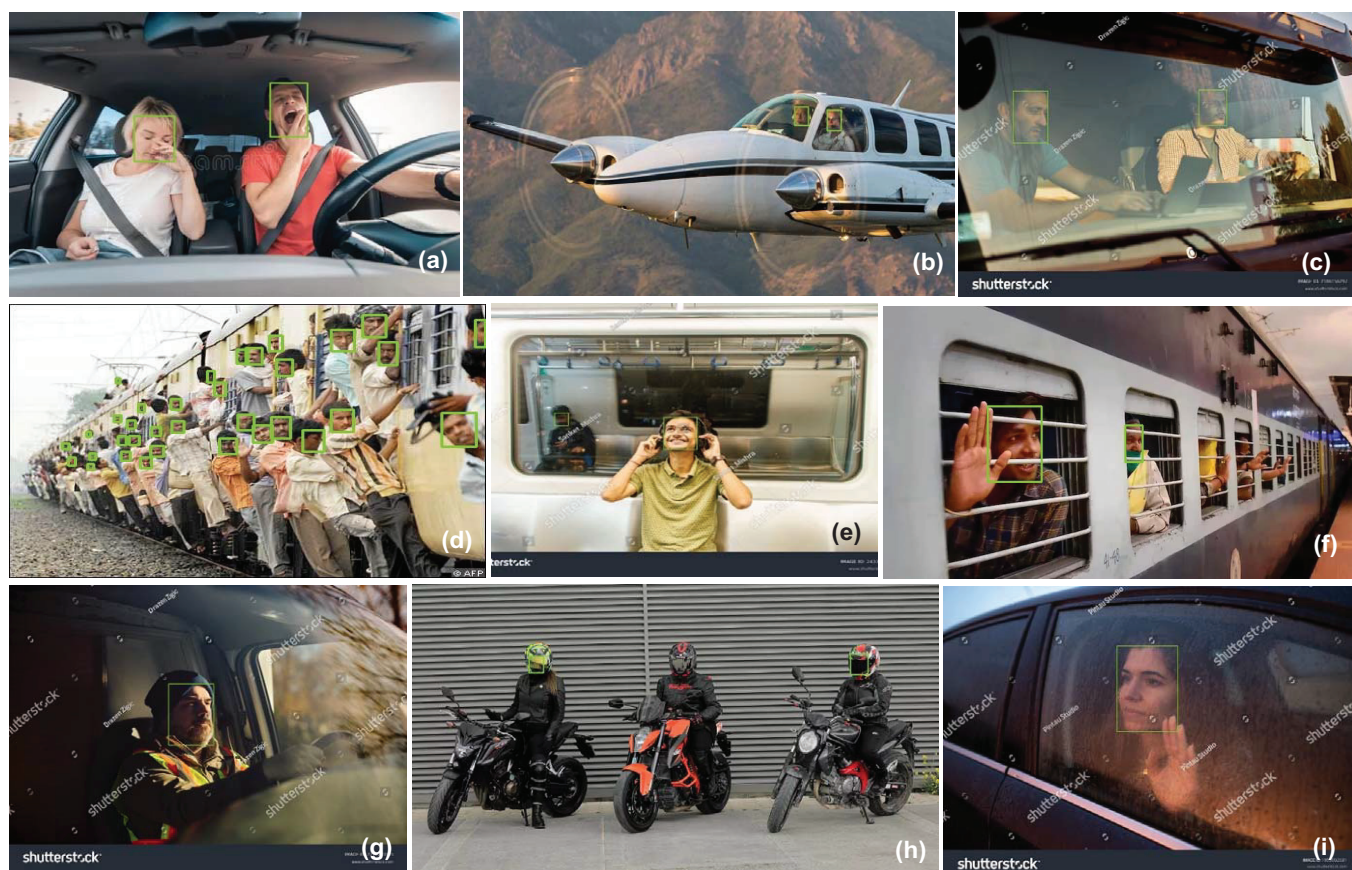


FIGURE 26. Demonstrates the effectiveness of the YOLOv9-based face detection and people counting method on diverse real-world scenarios involving human faces inside vehicles and behind glass surfaces. The examples include individuals inside cars (a, c, g, i), airplanes (b), and trains (d, e, f), as well as motorcyclists wearing helmets (h). Despite challenging conditions such as partial occlusion, glare, window reflections, varying poses, and motion blur, the model successfully detects most visible faces. Notably, it performs robustly even when subjects are partially hidden by tinted windows (g, i) or crowded environments (d, f), confirming the method's suitability for surveillance and transportation monitoring applications.

Image Sources: (a) <https://www.gettyimages.no/search/2/image?phrase=couple+sleeping+car> (b) <https://disciplesofflight.com/pilot-personality-traits/> (c) <https://www.shutterstock.com/es/image-photo/young-man-driving-truck-while-his-2186156297> (d) <https://www.ft.com/content/45ba425c-db78-11de-9424-00144feabdc0> (e) <https://www.shutterstock.com/es/image-photo/portrait-young-indian-man-passenger-traveling-2433403069> (f) <https://www.indiatoday.in/india/story/railways-minister-ashwini-vaiishnav-special-trains-passengers-festive-season-2607396-2024-09-27> (g) <https://www.shutterstock.com/es/image-photo/professional-truck-driver-on-road-view-2272203115> (h) <https://www.euronews.com/video/2022/03/21/meet-the-queen-bikers-tunisia-s-first-all-female-motorcycle-club> (i) <https://www.istockphoto.com/photo/close-up-young-woman-inside-car-looking-through-window-place-hand-on-glass-with-gm1311789843-400783236>

detecting faces under occlusion by windscreens, glass reflections, or metal bars.

We evaluated the model's effectiveness in vehicular scenes, as illustrated in Fig. 26. The model successfully identifies faces in diverse environments such as cars (Fig. 26a), aircraft (Fig. 26b), and trains (Fig. 26d, f), even behind glass (Fig. 26c, e, g, i) or helmets (Fig. 26h). Despite occlusions and reflections due to fogged or shadowed windscreens and structural elements such as window bars, the model consistently detects human faces, demonstrating robustness to real-world distortions and cluttered backgrounds. This confirms YOLOv9's suitability for applications like surveillance and passenger counting. Notably, in Fig. 26(e), the model even detected a human face reflected on glass, indicating high sensitivity to facial patterns. However, a failure case is observed in Fig. 26(h),

where one of three helmeted individuals was missed, likely due to partial occlusion or reduced visibility of facial features.

Quantitative evaluation was performed on a set of 101 images containing humans in or on vehicles. For a reference count of 395 human faces, the model achieved an MAE of 0.24 and an RMSE of 0.80 faces per image. As with earlier findings, performance degrades with increasing levels of blur and noise (Tables 13 and 14). The degradation was less pronounced for salt-and-pepper and speckle noise, indicating some resilience to high-frequency noise.

L. PERFORMANCE OF YOLOV9 FACE COUNTING UNDER HUMAN FACES WITH ARTIFICIAL FACES

Public spaces frequently include statues of people, which introduce ambiguity in face detection systems. We tested the YOLOv9 model on real-world images containing both human faces and artificial (statue) faces to evaluate its ability to

TABLE 10. Performance of YOLOv9-based people counting method on different blurring effects under a mix of human faces and animal faces
Number of images = 100; Reference count = 482
Results demonstrate an increased number of false positives due to animal faces, which decreases with increasing blurring levels.

Blur Type	Kernel Size	Count by Algorithm	MAE $\pm$ SD	RMSE
—	—	506	0.44 $\pm$ 0.86	0.97
Average	5	483	0.63 $\pm$ 1.21	1.36
	10	390	1.36 $\pm$ 3.53	3.79
	15	296	2.22 $\pm$ 4.28	4.82
	20	207	3.07 $\pm$ 5.38	6.20
	25	131	3.73 $\pm$ 6.71	7.68
	30	82	4.10 $\pm$ 7.12	8.21
Disk	3	475	0.65 $\pm$ 1.42	1.57
	6	376	1.44 $\pm$ 3.57	3.85
	9	266	2.54 $\pm$ 4.69	5.33
	12	172	3.36 $\pm$ 5.87	6.77
	15	99	4.03 $\pm$ 7.02	8.09
Gaussian $\sigma = 1$	10	500	0.48 $\pm$ 0.88	1.00
	20	500	0.48 $\pm$ 0.88	1.00
	30	500	0.48 $\pm$ 0.88	1.00
Gaussian $\sigma = 2$	10	461	0.77 $\pm$ 1.78	1.94
	20	455	0.77 $\pm$ 1.91	2.06
	30	455	0.77 $\pm$ 1.91	2.06
Gaussian $\sigma = 4$	10	400	1.28 $\pm$ 3.44	3.67
	20	331	1.85 $\pm$ 3.94	4.35
	30	327	1.89 $\pm$ 3.93	4.36
Gaussian $\sigma = 8$	10	387	1.39 $\pm$ 3.53	3.80
	20	212	2.98 $\pm$ 5.18	5.98
	30	117	3.83 $\pm$ 6.64	7.67
Motion $\theta = 0$	10	401	1.29 $\pm$ 3.24	3.49
	20	273	2.49 $\pm$ 4.99	5.58
	30	143	3.61 $\pm$ 6.98	7.86
Motion $\theta = 15$	10	402	1.28 $\pm$ 3.25	3.49
	20	260	2.54 $\pm$ 5.27	5.85
	30	124	3.76 $\pm$ 7.15	8.08
Motion $\theta = 30$	10	391	1.21 $\pm$ 3.43	3.64
	20	210	2.94 $\pm$ 5.71	6.42
	30	93	3.99 $\pm$ 7.11	8.15
Motion $\theta = 45$	10	384	1.18 $\pm$ 3.48	3.67
	20	194	3.10 $\pm$ 5.85	6.62
	30	88	4.06 $\pm$ 7.00	8.09
Motion $\theta = 60$	10	382	1.24 $\pm$ 3.37	3.59
	20	220	2.80 $\pm$ 5.18	5.89
	30	97	3.99 $\pm$ 6.94	8.01

TABLE 11. Performance of YOLOv9-based people counting method for different noise levels on untrained infant faces
Number of images = 113; Reference count = 697
Results demonstrate good generalization of the model on infant faces. However, the performance is degraded with increasing noise levels.

Noise Type	Noise Level	Count by Algorithm	MAE $\pm$ MAE	RMSE
—	—	682	0.26 $\pm$ 1.04	1.07
Gaussian	0.01	552	1.44 $\pm$ 2.48	2.87
	0.05	160	4.86 $\pm$ 4.07	6.34
	0.1	50	5.76 $\pm$ 4.04	7.03
Salt and Pepper	0.01	671	0.53 $\pm$ 1.19	1.30
	0.05	441	2.32 $\pm$ 3.33	4.06
	0.1	224	4.22 $\pm$ 4.20	5.96
Speckle	0.01	626	0.73 $\pm$ 1.40	1.58
	0.05	443	2.42 $\pm$ 3.35	4.13
	0.1	313	3.49 $\pm$ 4.00	5.30

TABLE 12. Performance of YOLOv9-based people counting method for different blurring effects on untrained infant faces
Number of images = 113; Reference count = 697
Results demonstrate good generalization of the model on infant faces. However, the performance is degraded with increasing blurring levels.

Blur Type	Kernel Size	Count by Algorithm	MAE $\pm$ SD	RMSE
—	—	682	0.26 $\pm$ 1.04	1.07
Average	5	614	0.79 $\pm$ 2.10	2.24
	10	304	3.57 $\pm$ 4.22	5.53
	15	171	4.71 $\pm$ 4.14	6.27
	20	94	5.39 $\pm$ 4.20	6.83
	25	69	5.58 $\pm$ 4.18	6.97
	30	56	5.71 $\pm$ 4.20	7.09
Disk	3	596	0.98 $\pm$ 2.34	2.54
	6	264	3.90 $\pm$ 4.29	5.80
	9	131	5.04 $\pm$ 4.12	6.51
	12	100	5.30 $\pm$ 4.27	6.81
	15	67	5.59 $\pm$ 4.21	7.00
Gaussian $\sigma = 1$	10	661	0.50 $\pm$ 1.59	1.66
	20	661	0.50 $\pm$ 1.59	1.66
	30	661	0.50 $\pm$ 1.59	1.66
Gaussian $\sigma = 2$	10	564	1.19 $\pm$ 2.66	2.92
	20	565	1.22 $\pm$ 2.69	2.95
	30	565	1.22 $\pm$ 2.69	2.95
Gaussian $\sigma = 4$	10	324	3.34 $\pm$ 4.25	5.40
	20	216	4.31 $\pm$ 4.21	6.02
	30	215	4.32 $\pm$ 4.24	6.05
Gaussian $\sigma = 8$	10	306	3.55 $\pm$ 4.23	5.52
	20	109	5.27 $\pm$ 4.06	6.65
	30	76	5.53 $\pm$ 4.27	6.98
Motion $\theta = 0$	10	437	2.32 $\pm$ 3.51	4.20
	20	177	4.60 $\pm$ 4.20	6.23
	30	102	5.30 $\pm$ 4.18	6.75
Motion $\theta = 15$	10	455	2.14 $\pm$ 3.24	3.88
	20	183	4.55 $\pm$ 4.19	6.18
	30	88	5.39 $\pm$ 4.24	6.86
Motion $\theta = 30$	10	399	2.65 $\pm$ 3.64	4.51
	20	131	5.04 $\pm$ 4.30	6.63
	30	61	5.63 $\pm$ 4.28	7.07
Motion $\theta = 45$	10	394	2.73 $\pm$ 3.63	4.55
	20	117	5.19 $\pm$ 4.29	6.73
	30	54	5.69 $\pm$ 4.25	7.10
Motion $\theta = 60$	10	381	2.90 $\pm$ 3.77	4.75
	20	123	5.15 $\pm$ 4.27	6.69
	30	53	5.70 $\pm$ 4.27	7.12

TABLE 13. Performance of YOLOv9-based people counting method on different noise levels on human faces inside vehicles
Number of images = 101; Reference count = 395
Results demonstrate the performance of the model even in challenging conditions and between glass panes. However, the performance is degraded for increased noise levels.

Noise Type	Noise Level	Count by Algorithm	MAE $\pm$ SD	RMSE
—	—	393	0.24 $\pm$ 0.76	0.80
Gaussian	0.01	370	0.78 $\pm$ 2.24	2.37
	0.05	226	2.13 $\pm$ 4.36	4.86
	0.1	91	3.07 $\pm$ 4.76	5.66
Salt and Pepper	0.01	383	0.24 $\pm$ 0.95	0.97
	0.05	336	0.88 $\pm$ 2.48	2.63
	0.1	250	1.55 $\pm$ 4.07	4.35
Speckle	0.01	387	0.26 $\pm$ 0.91	0.94
	0.05	373	0.65 $\pm$ 1.92	2.03
	0.1	349	1.03 $\pm$ 2.91	3.09

TABLE 14. Performance of YOLOv9-based people counting method on different blurring effects on human faces inside vehicles
Number of images = 101; Reference count = 395
Results demonstrate the performance of the model even in challenging conditions and between glass panes. However, the performance is degraded for increased blurring levels.

Blur Type	Kernel Size	Count by Algorithm	MAE $\pm$ SD	RMSE
–	–	393	0.24 $\pm$ 0.76	0.80
Average	5	360	0.43 $\pm$ 2.13	2.17
	10	293	1.05 $\pm$ 3.98	4.12
	15	240	1.59 $\pm$ 4.35	4.63
	20	194	2.03 $\pm$ 4.55	4.98
	25	152	2.47 $\pm$ 4.73	5.33
Disk	30	110	2.84 $\pm$ 4.75	5.53
	3	352	0.50 $\pm$ 2.52	2.57
	6	277	1.23 $\pm$ 4.25	4.42
	9	227	1.72 $\pm$ 4.41	4.74
	12	182	2.15 $\pm$ 4.59	5.07
Gaussian $\sigma = 1$	15	127	2.73 $\pm$ 4.76	5.50
	10	375	0.28 $\pm$ 1.62	1.64
	20	375	0.28 $\pm$ 1.62	1.64
Gaussian $\sigma = 2$	30	375	0.28 $\pm$ 1.62	1.64
	10	347	0.57 $\pm$ 2.58	2.64
	20	344	0.58 $\pm$ 2.67	2.73
Gaussian $\sigma = 4$	30	344	0.58 $\pm$ 2.67	2.73
	10	302	1.00 $\pm$ 3.87	4.00
	20	266	1.34 $\pm$ 4.26	4.47
Gaussian $\sigma = 8$	30	266	1.34 $\pm$ 4.26	4.47
	10	298	1.06 $\pm$ 3.97	4.11
	20	205	1.92 $\pm$ 4.50	4.89
Motion $\theta = 0$	30	158	2.39 $\pm$ 4.72	5.29
	10	332	0.74 $\pm$ 3.15	3.23
	20	238	1.63 $\pm$ 4.34	4.64
Motion $\theta = 15$	30	176	2.21 $\pm$ 4.67	5.17
	10	332	0.70 $\pm$ 3.32	3.40
	20	232	1.67 $\pm$ 4.35	4.66
Motion $\theta = 30$	30	155	2.40 $\pm$ 4.79	5.36
	10	324	0.76 $\pm$ 3.65	3.72
	20	206	1.89 $\pm$ 4.52	4.90
Motion $\theta = 45$	30	120	2.74 $\pm$ 4.96	5.67
	10	319	0.83 $\pm$ 3.74	3.84
	20	193	2.04 $\pm$ 4.58	5.01
Motion $\theta = 60$	30	105	2.87 $\pm$ 4.92	5.70
	10	310	0.88 $\pm$ 3.93	4.03
	20	197	2.00 $\pm$ 4.45	4.88
	30	111	2.83 $\pm$ 4.89	5.65

differentiate between animate and inanimate faces (Fig. 27). The results show that the model consistently detects both real and artificial faces, including sculpted statues. This behavior suggests that the YOLOv9 model relies primarily on geometric and texture-based features, without any contextual inference of vitality or material properties. In Fig. 27(c), the artificial face closely mimics real facial cues such as eye structure, wrinkles, and skin tone, leading to confident false positives. Attempts to suppress such detections by training on negative examples (e.g., statue faces) may adversely impact sensitivity to genuine human faces with similar characteristics, such as elderly individuals with prominent facial textures. These findings highlight a limitation in deep learning-based face detectors: the absence of a high-level semantic understanding of object context. Incorporating additional modalities or semantic priors, such as infrared

signatures or micro-expression cues, may help distinguish living from non-living faces, though such solutions are challenging to scale in real-world scenarios with large crowds or grayscale imagery.

M. IMPACT OF ARTIFICIALLY CREATED AMBIGUOUS FACIAL VISUAL PATTERNS

Fig. 28 presents examples where the YOLOv9 model generates bounding boxes in regions devoid of any human or animal faces. Unlike Fig. 24 where an animal face was detected as a human face, these false positives do not correspond to any meaningful object but appear to result from complex background textures or fused shapes that resemble facial structures. It is logical to claim that the fusion of portions of different objects may have created ambiguous visual facial patterns having some kind of facial relevant information, such as color, shape, or any other details of faces.

The human brain is often said to showcase the phenomenon called "pareidolia"³ characterized by the illusory perception of objects on random objects or shapes. For example, we often see recognizable patterns like faces or objects in clouds, rocks, bushes, or constellations, which do not exist, but create an illusory picture of their existence. Face pareidolia refers to the perception of human faces on random objects or shapes [61]. Analogous to the face pareidolia of the human brain, we term the phenomenon of perceiving meaningful shapes, such as faces, by an object detection model due to the fusion of different portions of objects as demonstrated in Fig. 28 as "machine pareidolia". In our study, this effect is mostly observed in low-resolution images or after significant blurring or noise addition. In [61], Hamilton et al. (2024) examined whether computer vision-based face detection models trained to detect human faces exhibit the pareidolia effect. We infer that this pareidolia effect causes the false positives in the images. These findings emphasize the need for further research into improving model robustness against such perceptual illusions. Approaches such as attention-guided learning, uncertainty modeling, or hybrid architectures that incorporate both geometric and semantic constraints may offer promising directions to reduce such hallucinations in real-world applications.

N. YOLOV12-BASED FACE DETECTION MODEL

To further assess the robustness and generalizability of our proposed YOLOv9-based face detection and counting method, we conducted a comparative evaluation using the latest attention-centric YOLOv12n architecture [62]. The YOLOv12n model has recently demonstrated high accuracy and exceptional inference speed across multiple object detection benchmarks. For a fair comparison, we trained and validated the YOLOv12n architecture on the same Wider Face dataset using 5-fold cross-validation to mitigate bias

³"Pareidolia (Noun) (Psychology): The perception of recognizable patterns or images, in random or vague arrangements of shapes, lines, colors, etc" - Oxford English Dictionary [60]



FIGURE 27. Demonstrates the capability of the YOLOv9-based face detection and face counting method to identify both human and artificial faces in real-world environments. The model successfully detects human faces even when they are positioned close to statues (a–b). However, it also produces false positives by detecting non-human faces from statues and sculptures (a–c), highlighting the model's sensitivity to facial-like patterns and the potential for over-detection in the presence of artistic or artificial facial structures.

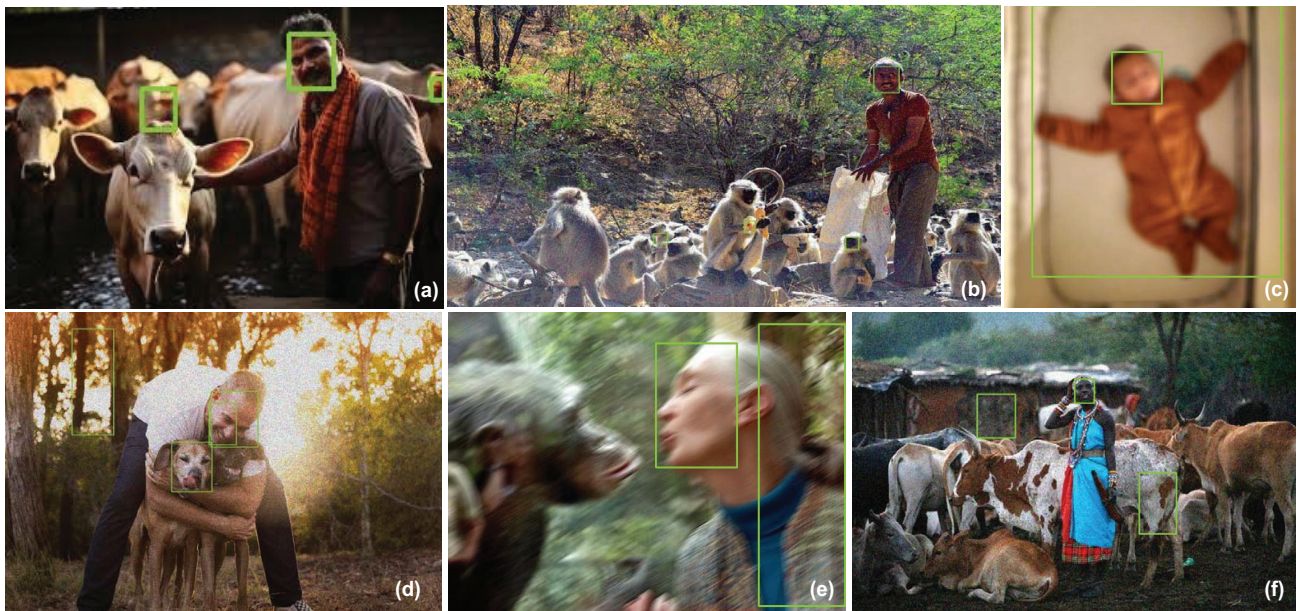


FIGURE 28. Highlights common sources of false positives in the YOLOv9-based face detection model, caused by ambiguous visual patterns that resemble human facial features. These results underscore the model's vulnerability to misidentifying non-human objects with facial-like structures, which may impact detection accuracy in real-world natural environments.

Image Sources: (a) https://www.freepik.com/premium-ai-image/senior-indian-man-taking-care-cow-dairy-cattle-farm-village_59566665.htm
 (b) <https://ruralindiaonline.org/en/articles/in-sheoganj-monkey-business-is-a-great-service/>
 (c) <https://myfirstnursery.co.uk/blogs/news/what-does-sids-stand-for-can-sids-be-prevented>
 (d) <https://www.aliciariushphotography.com/blog-content/2016/2/25/how-to-get-real-emotions-from-people-and-their-pets>
 (e) <https://www.abc.net.au/listen/radionational/archived/animalpeople/wildlife-documentaries-and-environmental-ignorance/5700890>
 (f) <https://www.linkedin.com/pulse/origin-cattle-maasai-fable-story-liz-madoka/>

associated with a single train-test split. Table 15 summarizes the face detection performance across the five folds. At the end of the cross-validation, the model achieved a mean precision of 0.847, recall of 0.603, and mAP@50 of 0.685.

To evaluate robustness under image degradation, we tested the YOLOv12-based face detection and counting method on the Wider Face dataset with various blur and noise conditions. The results are detailed in Tables 16, 17, and 18, which

capture performance under average and disk blurring, motion blur, and different noise types. For the original (clean) test set, the YOLOv12-based model achieved a precision of 0.831, recall of 0.577, and mAP@50 of 0.654 for face detection, with a mean absolute error (MAE) of 3.92 and a root mean square error (RMSE) of 23.88 for face counting. As observed previously with YOLOv9, performance consistently degrades with increasing blur and noise intensity, confirming the

TABLE 15. Performance of the YOLOv12-based face detection model with 5-fold cross validation

Fold	mAP50	Precision	Recall
1	0.676897	0.844417	0.593970
2	0.651558	0.835937	0.569118
3	0.631400	0.825049	0.549059
4	0.678229	0.841597	0.594041
5	0.685429	0.846758	0.602515

TABLE 16. Performance of YOLOv12-based face detection and people counting model on different blurring effects on the Wider Face database. Results demonstrate performance degradation of the model with increased blurring levels.

Blur Type	Kernel Size	Face Detection				Counting	
		Precision	Recall	mAP50	mAP50-95	MAE ± SD	RMSE
—	—	0.831	0.577	0.654	0.341	3.92 ± 23.56	23.88
Average	5	0.821	0.537	0.619	0.324	5.16 ± 28.24	28.70
	10	0.778	0.409	0.489	0.258	7.53 ± 35.72	36.50
	15	0.751	0.29	0.361	0.181	9.03 ± 38.89	39.92
	20	0.748	0.212	0.286	0.141	10.02 ± 39.61	40.86
	25	0.759	0.149	0.237	0.114	10.76 ± 39.86	41.28
	30	0.179	0.185	0.207	0.100	11.26 ± 39.95	41.50
Disk	3	0.819	0.526	0.611	0.320	5.36 ± 29.12	29.61
	6	0.777	0.377	0.456	0.236	7.93 ± 37.11	37.94
	9	0.754	0.266	0.337	0.171	9.36 ± 39.25	40.34
	12	0.759	0.181	0.262	0.131	10.39 ± 39.76	41.09
	15	0.785	0.119	0.221	0.110	11.11 ± 39.92	41.43
Gaussian $\sigma = 1$	10	0.841	0.584	0.666	0.366	4.35 ± 25.07	25.44
	20	0.841	0.584	0.666	0.366	4.35 ± 25.07	25.44
	30	0.841	0.584	0.666	0.366	4.35 ± 25.07	25.44
Gaussian $\sigma = 2$	10	0.821	0.518	0.603	0.329	5.83 ± 30.74	31.28
	20	0.824	0.515	0.600	0.328	5.88 ± 30.85	31.40
	30	0.824	0.515	0.600	0.328	5.88 ± 30.85	31.40
Gaussian $\sigma = 4$	10	0.790	0.433	0.512	0.274	7.32 ± 35.31	36.05
	20	0.786	0.336	0.415	0.225	8.54 ± 38.34	39.27
	30	0.783	0.334	0.412	0.224	8.58 ± 38.34	39.31
Gaussian $\sigma = 8$	10	0.784	0.413	0.494	0.262	7.53 ± 35.78	36.56
	20	0.766	0.224	0.303	0.158	9.88 ± 39.57	40.78
	30	0.810	0.142	0.248	0.133	10.85 ± 39.89	41.33

sensitivity of both models to image distortion.

Interestingly, despite employing k-fold cross-validation and tuning YOLOv12 hyperparameters accordingly, the YOLOv12-based method underperformed when compared to the YOLOv9-based method under identical test conditions. This is evident in both detection metrics (precision, recall, mAP@50) and counting errors (MAE, RMSE). These results suggest that while k-fold cross-validation improves generalization consistency, it does not guarantee superior performance if the underlying architecture is less suited to the domain-specific characteristics of the task—in this case, densely packed, small-scale faces. The findings reinforce the suitability of YOLOv9 for robust face detection and counting in challenging real-world scenarios.

O. COMPARISON WITH BENCHMARK FACE DETECTORS

In this subsection, we compare the performance of the YOLOv9-based face detection and people counting method with other benchmark face detection methods such as Haar

TABLE 17. Performance of YOLOv12-based face detection and people counting model under motion blurring effects on the Wider Face database. Results demonstrate performance degradation of the model with increased blurring levels.

len	θ	Precision	Recall	mAP50	mAP50-95	MAE ± SD	RMSE
—	0	0.831	0.577	0.654	0.3410	3.9154 ± 23.56	23.88
10	0	0.795	0.461	0.536	0.2820	6.2988 ± 32.28	32.88
	15	0.805	0.465	0.541	0.2860	6.34966 ± 32.24	32.86
	30	0.811	0.453	0.532	0.2850	6.89368 ± 34.47	35.15
	45	0.813	0.441	0.523	0.2810	7.17483 ± 35.33	36.04
	60	0.807	0.437	0.523	0.2760	7.39554 ± 35.54	36.30
20	0	0.751	0.285	0.345	0.1790	9.05889 ± 39.16	40.19
	15	0.780	0.271	0.337	0.1790	9.2579 ± 39.31	40.38
	30	0.789	0.238	0.307	0.1660	9.7576 ± 39.63	40.81
	45	0.778	0.221	0.291	0.1580	9.9789 ± 39.70	40.93
	60	0.775	0.231	0.305	0.1620	9.9151 ± 39.61	40.83
30	0	0.700	0.192	0.241	0.1180	10.1934 ± 39.81	41.09
	15	0.753	0.162	0.229	0.1160	10.5862 ± 39.92	41.29
	30	0.733	0.113	0.184	0.0972	11.1066 ± 39.97	41.48
	45	0.651	0.101	0.165	0.0895	11.1981 ± 39.93	41.46
	60	0.658	0.111	0.178	0.0951	11.1435 ± 39.91	41.43

TABLE 18. Performance of YOLOv12-based face detection and people counting model under different noise conditions on the Wider Face database. Results demonstrate performance degradation of the model with increased noise levels.

Noise Type	Noise Level	Precision	Recall	mAP50	mAP50-95	MAE ± SD	RMSE
—	—	0.831	0.577	0.654	0.341	3.92 ± 23.56	23.88
Gaussian	0.01	0.779	0.479	0.550	0.271	5.25 ± 25.38	25.91
	0.05	0.628	0.273	0.299	0.136	8.59 ± 31.98	33.11
	0.100	0.485	0.100	0.099	0.045	10.85 ± 37.05	38.60
Salt and Pepper	0.01	0.822	0.551	0.626	0.322	4.20 ± 24.06	24.42
	0.05	0.764	0.444	0.510	0.250	5.74 ± 26.19	26.81
	0.1	0.680	0.335	0.376	0.177	7.59 ± 29.78	30.72
Speckle	0.01	0.825	0.565	0.641	0.330	4.00 ± 23.71	24.04
	0.05	0.791	0.515	0.586	0.287	4.64 ± 24.40	24.83
	0.1	0.746	0.464	0.524	0.248	5.52 ± 25.83	26.41

Cascade [63], MTCNN [54], RetinaFace [64], Mediapipe [65], and ResNet SSD [66], which are widely used for face detection in past studies. Each model was tested under identical conditions across multiple datasets, including Wider Face, human-animal mixed scenes, vehicular images, and infant face datasets. Haar Cascades are a traditional machine learning method that uses edge and line features (Haar features) extracted from images and a cascade of boosted classifiers (AdaBoost) to detect objects, including faces [63]. MTCNN is a deep learning-based method that uses a cascade of three CNNs (Proposal Network, Refine Network, and Output Network) to detect faces and facial landmarks simultaneously [54]. RetinaFace is a multi-task learning-based single-stage dense face localization model, which can detect face bounding boxes and five facial landmark points simultaneously [64]. MediaPipe is an open-source framework from Google that uses its Face Mesh model to detect and localize 468 3D facial landmarks in real-time [65]. The model uses a deep learning architecture and CNNs trained on vast datasets [65]. It processes input

TABLE 19. Performance comparison of benchmark face detection models in terms of MAE $\pm$ SD for face counting on the Wider face database. Results demonstrate that the YOLOv9-based model achieved better performance compared to benchmark models.

Database	Image Type	Noise/Blur Level	Haar Cascade	MTCNN	RetinaFace	Mediapipe	ResNet SSD	YOLOv12	YOLOv9	
Wider Face	Org	Original	—	10.21 ± 38.93	9.55 ± 37.20	3.78 ± 23.04	11.80 ± 39.94	11.64 ± 39.93	3.92 ± 23.56	3.36 ± 22.20
	Noises	Gaussian	$\sigma = 0.01$	10.35 ± 39.11	11.20 ± 39.40	5.87 ± 29.31	11.91 ± 39.92	11.76 ± 39.91	5.25 ± 25.38	4.93 ± 24.94
			$\sigma = 0.05$	10.78 ± 39.45	12.17 ± 39.82	9.73 ± 38.24	12.15 ± 39.87	12.13 ± 39.86	8.59 ± 31.98	8.86 ± 35.64
			$\sigma = 0.1$	11.20 ± 39.62	12.29 ± 39.82	10.97 ± 39.57	12.27 ± 39.83	12.28 ± 39.83	10.85 ± 37.05	10.91 ± 39.64
		Salt & Pepper	$d = 0.01$	10.27 ± 38.98	10.02 ± 37.90	4.57 ± 25.46	11.83 ± 39.94	11.67 ± 39.93	4.20 ± 24.06	3.65 ± 22.78
			$d = 0.05$	10.42 ± 39.17	11.39 ± 39.43	6.94 ± 31.78	11.96 ± 39.91	11.84 ± 39.91	5.74 ± 26.19	5.54 ± 26.35
			$d = 0.1$	10.67 ± 39.40	11.98 ± 39.76	9.10 ± 36.81	12.09 ± 39.88	12.05 ± 39.88	7.59 ± 29.78	7.69 ± 31.85
		Speckle	$\sigma = 0.01$	10.25 ± 38.95	9.93 ± 37.73	4.10 ± 23.96	11.83 ± 39.93	11.66 ± 39.92	4.00 ± 23.71	3.45 ± 22.35
			$\sigma = 0.05$	10.33 ± 39.09	11.16 ± 39.24	5.34 ± 27.52	11.92 ± 39.92	11.80 ± 39.90	4.64 ± 24.40	4.17 ± 23.38
			$\sigma = 0.1$	10.48 ± 39.21	11.78 ± 39.66	6.66 ± 31.01	12.01 ± 39.90	11.97 ± 39.88	5.52 ± 25.83	5.24 ± 25.67
	Average	kernel = 5	10.42 ± 39.28	10.54 ± 39.36	6.10 ± 32.62	11.77 ± 39.94	11.68 ± 39.93	5.16 ± 28.24	4.86 ± 28.09	
		kernel = 10	10.87 ± 39.63	11.46 ± 39.87	8.77 ± 38.79	11.73 ± 39.95	11.83 ± 39.92	7.53 ± 35.72	7.56 ± 37.34	
		kernel = 15	11.28 ± 39.76	11.82 ± 39.89	10.22 ± 39.77	11.71 ± 39.94	11.97 ± 39.90	9.03 ± 38.89	9.17 ± 39.37	
		kernel = 20	11.55 ± 39.79	11.98 ± 39.89	11.00 ± 39.94	11.71 ± 39.93	12.08 ± 39.88	10.02 ± 39.61	10.29 ± 39.81	
		kernel = 25	11.71 ± 39.84	12.07 ± 39.88	11.40 ± 39.96	11.71 ± 39.93	12.15 ± 39.87	10.76 ± 39.86	11.05 ± 39.94	
		kernel = 30	11.82 ± 39.82	12.12 ± 39.87	11.62 ± 39.95	11.73 ± 39.91	12.19 ± 39.86	11.26 ± 39.95	11.46 ± 39.96	
		Disk	kernel = 3	10.42 ± 39.29	10.64 ± 39.45	6.38 ± 33.41	11.76 ± 39.94	11.69 ± 39.93	5.36 ± 29.12	5.11 ± 29.18
			kernel = 6	10.92 ± 39.66	11.53 ± 39.89	9.08 ± 39.16	11.73 ± 39.95	11.84 ± 39.91	7.93 ± 37.11	7.92 ± 38.13
			kernel = 9	11.32 ± 39.77	11.86 ± 39.90	10.51 ± 39.87	11.71 ± 39.94	12.00 ± 39.90	9.36 ± 39.25	9.56 ± 39.59
			kernel = 12	11.59 ± 39.83	12.00 ± 39.89	11.19 ± 39.95	11.71 ± 39.94	12.11 ± 39.88	10.39 ± 39.76	10.66 ± 39.89
			kernel = 15	11.76 ± 39.86	12.08 ± 39.88	11.53 ± 39.96	11.71 ± 39.93	12.16 ± 39.86	11.11 ± 39.92	11.31 ± 39.96
		Gaussian $\sigma = 1$	kernel = 10	10.28 ± 39.10	10.00 ± 38.51	4.75 ± 27.07	11.78 ± 39.94	11.65 ± 39.93	4.35 ± 25.07	3.89 ± 24.16
			kernel = 20	10.28 ± 39.10	10.00 ± 38.51	4.75 ± 27.07	11.78 ± 39.94	11.65 ± 39.93	4.35 ± 25.07	3.89 ± 24.16
			kernel = 30	10.28 ± 39.10	10.00 ± 38.51	4.75 ± 27.07	11.78 ± 39.94	11.65 ± 39.93	4.35 ± 25.07	3.89 ± 24.16
		Gaussian $\sigma = 2$	kernel = 10	10.50 ± 39.39	10.80 ± 39.58	6.86 ± 34.74	11.75 ± 39.94	11.71 ± 39.93	5.83 ± 30.74	5.54 ± 30.86
	kernel = 20		10.51 ± 39.39	10.81 ± 39.59	6.90 ± 34.82	11.75 ± 39.94	11.72 ± 39.92	5.88 ± 30.85	5.58 ± 30.88	
	kernel = 30		10.51 ± 39.39	10.81 ± 39.59	6.90 ± 34.82	11.75 ± 39.94	11.72 ± 39.92	5.88 ± 30.85	5.58 ± 30.88	
	Gaussian $\sigma = 4$	kernel = 10	10.78 ± 39.58	11.34 ± 39.83	8.37 ± 38.10	11.73 ± 39.95	11.80 ± 39.92	7.32 ± 35.31	7.14 ± 36.21	
		kernel = 20	11.04 ± 39.71	11.65 ± 39.89	9.60 ± 39.49	11.71 ± 39.94	11.91 ± 39.91	8.54 ± 38.34	8.40 ± 36.21	
		kernel = 30	11.04 ± 39.69	11.66 ± 39.89	9.63 ± 39.51	11.71 ± 39.94	11.92 ± 39.91	8.58 ± 38.34	8.43 ± 38.75	
	Gaussian $\sigma = 8$	kernel = 10	10.86 ± 39.63	11.43 ± 39.86	8.69 ± 38.69	11.73 ± 39.94	11.82 ± 39.92	7.53 ± 35.78	7.48 ± 37.23	
		kernel = 20	11.44 ± 39.79	11.93 ± 39.89	10.89 ± 39.94	11.70 ± 39.94	12.05 ± 39.89	9.88 ± 39.57	10.10 ± 39.77	
		kernel = 30	11.67 ± 39.83	12.03 ± 39.88	11.41 ± 39.95	11.71 ± 39.94	12.13 ± 39.87	10.85 ± 39.89	11.06 ± 39.94	
	Motion $\theta = 0$	kernel = 10	10.64 ± 39.48	11.10 ± 39.74	7.28 ± 35.56	11.75 ± 39.94	11.66 ± 39.92	6.30 ± 32.28	6.13 ± 31.78	
		kernel = 20	11.09 ± 39.74	11.81 ± 39.88	9.69 ± 39.49	11.71 ± 39.94	11.86 ± 39.91	9.06 ± 39.16	9.10 ± 39.11	
		kernel = 30	11.36 ± 39.84	12.02 ± 39.88	10.78 ± 39.90	11.75 ± 39.93	12.03 ± 39.89	10.19 ± 39.81	10.46 ± 39.90	
	Motion $\theta = 15$	kernel = 10	10.68 ± 39.52	11.06 ± 39.73	7.32 ± 35.67	11.74 ± 39.94	11.67 ± 39.92	6.35 ± 32.24	6.12 ± 31.58	
		kernel = 20	11.18 ± 39.78	11.78 ± 39.87	9.87 ± 39.58	11.71 ± 39.94	11.85 ± 39.91	9.26 ± 39.31	9.28 ± 39.41	
		kernel = 30	11.51 ± 39.84	12.00 ± 39.88	11.04 ± 39.94	11.75 ± 39.93	12.02 ± 39.90	10.59 ± 39.92	10.77 ± 39.94	
	Motion $\theta = 30$	kernel = 10	10.68 ± 39.55	11.08 ± 39.71	7.69 ± 36.63	11.75 ± 39.94	11.70 ± 39.92	6.89 ± 34.47	6.63 ± 33.85	
		kernel = 20	11.26 ± 39.79	11.78 ± 39.89	10.36 ± 39.81	11.72 ± 39.94	11.90 ± 39.91	9.76 ± 39.63	9.84 ± 39.72	
		kernel = 30	11.64 ± 39.86	12.00 ± 39.88	11.38 ± 39.96	11.76 ± 39.93	12.06 ± 39.89	11.11 ± 39.97	11.24 ± 40.00	
	Motion $\theta = 45$	kernel = 10	10.68 ± 39.54	11.17 ± 39.75	7.89 ± 37.15	11.76 ± 39.94	11.73 ± 39.92	7.17 ± 35.33	6.95 ± 34.94	
		kernel = 20	11.32 ± 39.82	11.86 ± 39.90	10.52 ± 39.85	11.74 ± 39.94	11.94 ± 39.90	9.98 ± 39.70	10.09 ± 39.80	
		kernel = 30	11.69 ± 39.87	12.05 ± 39.88	11.43 ± 39.95	11.78 ± 39.93	12.09 ± 39.88	11.20 ± 39.93	11.35 ± 39.97	
	Motion $\theta = 60$	kernel = 10	10.67 ± 39.52	11.33 ± 39.81	8.10 ± 37.70	11.76 ± 39.94	11.76 ± 39.92	7.40 ± 35.54	7.11 ± 35.24	
		kernel = 20	11.31 ± 39.80	11.93 ± 39.89	10.55 ± 39.85	11.75 ± 39.94	11.97 ± 39.90	9.92 ± 39.61	9.95 ± 39.73	
		kernel = 30	11.67 ± 39.84	12.09 ± 39.87	11.49 ± 39.94	11.78 ± 39.93	12.11 ± 39.87	11.14 ± 39.91	11.24 ± 39.95	

images or frames from a video stream, predicting the locations of key facial landmarks like eyes, nose, mouth, and jawline [65]. Single Shot Detector with ResNet backbone (ResNet SSD) is a Caffe-based face detection model from the OpenCV Deep Neural Network module [66]. It is an object detection framework that predicts bounding boxes and class probabilities in a single pass. Using a ResNet backbone improves feature extraction, making it effective for face detection in diverse and cluttered scenes.

Tables 19–22 summarize the MAE $\pm$ SD results for people counting across original and degraded (blurred/noisy) versions of all four datasets. YOLOv9 consistently outperforms all other models across datasets and degradation

levels. Notably, YOLOv9 maintains low error rates under challenging image distortions, highlighting its robustness. RetinaFace and YOLOv12 show comparable performance to YOLOv9 under mild noise and blur but degrade faster under more severe distortions. Traditional models such as Haar Cascade and MTCNN yield significantly higher errors due to limited robustness to crowd density and image artifacts.

Fig. 29 qualitatively illustrates the performance of all seven models on a high-density image from the Wider Face dataset. While YOLOv12 detects more faces than other baselines, YOLOv9 achieves the best precision-recall balance, minimizing both missed detections and false positives. For instance, YOLOv9 detects 129 of 132 faces

TABLE 20. Performance comparison of benchmark face detection models in terms of MAE $\pm$ SD for face counting on the database with a mix of human and animal faces. Results demonstrate that the YOLOv9-based model achieved better performance compared to benchmark models.

Database	Image Type	Noise/Blur Level	Haar Cascade	MTCNN	RetinaFace	Mediapipe	ResNet SSD	YOLOv12	YOLOv9	
Humans with Animals	Org	Original	—	3.19 ± 5.40	1.94 ± 3.97	0.38 ± 0.79	4.30 ± 7.37	4.05 ± 7.40	0.60 ± 1.15	0.44 ± 0.86
	Noises	Gaussian	$\sigma = 0.01$	3.10 ± 5.61	3.26 ± 5.54	0.66 ± 1.41	4.38 ± 7.33	4.28 ± 7.36	0.69 ± 1.15	0.67 ± 0.94
			$\sigma = 0.05$	3.54 ± 6.10	4.50 ± 7.00	2.14 ± 4.49	4.67 ± 7.24	4.64 ± 7.24	2.33 ± 4.52	2.16 ± 4.15
			$\sigma = 0.1$	3.77 ± 6.70	4.76 ± 7.15	3.14 ± 5.58	4.75 ± 7.21	4.81 ± 7.18	4.24 ± 6.38	3.24 ± 5.64
		Salt & Pepper	$d = 0.01$	3.25 ± 5.57	2.36 ± 4.53	0.49 ± 0.92	4.35 ± 7.34	4.11 ± 7.39	0.58 ± 0.97	0.38 ± 0.76
			$d = 0.05$	3.31 ± 5.69	3.50 ± 5.82	0.84 ± 1.89	4.38 ± 7.36	4.35 ± 7.33	0.87 ± 1.78	0.68 ± 1.40
			$d = 0.1$	3.43 ± 6.16	4.21 ± 6.95	1.79 ± 3.99	4.65 ± 7.25	4.56 ± 7.27	1.88 ± 3.84	1.71 ± 3.49
		Speckle	$\sigma = 0.01$	3.04 ± 5.27	2.13 ± 4.13	0.42 ± 0.79	4.35 ± 7.35	4.09 ± 7.40	0.50 ± 0.93	0.46 ± 0.87
			$\sigma = 0.05$	3.16 ± 5.47	3.31 ± 5.27	0.58 ± 1.06	4.45 ± 7.32	4.31 ± 7.31	0.77 ± 1.29	0.67 ± 0.99
			$\sigma = 0.1$	3.22 ± 5.97	3.95 ± 5.99	0.96 ± 1.75	4.53 ± 7.31	4.48 ± 7.27	0.95 ± 1.53	1.07 ± 1.59
	Blur	Average	kernel = 5	3.31 ± 5.58	2.98 ± 5.49	0.77 ± 1.81	4.30 ± 7.36	4.27 ± 7.36	0.63 ± 1.34	0.63 ± 1.21
			kernel = 10	3.70 ± 6.49	4.13 ± 7.03	1.57 ± 3.57	4.31 ± 7.34	4.60 ± 7.27	1.42 ± 3.27	1.36 ± 3.53
			kernel = 15	3.95 ± 6.92	4.41 ± 7.08	2.67 ± 5.35	4.24 ± 7.30	4.67 ± 7.23	2.13 ± 4.04	2.22 ± 4.28
			kernel = 20	4.21 ± 7.06	4.60 ± 7.11	3.41 ± 6.62	4.23 ± 7.28	4.71 ± 7.23	2.72 ± 4.75	3.07 ± 5.38
			kernel = 25	4.30 ± 6.96	4.65 ± 7.13	3.87 ± 6.96	4.25 ± 7.30	4.73 ± 7.21	3.47 ± 6.42	3.73 ± 6.71
			kernel = 30	4.50 ± 7.12	4.71 ± 7.22	4.09 ± 7.16	4.27 ± 7.26	4.71 ± 7.22	3.93 ± 7.07	4.10 ± 7.12
		Disk	kernel = 3	3.40 ± 5.53	3.06 ± 5.64	0.87 ± 2.15	4.30 ± 7.36	4.32 ± 7.36	0.73 ± 1.66	0.65 ± 1.42
			kernel = 6	3.66 ± 6.44	4.17 ± 7.03	1.68 ± 3.84	4.39 ± 7.35	4.60 ± 7.27	1.58 ± 3.63	1.44 ± 3.57
			kernel = 9	4.03 ± 6.86	4.44 ± 7.07	2.90 ± 5.80	4.19 ± 7.30	4.72 ± 7.22	2.37 ± 4.32	2.54 ± 4.69
			kernel = 12	4.19 ± 7.06	4.60 ± 7.11	3.62 ± 6.76	4.16 ± 7.22	4.71 ± 7.23	3.11 ± 5.29	3.36 ± 5.87
			kernel = 15	4.37 ± 7.05	4.71 ± 7.13	3.99 ± 7.17	4.24 ± 7.27	4.73 ± 7.21	3.81 ± 6.94	4.03 ± 7.02
		Gaussian $\sigma = 1$	kernel = 10	3.21 ± 5.55	2.42 ± 4.55	0.53 ± 0.98	4.28 ± 7.37	4.18 ± 7.37	0.54 ± 1.04	0.48 ± 0.88
			kernel = 20	3.21 ± 5.55	2.42 ± 4.55	0.53 ± 0.98	4.28 ± 7.37	4.18 ± 7.37	0.54 ± 1.04	0.48 ± 0.88
			kernel = 30	3.21 ± 5.55	2.42 ± 4.55	0.53 ± 0.98	4.28 ± 7.37	4.18 ± 7.37	0.54 ± 1.04	0.48 ± 0.88
		Gaussian $\sigma = 2$	kernel = 10	3.41 ± 5.86	3.31 ± 5.95	0.87 ± 2.28	4.29 ± 7.38	4.39 ± 7.34	0.83 ± 2.12	0.77 ± 1.78
			kernel = 20	3.34 ± 5.81	3.29 ± 5.93	0.95 ± 2.42	4.29 ± 7.38	4.37 ± 7.35	0.78 ± 1.90	0.77 ± 1.91
			kernel = 30	3.34 ± 5.81	3.29 ± 5.93	0.95 ± 2.42	4.29 ± 7.38	4.37 ± 7.35	0.78 ± 1.90	0.77 ± 1.91
		Gaussian $\sigma = 4$	kernel = 10	3.68 ± 6.28	3.94 ± 6.83	1.42 ± 3.44	4.34 ± 7.35	4.59 ± 7.27	1.35 ± 3.18	1.28 ± 3.44
			kernel = 20	3.81 ± 6.71	4.28 ± 6.93	2.14 ± 4.44	4.28 ± 7.28	4.64 ± 7.25	1.91 ± 3.89	1.85 ± 3.94
			kernel = 30	3.78 ± 6.73	4.28 ± 6.91	2.20 ± 4.48	4.27 ± 7.28	4.65 ± 7.25	1.87 ± 3.85	1.89 ± 3.93
		Gaussian $\sigma = 8$	kernel = 10	3.68 ± 6.47	4.09 ± 6.98	1.50 ± 3.53	4.35 ± 7.34	4.59 ± 7.27	1.42 ± 3.39	1.39 ± 3.53
			kernel = 20	3.99 ± 6.98	4.55 ± 7.11	3.34 ± 6.50	4.26 ± 7.27	4.73 ± 7.22	2.75 ± 4.73	2.98 ± 5.18
			kernel = 30	4.26 ± 7.02	4.62 ± 7.14	3.91 ± 7.15	4.21 ± 7.31	4.73 ± 7.22	3.61 ± 6.41	3.83 ± 6.64
		Motion $\theta = 0$	kernel = 10	3.73 ± 6.28	3.84 ± 6.88	1.36 ± 3.30	4.30 ± 7.37	4.28 ± 7.38	1.32 ± 3.04	1.29 ± 3.24
			kernel = 20	3.74 ± 6.65	4.40 ± 7.12	2.47 ± 5.13	4.33 ± 7.32	4.55 ± 7.28	2.48 ± 5.23	2.49 ± 4.99
			kernel = 30	3.96 ± 6.84	4.65 ± 7.18	3.58 ± 6.83	4.40 ± 7.31	4.64 ± 7.24	3.22 ± 6.91	3.61 ± 6.98
		Motion $\theta = 15$	kernel = 10	3.59 ± 6.02	3.69 ± 6.63	1.38 ± 3.35	4.26 ± 7.38	4.26 ± 7.37	1.25 ± 3.03	1.28 ± 3.25
			kernel = 20	3.80 ± 6.88	4.36 ± 7.08	2.81 ± 5.79	4.29 ± 7.35	4.56 ± 7.28	2.54 ± 5.36	2.54 ± 5.27
			kernel = 30	4.05 ± 6.89	4.63 ± 7.19	3.84 ± 7.13	4.37 ± 7.33	4.60 ± 7.27	3.65 ± 7.12	3.76 ± 7.15
		Motion $\theta = 30$	kernel = 10	3.54 ± 6.07	3.75 ± 6.75	1.45 ± 3.45	4.28 ± 7.38	4.32 ± 7.36	1.37 ± 3.38	1.21 ± 3.43
			kernel = 20	3.79 ± 6.92	4.46 ± 7.13	3.14 ± 6.42	4.30 ± 7.31	4.60 ± 7.26	2.80 ± 5.52	2.94 ± 5.71
			kernel = 30	4.23 ± 7.11	4.58 ± 7.12	4.01 ± 7.19	4.47 ± 7.25	4.71 ± 7.22	3.94 ± 7.15	3.99 ± 7.11
		Motion $\theta = 45$	kernel = 10	3.57 ± 6.04	3.76 ± 6.62	1.37 ± 3.40	4.32 ± 7.35	4.43 ± 7.33	1.27 ± 3.28	1.18 ± 3.48
			kernel = 20	3.97 ± 6.99	4.48 ± 7.08	3.25 ± 6.33	4.32 ± 7.39	4.66 ± 7.24	2.85 ± 5.58	3.10 ± 5.85
			kernel = 30	4.28 ± 7.08	4.61 ± 7.11	3.98 ± 7.19	4.45 ± 7.29	4.73 ± 7.21	3.95 ± 7.10	4.06 ± 7.00
		Motion $\theta = 60$	kernel = 10	3.55 ± 5.90	3.84 ± 6.65	1.35 ± 3.35	4.30 ± 7.36	4.46 ± 7.32	1.17 ± 2.81	1.24 ± 3.37
			kernel = 20	3.91 ± 7.03	4.51 ± 7.07	3.24 ± 6.24	4.38 ± 7.32	4.66 ± 7.24	2.80 ± 5.20	2.80 ± 5.18
			kernel = 30	4.29 ± 7.11	4.63 ± 7.06	4.08 ± 7.16	4.37 ± 7.36	4.73 ± 7.22	3.86 ± 7.11	3.99 ± 6.94

with only one false positive, whereas YOLOv12 detects 124 faces with four false positives.

In addition to accuracy, inference time is critical in real-time systems. Table 23 compares the average execution time per image for each model. Mediapipe is the fastest due to its lightweight architecture, followed by ResNet SSD and Haar Cascade. Among deep learning detectors, YOLOv12 and YOLOv9 offer real-time performance, processing each image in under 16 ms. Although RetinaFace achieves high precision, its inference time exceeds 2 seconds per image, making it less suitable for time-sensitive applications.

Overall, the YOLOv9-based face detection and counting method achieves the best trade-off between accuracy,

robustness under distortion, and runtime efficiency. These qualities make it particularly suitable for deployment in dense crowd analytics, surveillance, and infant monitoring systems.

P. REAL-TIME IMPLEMENTATION OF YOLOV9-BASED FACE DETECTION AND PEOPLE COUNTING MODEL

To demonstrate the feasibility of deployment, the YOLOv9-based face detection and people counting method was implemented on the Raspberry Pi device, which is considered as an edge computing platform. Live images were collected for testing by interfacing the Raspberry Pi with an RGB camera. The experimental setup for image acquisition and face detection is shown in Fig. 30. The Raspberry Pi Model 4B has a 1.8GHz

TABLE 21. Performance comparison of benchmark face detection models in terms of MAE $\pm$ SD for face counting on the database with human faces inside vehicles. Results demonstrate that the YOLOv9-based model achieved better performance compared to benchmark models.

Database	Image Type	Noise/Blur Level	Haar Cascade	MTCNN	RetinaFace	Mediapipe	ResNet SSD	YOLOv12	YOLOv9	
Humans inside Vehicles	Org	Original	—	2.39 ± 4.59	2.39 ± 4.22	0.44 ± 1.63	3.23 ± 5.18	3.05 ± 5.00	0.32 ± 1.21	0.24 ± 0.76
	Noises	Gaussian	$\sigma = 0.01$	2.42 ± 4.59	3.10 ± 4.68	0.58 ± 2.47	3.25 ± 5.10	3.31 ± 4.97	0.78 ± 1.98	0.78 ± 2.24
			$\sigma = 0.05$	2.77 ± 4.75	3.80 ± 4.94	1.92 ± 4.32	3.72 ± 5.04	3.77 ± 4.96	2.41 ± 4.50	2.13 ± 4.36
			$\sigma = 0.1$	3.10 ± 4.83	3.90 ± 4.97	2.83 ± 4.73	3.89 ± 4.98	3.90 ± 4.97	3.27 ± 4.89	3.07 ± 4.76
		Salt & Pepper	$d = 0.01$	2.42 ± 4.58	2.51 ± 4.19	0.42 ± 1.61	3.24 ± 5.15	3.16 ± 5.00	0.38 ± 1.13	0.24 ± 0.95
			$d = 0.05$	2.45 ± 4.67	3.27 ± 4.83	0.82 ± 2.57	3.38 ± 5.14	3.47 ± 4.94	1.00 ± 2.80	0.88 ± 2.48
			$d = 0.1$	2.69 ± 4.71	3.71 ± 4.89	1.59 ± 3.78	3.58 ± 5.09	3.69 ± 4.88	1.91 ± 4.26	1.55 ± 4.07
		Speckle	$\sigma = 0.01$	2.44 ± 4.63	2.50 ± 4.23	0.40 ± 1.67	3.20 ± 5.18	3.14 ± 4.97	0.39 ± 1.45	0.26 ± 0.91
			$\sigma = 0.05$	2.47 ± 4.47	3.20 ± 4.82	0.54 ± 1.93	3.35 ± 5.14	3.44 ± 4.91	0.65 ± 1.92	0.65 ± 1.92
			$\sigma = 0.1$	2.64 ± 4.46	3.49 ± 4.81	0.90 ± 2.77	3.50 ± 5.10	3.62 ± 4.92	1.34 ± 2.90	1.03 ± 2.91
	Blur	Average	kernel = 5	2.52 ± 4.59	2.71 ± 4.72	0.67 ± 2.91	3.19 ± 5.16	3.24 ± 4.98	0.56 ± 2.39	0.43 ± 2.13
			kernel = 10	2.63 ± 4.63	3.32 ± 4.91	1.18 ± 3.98	3.20 ± 5.09	3.56 ± 4.99	1.15 ± 3.76	1.05 ± 3.98
			kernel = 15	2.97 ± 4.72	3.61 ± 4.96	1.74 ± 4.54	3.24 ± 5.04	3.68 ± 5.00	1.71 ± 4.38	1.59 ± 4.35
			kernel = 20	3.26 ± 4.94	3.70 ± 5.01	2.28 ± 4.74	3.27 ± 5.01	3.81 ± 4.98	2.13 ± 4.49	2.03 ± 4.55
			kernel = 25	3.45 ± 4.94	3.78 ± 4.98	2.82 ± 4.92	3.33 ± 4.99	3.83 ± 4.97	2.53 ± 4.76	2.47 ± 4.73
			kernel = 30	3.45 ± 4.96	3.85 ± 4.98	3.17 ± 5.00	3.26 ± 4.97	3.87 ± 5.00	2.87 ± 4.79	2.84 ± 4.75
		Disk	kernel = 3	2.66 ± 4.65	2.82 ± 4.72	0.64 ± 2.91	3.21 ± 5.16	3.26 ± 4.98	0.57 ± 2.56	0.50 ± 2.52
			kernel = 6	2.69 ± 4.55	3.39 ± 4.93	1.28 ± 4.31	3.17 ± 5.10	3.57 ± 5.01	1.33 ± 4.15	1.23 ± 4.25
			kernel = 9	2.96 ± 4.76	3.63 ± 4.97	1.85 ± 4.44	3.24 ± 5.05	3.70 ± 5.02	1.86 ± 4.41	1.72 ± 4.41
			kernel = 12	3.27 ± 4.99	3.74 ± 5.00	2.49 ± 4.77	3.24 ± 5.00	3.81 ± 4.98	2.24 ± 4.60	2.15 ± 4.59
			kernel = 15	3.48 ± 4.97	3.81 ± 4.97	2.98 ± 4.93	3.30 ± 5.01	3.85 ± 4.98	2.59 ± 4.70	2.73 ± 4.76
		Gaussian $\sigma = 1$	kernel = 10	2.39 ± 4.55	2.43 ± 4.50	0.41 ± 2.00	3.20 ± 5.15	3.15 ± 4.99	0.47 ± 2.09	0.28 ± 1.62
			kernel = 20	2.39 ± 4.55	2.43 ± 4.50	0.41 ± 2.00	3.20 ± 5.15	3.15 ± 4.99	0.47 ± 2.09	0.28 ± 1.62
			kernel = 30	2.39 ± 4.55	2.43 ± 4.50	0.41 ± 2.00	3.20 ± 5.15	3.15 ± 4.99	0.47 ± 2.09	0.28 ± 1.62
		Gaussian $\sigma = 2$	kernel = 10	2.49 ± 4.55	2.96 ± 4.78	0.67 ± 2.89	3.17 ± 5.16	3.29 ± 4.99	0.68 ± 2.82	0.57 ± 2.58
			kernel = 20	2.50 ± 4.59	2.95 ± 4.79	0.71 ± 3.00	3.17 ± 5.15	3.29 ± 4.99	0.66 ± 2.82	0.58 ± 2.67
			kernel = 30	2.50 ± 4.59	2.95 ± 4.79	0.71 ± 3.00	3.17 ± 5.15	3.29 ± 4.99	0.67 ± 2.82	0.58 ± 2.67
		Gaussian $\sigma = 4$	kernel = 10	2.62 ± 4.71	3.20 ± 4.85	1.07 ± 3.76	3.20 ± 5.08	3.50 ± 4.98	0.99 ± 3.62	1.00 ± 3.87
	kernel = 20		2.77 ± 4.71	3.50 ± 4.91	1.50 ± 4.38	3.16 ± 5.07	3.64 ± 4.99	1.55 ± 4.34	1.34 ± 4.26	
	kernel = 30		2.88 ± 4.68	3.54 ± 4.94	1.48 ± 4.39	3.20 ± 5.05	3.64 ± 4.99	1.56 ± 4.34	1.34 ± 4.26	
	Gaussian $\sigma = 8$	kernel = 10	2.68 ± 4.64	3.28 ± 4.87	1.16 ± 4.07	3.20 ± 5.09	3.54 ± 4.99	1.11 ± 3.64	1.06 ± 3.97	
		kernel = 20	3.08 ± 4.86	3.72 ± 4.94	2.13 ± 4.51	3.20 ± 5.04	3.76 ± 5.00	2.09 ± 4.50	1.92 ± 4.50	
		kernel = 30	3.30 ± 4.97	3.79 ± 4.97	2.77 ± 4.84	3.26 ± 5.00	3.82 ± 4.99	2.44 ± 4.72	2.39 ± 4.72	
	Motion $\theta = 0$	kernel = 10	2.60 ± 4.52	3.06 ± 4.76	0.86 ± 3.50	3.14 ± 5.16	3.23 ± 5.00	0.79 ± 3.24	0.74 ± 3.15	
		kernel = 20	2.89 ± 4.75	3.55 ± 4.96	1.53 ± 4.27	3.25 ± 5.11	3.51 ± 4.99	1.69 ± 4.40	1.63 ± 4.34	
		kernel = 30	3.04 ± 4.92	3.71 ± 5.02	2.21 ± 4.68	3.32 ± 5.11	3.58 ± 4.99	2.22 ± 4.68	2.21 ± 4.67	
	Motion $\theta = 15$	kernel = 10	2.71 ± 4.60	2.95 ± 4.71	0.85 ± 3.68	3.18 ± 5.18	3.25 ± 4.98	0.81 ± 3.20	0.70 ± 3.32	
		kernel = 20	2.94 ± 4.69	3.52 ± 4.93	1.81 ± 4.55	3.26 ± 5.12	3.51 ± 4.99	1.67 ± 4.47	1.67 ± 4.35	
		kernel = 30	3.37 ± 4.93	3.71 ± 4.98	2.43 ± 4.88	3.37 ± 5.10	3.64 ± 4.98	2.46 ± 4.76	2.40 ± 4.79	
	Motion $\theta = 30$	kernel = 10	2.65 ± 4.58	3.01 ± 4.79	0.90 ± 3.60	3.12 ± 5.18	3.30 ± 4.99	0.87 ± 3.76	0.76 ± 3.65	
		kernel = 20	2.95 ± 4.77	3.53 ± 4.93	1.95 ± 4.68	3.29 ± 5.14	3.62 ± 5.00	1.87 ± 4.49	1.89 ± 4.52	
		kernel = 30	3.36 ± 4.98	3.71 ± 4.95	2.80 ± 4.98	3.38 ± 5.05	3.77 ± 4.99	2.74 ± 4.86	2.74 ± 4.96	
	Motion $\theta = 45$	kernel = 10	2.67 ± 4.61	3.13 ± 4.82	0.91 ± 3.76	3.22 ± 5.17	3.37 ± 4.98	0.81 ± 3.75	0.83 ± 3.74	
		kernel = 20	3.00 ± 4.81	3.60 ± 4.92	2.10 ± 4.76	3.31 ± 5.09	3.67 ± 5.01	2.00 ± 4.45	2.04 ± 4.58	
		kernel = 30	3.35 ± 5.00	3.76 ± 4.95	2.91 ± 4.99	3.35 ± 5.04	3.80 ± 4.97	2.70 ± 4.85	2.87 ± 4.92	
	Motion $\theta = 60$	kernel = 10	2.69 ± 4.62	3.28 ± 4.86	1.06 ± 4.03	3.24 ± 5.16	3.44 ± 4.96	0.91 ± 3.84	0.88 ± 3.93	
		kernel = 20	3.05 ± 4.79	3.75 ± 4.96	2.14 ± 4.72	3.32 ± 5.10	3.67 ± 5.04	2.02 ± 4.52	2.00 ± 4.45	
		kernel = 30	3.34 ± 4.98	3.81 ± 4.94	3.03 ± 5.01	3.39 ± 5.08	3.83 ± 5.00	2.68 ± 4.84	2.83 ± 4.89	

64-bit Arm Cortex-A72 quad-core processor and 8GB RAM. The Raspberry Pi is equipped with the PyTorch, Ultralytics, and Pandas library packages, and the Jupyter Notebook IDE. The camera used was the Raspberry Pi Camera Module 3, which has a 12MP Sony IMX708 sensor. The camera has a diagonal angle of view of 75° . The implementation of the YOLOv9-based face detection and people counting method on the Raspberry Pi shows the adaptability of the model for real-time applications with limited computing resources. This edge implementation can be extended to smart surveillance, mobile monitoring units, or wearable diagnostic systems, where data privacy and offline processing are critical.

Table 24 summarizes the performance of the YOLOv9-based

model on three implementation platforms, namely, Tesla V100-SXM3-32GB GPU, a personal computer with an 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz processor and 16GB RAM, and the Raspberry Pi Model 4B. The model was tested using the 3221 images of the Wider Face database. The results confirm that the face detection and counting performance, in terms of accuracy metrics, remains consistent across platforms, demonstrating the robustness and platform independence of the model. However, a notable variation was observed in inference times. The PC-CPU and Raspberry Pi reported inference times of 457.1ms and 6060.9ms per image, respectively. The implementation on Raspberry Pi

TABLE 22. Performance comparison of benchmark face detection models in terms of MAE $\pm$ SD for face counting on the infant face database. Results demonstrate that the YOLOv9-based model achieved better performance compared to benchmark models.

Database	Image Type	Noise/Blur Level	Haar Cascade	MTCNN	RetinaFace	Mediapipe	ResNet SSD	YOLOv12	YOLOv9
Infant Faces	Org	Original	4.30 $\pm$ 3.97	3.87 $\pm$ 3.80	0.38 $\pm$ 1.35	3.96 $\pm$ 4.23	3.73 $\pm$ 3.84	0.40 $\pm$ 1.19	0.26 $\pm$ 1.04
	Noises	Gaussian	$\sigma = 0.01$	4.55 $\pm$ 4.17	5.21 $\pm$ 4.28	0.96 $\pm$ 2.04	4.65 $\pm$ 4.43	4.52 $\pm$ 4.05	1.41 $\pm$ 2.42
			$\sigma = 0.05$	5.04 $\pm$ 4.26	6.11 $\pm$ 4.07	4.49 $\pm$ 3.76	5.65 $\pm$ 4.27	6.10 $\pm$ 4.07	5.06 $\pm$ 4.07
			$\sigma = 0.1$	5.41 $\pm$ 4.32	6.17 $\pm$ 4.07	5.64 $\pm$ 4.10	6.04 $\pm$ 4.11	6.14 $\pm$ 4.02	5.99 $\pm$ 4.16
		Salt & Pepper	$d = 0.01$	4.35 $\pm$ 4.02	4.34 $\pm$ 4.09	0.61 $\pm$ 1.58	4.13 $\pm$ 4.26	3.99 $\pm$ 3.93	0.75 $\pm$ 1.43
			$d = 0.05$	4.59 $\pm$ 4.20	5.44 $\pm$ 4.27	1.84 $\pm$ 2.77	4.88 $\pm$ 4.45	5.02 $\pm$ 4.23	1.94 $\pm$ 3.21
			$d = 0.1$	4.88 $\pm$ 4.32	5.98 $\pm$ 4.12	4.05 $\pm$ 4.11	5.43 $\pm$ 4.35	5.94 $\pm$ 4.11	4.34 $\pm$ 4.02
		Speckle	$\sigma = 0.01$	4.35 $\pm$ 4.07	4.60 $\pm$ 4.24	0.65 $\pm$ 1.60	4.27 $\pm$ 4.35	4.06 $\pm$ 3.94	0.79 $\pm$ 1.61
			$\sigma = 0.05$	4.70 $\pm$ 4.24	5.42 $\pm$ 4.29	2.19 $\pm$ 3.13	4.89 $\pm$ 4.41	5.07 $\pm$ 4.36	2.26 $\pm$ 3.26
			$\sigma = 0.1$	4.96 $\pm$ 4.33	5.81 $\pm$ 4.19	3.55 $\pm$ 3.96	5.34 $\pm$ 4.40	5.83 $\pm$ 4.14	3.77 $\pm$ 4.04
	Blur	Average	kernel = 5	4.35 $\pm$ 4.11	4.77 $\pm$ 4.18	0.96 $\pm$ 2.18	4.04 $\pm$ 4.11	5.11 $\pm$ 4.43	0.78 $\pm$ 2.05
			kernel = 10	5.02 $\pm$ 4.17	5.57 $\pm$ 4.22	2.62 $\pm$ 3.79	4.87 $\pm$ 4.28	5.70 $\pm$ 4.20	3.29 $\pm$ 4.09
			kernel = 15	5.58 $\pm$ 4.11	5.86 $\pm$ 4.17	3.83 $\pm$ 4.01	5.25 $\pm$ 4.27	5.70 $\pm$ 4.10	4.96 $\pm$ 4.23
			kernel = 20	5.81 $\pm$ 4.02	5.96 $\pm$ 4.13	4.87 $\pm$ 4.02	5.48 $\pm$ 4.26	5.88 $\pm$ 4.17	5.47 $\pm$ 4.33
			kernel = 25	5.91 $\pm$ 4.09	6.02 $\pm$ 4.11	5.18 $\pm$ 4.07	5.61 $\pm$ 4.29	5.96 $\pm$ 4.22	5.58 $\pm$ 4.31
			kernel = 30	6.01 $\pm$ 4.09	6.05 $\pm$ 4.12	5.50 $\pm$ 4.20	5.68 $\pm$ 4.29	6.05 $\pm$ 4.12	5.73 $\pm$ 4.31
		Disk	kernel = 3	4.50 $\pm$ 4.15	4.82 $\pm$ 4.21	1.15 $\pm$ 2.48	4.05 $\pm$ 4.07	5.21 $\pm$ 4.39	0.96 $\pm$ 2.19
			kernel = 6	5.14 $\pm$ 4.21	5.62 $\pm$ 4.18	2.89 $\pm$ 3.80	4.88 $\pm$ 4.24	5.71 $\pm$ 4.11	3.76 $\pm$ 4.16
			kernel = 9	5.65 $\pm$ 4.11	5.89 $\pm$ 4.15	4.12 $\pm$ 4.04	5.22 $\pm$ 4.29	5.70 $\pm$ 4.11	5.05 $\pm$ 4.17
			kernel = 12	5.85 $\pm$ 4.04	5.96 $\pm$ 4.11	4.96 $\pm$ 4.03	5.48 $\pm$ 4.26	5.85 $\pm$ 4.21	5.53 $\pm$ 4.33
			kernel = 15	5.96 $\pm$ 4.12	6.01 $\pm$ 4.11	5.36 $\pm$ 4.17	5.70 $\pm$ 4.29	5.96 $\pm$ 4.19	5.62 $\pm$ 4.31
		Gaussian $\sigma = 1$	kernel = 10	4.37 $\pm$ 4.09	4.25 $\pm$ 4.00	0.59 $\pm$ 1.70	3.94 $\pm$ 4.22	4.73 $\pm$ 4.36	0.36 $\pm$ 1.52
			kernel = 20	4.37 $\pm$ 4.09	4.25 $\pm$ 4.00	0.59 $\pm$ 1.70	3.94 $\pm$ 4.22	4.73 $\pm$ 4.36	0.36 $\pm$ 1.52
			kernel = 30	4.37 $\pm$ 4.09	4.25 $\pm$ 4.00	0.59 $\pm$ 1.70	3.94 $\pm$ 4.22	4.73 $\pm$ 4.36	0.36 $\pm$ 1.52
		Gaussian $\sigma = 2$	kernel = 10	4.63 $\pm$ 4.19	5.02 $\pm$ 4.12	1.42 $\pm$ 2.68	4.20 $\pm$ 4.14	5.29 $\pm$ 4.40	1.47 $\pm$ 2.75
			kernel = 20	4.60 $\pm$ 4.13	5.04 $\pm$ 4.13	1.32 $\pm$ 2.59	4.22 $\pm$ 4.16	5.31 $\pm$ 4.38	1.53 $\pm$ 2.91
			kernel = 30	4.60 $\pm$ 4.13	5.04 $\pm$ 4.13	1.32 $\pm$ 2.59	4.22 $\pm$ 4.16	5.31 $\pm$ 4.38	1.53 $\pm$ 2.91
		Gaussian $\sigma = 4$	kernel = 10	4.96 $\pm$ 4.10	5.50 $\pm$ 4.18	2.52 $\pm$ 3.64	4.72 $\pm$ 4.27	5.65 $\pm$ 4.19	3.20 $\pm$ 4.14
			kernel = 20	5.31 $\pm$ 4.25	5.77 $\pm$ 4.17	3.35 $\pm$ 3.87	5.08 $\pm$ 4.27	5.72 $\pm$ 4.08	4.49 $\pm$ 4.22
			kernel = 30	5.31 $\pm$ 4.22	5.79 $\pm$ 4.17	3.41 $\pm$ 3.93	5.12 $\pm$ 4.27	5.72 $\pm$ 4.08	4.50 $\pm$ 4.25
		Gaussian $\sigma = 8$	kernel = 10	5.06 $\pm$ 4.14	5.52 $\pm$ 4.24	2.56 $\pm$ 3.75	4.81 $\pm$ 4.22	5.67 $\pm$ 4.17	3.35 $\pm$ 4.16
			kernel = 20	5.72 $\pm$ 4.05	5.93 $\pm$ 4.17	4.65 $\pm$ 3.88	5.43 $\pm$ 4.30	5.78 $\pm$ 4.15	5.42 $\pm$ 4.23
			kernel = 30	5.88 $\pm$ 4.04	6.00 $\pm$ 4.12	5.26 $\pm$ 4.00	5.58 $\pm$ 4.32	5.93 $\pm$ 4.21	5.56 $\pm$ 4.29
		Motion $\theta = 0$	kernel = 10	4.87 $\pm$ 4.16	5.37 $\pm$ 4.13	1.96 $\pm$ 3.12	4.59 $\pm$ 4.30	5.12 $\pm$ 4.37	1.36 $\pm$ 2.74
			kernel = 20	5.57 $\pm$ 4.09	5.88 $\pm$ 4.12	4.41 $\pm$ 3.97	5.38 $\pm$ 4.39	5.70 $\pm$ 4.23	3.93 $\pm$ 4.12
			kernel = 30	5.85 $\pm$ 4.02	6.04 $\pm$ 4.11	5.33 $\pm$ 4.04	5.74 $\pm$ 4.26	5.72 $\pm$ 4.26	5.19 $\pm$ 4.19
		Motion $\theta = 15$	kernel = 10	4.81 $\pm$ 4.16	5.38 $\pm$ 4.18	1.96 $\pm$ 3.05	4.56 $\pm$ 4.28	5.14 $\pm$ 4.41	1.51 $\pm$ 2.89
			kernel = 20	5.60 $\pm$ 4.07	5.86 $\pm$ 4.14	4.65 $\pm$ 4.11	5.32 $\pm$ 4.32	5.65 $\pm$ 4.17	4.34 $\pm$ 4.10
			kernel = 30	5.86 $\pm$ 3.99	6.00 $\pm$ 4.13	5.50 $\pm$ 4.15	5.70 $\pm$ 4.25	5.67 $\pm$ 4.16	5.20 $\pm$ 4.30
		Motion $\theta = 30$	kernel = 10	4.87 $\pm$ 4.21	5.43 $\pm$ 4.10	2.25 $\pm$ 3.27	4.59 $\pm$ 4.31	5.35 $\pm$ 4.36	2.12 $\pm$ 3.30
			kernel = 20	5.60 $\pm$ 4.10	5.89 $\pm$ 4.13	4.88 $\pm$ 4.28	5.37 $\pm$ 4.34	5.79 $\pm$ 4.15	4.99 $\pm$ 4.28
			kernel = 30	5.89 $\pm$ 4.01	6.03 $\pm$ 4.13	5.61 $\pm$ 4.32	5.73 $\pm$ 4.24	5.86 $\pm$ 4.12	5.55 $\pm$ 4.38
		Motion $\theta = 45$	kernel = 10	4.78 $\pm$ 4.16	5.55 $\pm$ 4.15	2.50 $\pm$ 3.44	4.50 $\pm$ 4.31	5.42 $\pm$ 4.28	2.62 $\pm$ 3.70
			kernel = 20	5.57 $\pm$ 4.16	5.96 $\pm$ 4.12	4.88 $\pm$ 4.30	5.34 $\pm$ 4.33	5.77 $\pm$ 4.24	5.23 $\pm$ 4.18
			kernel = 30	5.92 $\pm$ 4.07	6.04 $\pm$ 4.11	5.60 $\pm$ 4.37	5.58 $\pm$ 4.30	5.93 $\pm$ 4.14	5.62 $\pm$ 4.30
		Motion $\theta = 60$	kernel = 10	4.81 $\pm$ 4.16	5.60 $\pm$ 4.21	2.94 $\pm$ 3.58	4.52 $\pm$ 4.25	5.42 $\pm$ 4.29	2.94 $\pm$ 3.80
			kernel = 20	5.57 $\pm$ 4.16	5.96 $\pm$ 4.14	4.99 $\pm$ 4.25	5.04 $\pm$ 4.27	5.81 $\pm$ 4.27	5.16 $\pm$ 4.21
			kernel = 30	5.87 $\pm$ 4.05	6.05 $\pm$ 4.09	5.53 $\pm$ 4.31	5.47 $\pm$ 4.24	5.95 $\pm$ 4.20	5.64 $\pm$ 4.35

TABLE 23. Comparison of different benchmark face detection models in terms of execution time per image.

Model	Execution Time (ms)
Haar Cascade	13.2
MTCNN	620.3
RetinaFace	2182.5
Mediapipe	4.2
ResNet SSD	11.3
YOLOv12	11.7
YOLOv9	15.2

underscores the model's potential for portable, on-device face analytics in constrained environments.

DISCUSSION

The proposed YOLOv9-based face detection and people counting model demonstrated strong accuracy across the Wider Face database and real-world test cases. It maintained high detection rates across diverse human appearances, including variations in skin tone, background complexity, scale, pose, and lighting conditions. Notably, the model was also effective in detecting infant faces and faces through partial occlusions such as vehicle windscreens, indicating good generalization beyond the original training distribution.

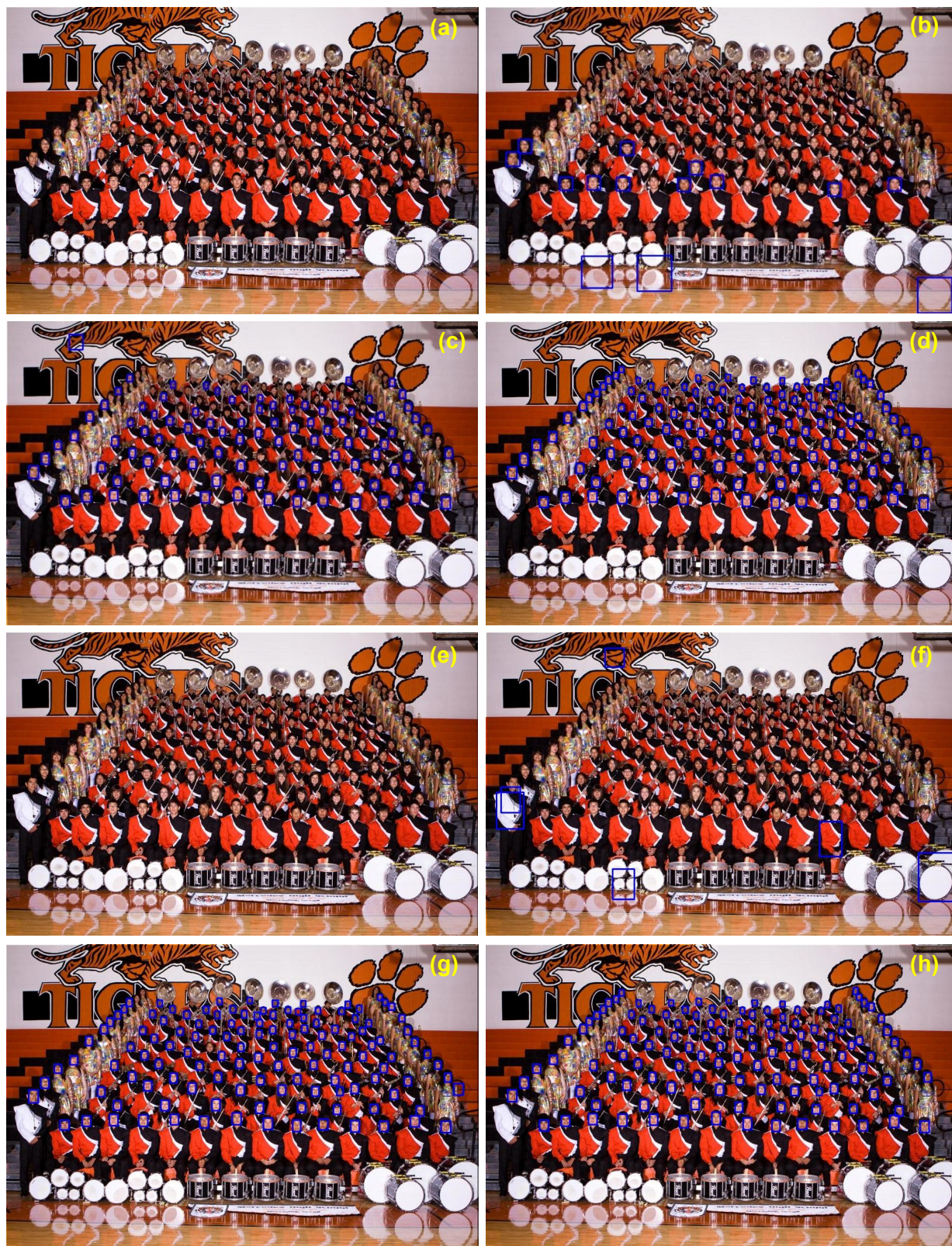


FIGURE 29. Illustrates the performance of the different benchmark face detection models for a confidence score of 0.15: (a) Original image with 132 faces; (b) Haar Cascade with 11 true positives and 3 false positives; (c) MTCNN with 96 true positives and 1 false positive; (d) RetinaFace with 126 true positives; (e) Mediapipe with zero detections; (f) ResNet SSD with zero true positives and 6 false positives; (g) YOLOv12 with 124 true positives and 4 false positives; (h) YOLOv9 achieves the best results with 129 true positives and 1 false positive. The original image is taken from the Wider Face database. YOLOv9 outperforms others in dense scenes, balancing precision and recall effectively, making it highly suitable for crowd analytics.

TABLE 24. Performance comparison of the YOLOv9-based face detection and people counting model on different implementation platforms

Implementation Platform	Database	N	Face Detection				Counting		Model Size (MB)	PT (ms)
			Precision	Recall	mAP50	mAP50-95	MAE $\pm$ SD	RMSE		
GPU	Wider Face	3221	0.860	0.628	0.708	0.368	3.36 $\pm$ 22.20	22.38	49.2	15.2
PC-CPU			0.855	0.625	0.705	0.361	3.36 $\pm$ 22.20	22.38	49.1	457.1
R-Pi			0.855	0.625	0.705	0.361	3.36 $\pm$ 22.20	22.38	49.1	6060.9

GPU: Tesla V100-SXM3-32GB; PC-CPU: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz 16GB RAM; R-Pi: Raspberry Pi 4B, Quad core Cortex-A72 8GB RAM; N: Number of test segments; PT: Processing time.

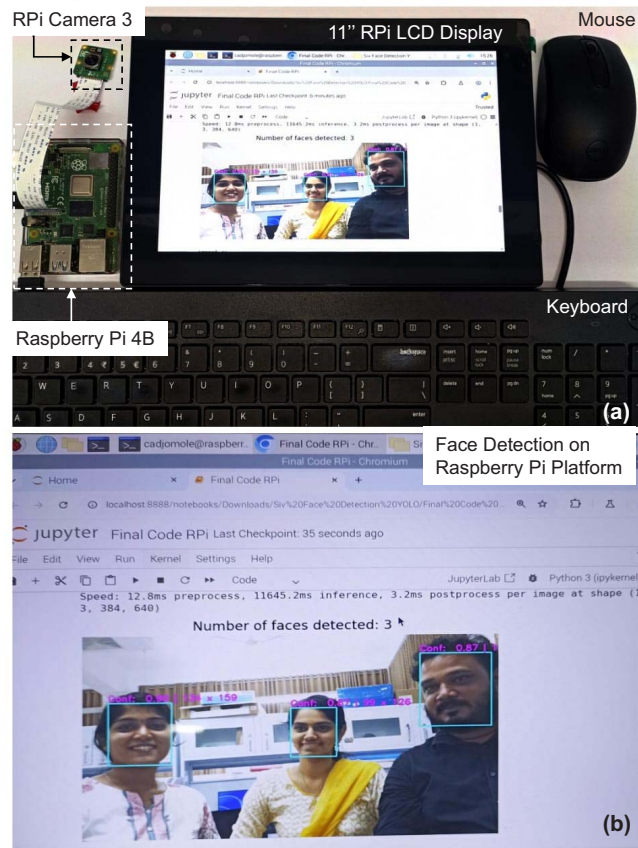


FIGURE 30. Real-time implementation of the YOLOv9-based face detection and people counting method on Raspberry Pi 4B edge computing platform using live images from the camera. (a) Experimental setup for implementation on Raspberry Pi; (b) Live face detection output from the camera using Raspberry Pi, showing three correctly identified faces. This proves the model's efficiency on resource-constrained edge devices, supporting practical implementation for low-cost, real-time people counting or monitoring systems.

However, performance significantly deteriorated under increasing levels of image degradation. Our experiments revealed that both blurring and noise, particularly motion blur and Gaussian noise, adversely affected model accuracy. While filtering and denoising methods can mitigate these effects, a key finding of this work is the need for automatic, reference-free image quality assessment to identify degradation types and selectively apply preprocessing. This ensures optimal trade-offs between restoration quality and model sensitivity.

A more nuanced challenge emerges when the scene

contains animal or artificial faces (e.g., statues or wall paintings). In such cases, the YOLOv9 model failed to distinguish live human faces from visually similar non-human entities. Although fine-tuning with additional data may alleviate these misclassifications, such strategies risk reducing overall generalizability. For instance, tuning to suppress monkey-like animal faces may lead to missed detections of certain human faces with similar geometric features (e.g., jawline, eye spacing).

When face detection and counting are used in surveillance and people tracking applications, particularly in people monitoring and tracking, blurred images can often be encountered due to the movement of objects (humans, animals, etc.) and/or movement of the camera (in drone/robot-based people monitoring). Although the model was created with the standard Wider Face database and YOLOv9 architecture (extensively used in past studies), in this paper, we studied the performance of the trained YOLOv9 model in different kinds of blurring effects (average, disk, Gaussian, and motion) and observed a degradation in performance with higher levels of blurring. To improve the performance of the deep model under blurred images, the following strategies can be adapted:

- 1) Incorporating an automated reference-free image quality assessment method that can detect blurred images and identify types of blurs, as the preprocessing step for improving the performance of the deep learning-based face detection method by applying a suitable deblurring technique followed by the deep learning process, wherein the trained model is created with a database that does not have any blurred images.
- 2) The performance of the YOLOv9 architecture may be improved if the trained deep face detection model is created with a wide variety of blurred images with different levels of severity, in addition to images with different qualities and orientations. However, the deep model generated with blurred images may not ensure reliable face detection under a wide variety of blurred images that can be encountered in real-world application environments. Therefore, automated detection of blurred images and their blur types is highly demanded to improve the robustness of the deep learning-based face detection method, either by discarding severely blurred images or applying deblurring techniques when the level of blurring is minimal.

The purpose of this paper is to study the accuracy and robustness of the YOLOv9 architecture under different indoor and outdoor environments where the image may include artificial faces (wall-painted faces, crafted faces, statue faces), animal faces, and face pareidolia (the misperception of illusory faces formed by objects in the image), in addition to human faces. We have validated the model, which was created using the Wider Face dataset (extensively used in past studies), under these application scenarios. In this paper, fine-tuning was suggested as a future solution in discriminating human faces from pareidolia effect-induced faces and a mixture of human and animal faces. For different key challenges with preliminary evaluation results, this paper suggested different strategies such as automated image quality assessment, noise type identification, and noise level estimation as preprocessing steps to improve the robustness of the deep face model, which were not highlighted in past studies, although face detection methods were reported using the Wider Face database with the performance of the model evaluated using images from the same dataset. The observations in this study also reveal the lack of generalization and the failure of the existing methods and databases in incorporating diverse operating environments.

From a theoretical standpoint, the model approximates a mapping $f_{\theta}(x)$ from the image domain $\mathbb{R}^{H \times W \times 3}$ to structured predictions (bounding boxes, class scores), where θ is a fixed parameter set learned via global optimization. However, this mapping assumes that all test inputs lie within the support of the training distribution. Our robustness analysis reveals that $f_{\theta}(x)$ exhibits high variance under out-of-distribution perturbations (e.g., occlusion, blur, noise), due to a lack of adaptive or probabilistic control in the learned parameter space. Furthermore, the loss function L_{total} —composed of box, classification, and distribution focal loss terms—does not encode robustness constraints explicitly. As a result, the trained model optimizes for nominal performance while remaining vulnerable to structured noise. This is reflected in the long-tailed per-image loss distributions under distortion (see Figs 5, 12, and 16). A non-parametric Friedman test and post-hoc Wilcoxon signed-rank analysis (Section III.E) confirm statistically significant performance degradation across increasing noise levels ($p < 0.001$).

In summary, while YOLOv9 achieves state-of-the-art performance on canonical test sets, its deployment in unconstrained environments necessitates further enhancement. This includes adaptive inference mechanisms, robust training paradigms, and integration with image quality assessment modules to ensure reliable face detection and counting in real-world conditions.

IV. CONCLUSION

In this paper, we presented a face detection and people counting method using the YOLOv9 object detection model trained using the Wider Face database and evaluated the model in terms of standard performance metrics, computational time, and model size. We extensively studied

the performance of the YOLOv9-based face detection model in terms of accurately counting the number of faces and accurately determining the face boundary under skin color variations, different face sizes and number of faces, mixture of human and animal faces, image properties (brightness, contrast, illumination, glare), image qualities (image blurring, low-light facial images, lens flared images, grainy images taken at night or in low light, different types of noises), and pareidolia or apophenia effects. We created image databases with different kinds of image qualities and image characteristics with the above-mentioned challenges. Evaluation of the model on a variety of real-world scenarios, such as using a variety of animal faces, infant faces, and artificial faces, revealed the lack of generalizability of the existing databases and trained models, which causes many false positives. In the study, we also observed a "pareidolia" effect, which caused false positives at random places in the image. Results further showed that the deep face detection model can detect animal faces, which are similar to human faces, that are unavoidable in many practical applications where people live with animals or pets in both indoor and outdoor applications. In addition, the robustness of the face detection and counting method was studied with different levels of blurring effects and different levels of salt and pepper, Gaussian, and speckle noises. Results showed that there is a need for image quality assessment and a noise-specific filtering approach with optimal denoising parameters. The YOLOv9 people counting model achieved a mean absolute error (MAE) of 3.36 and a root mean square error (RMSE) of 22.38 on the Wider Face database with an inference time of 15.2 ms and model size of 49.2MB, which outperformed existing deep learning-based people-counting methods reported in the past studies. In the future direction, we attempt to improve the robustness of the YOLOv9-based face detection method by incorporating image quality assessment, image noise identification, and noise-specific filtering techniques.

REFERENCES

- [1] N. AlDahoul, M. J. T. Tan, H. R. Kasireddy, and Y. Zaki, "Exploring vision language models for facial attribute recognition: Emotion, race, gender, and age," *arXiv preprint arXiv:2410.24148*, 2024.
- [2] Z. Zhang, W. Shen, S. Qiao, Y. Wang, B. Wang, and A. Yuille, "Robust face detection via learning small faces on hard images," in *2020 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1350–1359.
- [3] X. Zhang, "Research on facial expression recognition based on yolov9," in *2024 IEEE 2nd International Conference on Sensors, Electronics and Computer Engineering (ICSECE)*, 2024, pp. 208–213.
- [4] X. Long, J. Zhang, and S. Shan, "Generalized face liveness detection via de-fake face generator," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 3, pp. 1818–1831, 2025.
- [5] B. Wang, H. Liu, D. Samarasinghe, and M. H. Nguyen, "Distribution Matching for Crowd Counting," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1595–1607, 2020.
- [6] L. Liu, Z. Qiu, G. Li, S. Liu, W. Ouyang, and L. Lin, "Crowd Counting with Deep Structured Scale Integration Network," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1774–1783.
- [7] D. B. Sam, S. V. Peri, M. N. Sundararaman, A. Kamath, and R. V. Babu, "Locate, Size, and Count: Accurately Resolving People in Dense Crowds

- via Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 8, pp. 2739–2751, 2021.
- [8] Y.-L. Hou and G. K. Pang, “People Counting and Human Detection in a Challenging Situation,” *IEEE Transactions on Systems, Man, and Cybernetics-part A: Systems and Humans*, vol. 41, no. 1, pp. 24–33, 2010.
- [9] S. Vasantha, B. Kiranmai, M. A. Hussain, S. S. Hashmi, L. Nelson, and S. Hariharan, “Face and Object Detection Algorithms for People Counting Applications,” in *2nd International Conference on Automation, Computing and Renewable Systems (ICACRS)*. IEEE, 2023, pp. 1188–1193.
- [10] K. Badr, K. Iqbal, S. Mahboob, and M. Y. Khan, “Processing Fire Detection, Face Recognition, and Head Counting for an Iot-based Application in Emergency Exit,” in *2021 Mohammad Ali Jinnah University International Conference on Computing (MAJICC)*. IEEE, 2021, pp. 1–6.
- [11] V. Ranjan, H. Le, and M. Hoai, “Iterative Crowd Counting,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 270–285.
- [12] D. Garg, P. Goel, S. Pandya, A. Ganatra, and K. Kotecha, “A Deep Learning Approach for Face Detection using YOLO,” in *2018 IEEE Punecon*, 2018, pp. 1–4.
- [13] L. P. e. Silva, J. C. Batista, O. R. Bellon, and L. Silva, “YOLO-FD: YOLO for Face Detection,” in *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications: 24th Iberoamerican Congress, CIARP 2019, Havana, Cuba, October 28-31, 2019, Proceedings 24*. Springer, 2019, pp. 209–218.
- [14] C. Li, R. Wang, J. Li, and L. Fei, “Face Detection based on YOLOv3,” in *Recent Trends in Intelligent Computing, Communication and Devices: Proceedings of ICCD 2018*. Springer, 2020, pp. 277–284.
- [15] S. H. Tuli, A. Mao, and W. Liu, “A Novel Face Detector based on YOLOv3,” in *AI 2020: Advances in Artificial Intelligence: 33rd Australasian Joint Conference, AI 2020, Canberra, ACT, Australia, November 29–30, 2020, Proceedings 33*. Springer, 2020, pp. 55–68.
- [16] W. Chen, H. Huang, S. Peng, C. Zhou, and C. Zhang, “YOLO-FACE: A Real-Time Face Detector,” *The Visual Computer*, vol. 37, pp. 805–813, 2021.
- [17] H. Aung, A. V. Bobkov, and N. L. Tun, “Face Detection in Real Time Live Video Using YOLO Algorithm Based on VGG16 Convolutional Neural Network,” in *2021 International Conference on Industrial Engineering, Applications and Manufacturing (ICIEAM)*, 2021, pp. 697–702.
- [18] G. Castellano, B. De Carolis, N. Marvulli, M. Sciancalepore, and G. Vessio, “Real-time Age Estimation from Facial Images using YOLO and EfficientNet,” in *Computer Analysis of Images and Patterns: 19th International Conference, CAIP 2021, Virtual Event, September 28–30, 2021, Proceedings, Part II 19*. Springer, 2021, pp. 275–284.
- [19] S. Abbasi, H. Abdi, and A. Ahmadi, “A Face-Mask Detection Approach based on YOLO Applied for a New Collected Dataset,” in *2021 26th International Computer Conference, Computer Society of Iran (CSICC)*, 2021, pp. 1–6.
- [20] A. Ali-Gombe, E. Elyan, C. F. Moreno-García, and J. Zwiagelaar, “Face Detection with YOLO on Edge,” in *International Conf. on Engineering Applications of Neural Networks*. Springer, 2021, pp. 284–292.
- [21] A. S. Sanchez-Moreno *et al.*, “Efficient Face Recognition System for Operating in Unconstrained Environments,” *Journal of Imaging*, vol. 7, no. 9, p. 161, 2021.
- [22] Z. Yu, H. Huang, W. Chen, Y. Su, Y. Liu, and X. Wang, “YOLO-Facev2: A Scale and Occlusion Aware Face Detector,” *arXiv preprint arXiv:2208.02019*, 2022.
- [23] W. Pebrianto, P. Mudjirahardjo, and S. H. Pramono, “YOLO Method Analysis and Comparison for Real-Time Human Face Detection,” in *2022 11th Electrical Power, Electronics, Communications, Controls and Informatics Seminar (EECCIS)*, 2022, pp. 333–338.
- [24] S. Khalili and A. Shakiba, “A Face Detection Method via Ensemble of Four Versions of YOLOs,” in *2022 International Conference on Machine Vision and Image Processing (MVIP)*, 2022, pp. 1–4.
- [25] J. Gao and T. Yang, “Face Detection Algorithm based on Improved TinyYOLOv3 and Attention Mechanism,” *Computer Communications*, vol. 181, pp. 329–337, 2022.
- [26] D. Qi, W. Tan, Q. Yao, and J. Liu, “YOLO5Face: Why Reinventing a Face Detector,” in *Computer Vision – ECCV 2022 Workshops*, L. Karlinsky, T. Michaeli, and K. Nishino, Eds. Cham: Springer Nature Switzerland, 2023, pp. 228–244.
- [27] A. A. Sufian Chan, M. Abdullah, S. M. Mustam, F. A. Poat, and A. Joret, “Face Detection with YOLOv7: A Comparative Study of YOLO-Based Face Detection Models,” in *International Conference on Green Energy, Computing & Sustainable Technology (GECOST)*, 2024, pp. 105–109.
- [28] I. Al Amoudi and D. A. Ramli, “YOLOv7-Tiny and YOLOv8n Evaluation for Face Detection,” in *International Conference on Robotics, Vision, Signal Processing and Power Applications*. Springer, 2024, pp. 477–483.
- [29] M. Zulfiqar, F. Syed, M. J. Khan, and K. Khurshid, “Deep Face Recognition for Biometric Authentication,” in *2019 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*. IEEE, 2019, pp. 1–6.
- [30] G. Y. Chen, “An Experimental Study for the Effects of Noise on Face Recognition Algorithms under Varying Illumination,” *Multimedia Tools and Applications*, vol. 78, pp. 26 615–26 631, 2019.
- [31] J. Yu, K. Sun, F. Gao, and S. Zhu, “Face Biometric Quality Assessment via Light CNN,” *Pattern Recognition Letters*, vol. 107, pp. 25–32, 2018.
- [32] A. Kumar, K. Jani, A. K. Jishu, H. Patel, A. K. Sharma, and M. Khare, “Evaluation of Deep Learning Based Human Expression Recognition on Noisy Images,” in *2020 7th International Conference on Soft Computing & Machine Intelligence (ISCMI)*, 2020, pp. 187–191.
- [33] J. Guo *et al.*, “Dominant and Complementary Emotion Recognition from Still Images of Faces,” *IEEE Access*, vol. 6, pp. 26 391–26 403, 2018.
- [34] D. Qiao, A. H. Ayesha, F. Zulkernine, N. Jaffar, and R. Masroor, “ReViSe: Remote Vital Signs Measurement Using Smartphone Camera,” *IEEE Access*, vol. 10, pp. 131 656–131 670, 2022.
- [35] A. Neethu, K. Bijlani *et al.*, “People Count Estimation using Hybrid Face Detection Method,” in *2016 International Conference on Information Science (ICIS)*. IEEE, 2016, pp. 144–148.
- [36] Y. al Atrash, M. Saad, and I. H. Alshami, “Detecting and Counting People’s Faces in Images Using Convolutional Neural Networks,” in *2021 Palestinian International Conference on Information and Communication Technology (PICICT)*. IEEE, 2021, pp. 116–122.
- [37] Y. Wang, J. Hou, X. Hou, and L.-P. Chau, “A Self-Training Approach for Point-Supervised Object Detection and Counting in Crowds,” *IEEE Transactions on Image Processing*, vol. 30, pp. 2876–2887, 2021.
- [38] T.-O. Ha *et al.*, “Improvement of People Counting by Pairing Head and Face Detections from Still Images,” in *International Conference on Multimedia Analysis and Pattern Recognition (MAPR)*. IEEE, 2021, pp. 1–6.
- [39] A. K. Jaiswal, D. A. Subhash, R. Raj, and Y. Arora, “Multi-version analysis of yolo: A deep dive into yolov5, yolov7, and yolov9 for face and person detection,” in *2024 International Conference on Progressive Innovations in Intelligent Systems and Data Science (ICPIDS)*, 2024, pp. 283–288.
- [40] J. A. Rodríguez-Rodríguez, E. López-Rubio, J. A. Ángel-Ruiz, and M. A. Molina-Cabello, “The Impact of Noise and Brightness on Object Detection Methods,” *Sensors*, vol. 24, no. 3, p. 821, 2024.
- [41] Y. Zhou, D. Liu, and T. Huang, “Survey of Face Detection on Low-Quality Images,” in *2018 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018)*. IEEE, 2018, pp. 769–773.
- [42] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information,” *arXiv preprint arXiv:2402.13616*, 2024.
- [43] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [44] S. Li, H. Huang, X. Meng, M. Wang, Y. Li, and L. Xie, “A Glove-Wearing Detection Algorithm based on Improved YOLOv8,” *Sensors*, vol. 23, no. 24, p. 9906, 2023.
- [45] F. Massimi, P. Ferrara, R. Petrucci, and F. Benedetto, “Deep Learning-Based Space Debris Detection for Space Situational Awareness: A Feasibility Study Applied to the Radar Processing,” *IET Radar, Sonar & Navigation*, 2024.
- [46] W. Zhang, Y. Zhou, Y. Wang, R. Wang, and H. Yang, “Automatic Crack Detection and Segmentation of Masonry Structure Based on YOLOv9-Seg and Edge Detection,” *Available at SSRN 4812249*.
- [47] Y. Li, Z. Hu, Y. Zhang, J. Liu, W. Tu, and H. Yu, “DDEYOLOv9: Network for Detecting and Counting Abnormal Fish Behaviors in Complex Water Environments,” *Fishes*, vol. 9, no. 6, p. 242, 2024.
- [48] L. Song, Y. Wu, and W. Zhang, “Research on Fire Detection Based on the YOLOv9 Algorithm,” in *2024 IEEE 4th International Conference on Electronic Technology, Communication and Information (ICETCI)*, 2024, pp. 1398–1406.
- [49] S. Yang, P. Luo, C. C. Loy, and X. Tang, “WIDER FACE: A Face Detection Benchmark,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

- [50] E. Casas, L. Ramos, E. Bendek, and F. Rivas-Echeverría, "Assessing the Effectiveness of YOLO Architectures for Smoke and Wildfire Detection," *IEEE Access*, vol. 11, pp. 96554–96583, 2023.
- [51] Y. Al Othmani, P. Kanungo, S. Ghanem, and A. A. Alnaeb, "Customized YOLOv3 Model for Face Detection in Crowded Images," in *2022 6th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*, 2022, pp. 1–6.
- [52] R. Shaik and R. SR, "OIPFST: AI-based Fitzpatrick Skin Tone Labelling Utilizing Skin Lesions," *Multimedia Tools and Applications*, pp. 1–21, 2024.
- [53] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 589–597.
- [54] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, Oct 2016.
- [55] G. W. Corder and D. I. Foreman, *Nonparametric statistics: A step-by-step approach*. John Wiley & Sons, 2014.
- [56] A. C. Bovik, *The Essential Guide to Image Processing*. Academic Press, 2009.
- [57] R. C. Gonzalez, *Digital Image Processing*. Pearson Education India, 2009.
- [58] Y. Wu, H. Guo, C. Chakraborty, M. R. Khosravi, S. Berretti, and S. Wan, "Edge Computing Driven Low-Light Image Dynamic Enhancement for Object Detection," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 5, pp. 3086–3098, 2023.
- [59] A. Zein-Eddine, O. Messai, A. Bentamou, and Y. Gavet, "Enhancing Object Detection in Distorted Environments: A Seamless Integration of Classical Image Processing with Deep Learning Models," in *2024 IEEE 12th International Symposium on Signal, Image, Video, and Communications (ISIVC)*, 2024, pp. 1–5.
- [60] Oxford English Dictionary. "Pareidolia (n.). S.v. "pareidolia (n.)." [Online]. Available: <https://doi.org/10.1093/OED/1305332741>
- [61] M. Hamilton, S. Stent, V. DuTelle, A. Harrington, J. Corbett, R. Rosenholtz, and W. T. Freeman, "Seeing Faces in Things: A Model and Dataset for Pareidolia," *arXiv preprint arXiv:2409.16143*, 2024.
- [62] Y. Tian, Q. Ye, and D. Doermann, "Yolov12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.
- [63] L. Cuimei, Q. Zhiliang, J. Nan, and W. Jianhua, "Human face detection algorithm via haar cascade classifier combined with three additional classifiers," in *2017 13th IEEE International Conference on Electronic Measurement & Instruments (ICEMI)*, 2017, pp. 483–487.
- [64] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, "Retinaface: Single-stage dense face localisation in the wild," *arXiv preprint arXiv:1905.00641*, 2019.
- [65] S. A. Jakhete and N. Kulkarni, "A comprehensive survey and evaluation of mediapipe face mesh for human emotion recognition," in *2024 8th International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, 2024, pp. 1–8.
- [66] S. Vignesh Baalaji, S. Sandhya, S. Sajidha, V. Nisha, M. Vimalapriya, and A. K. Tyagi, "Autonomous face mask detection using single shot multibox detector, and resnet-50 with identity retrieval through face matching using deep siamese neural network," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 8, pp. 11 195–11 205, 2023.



SIVARANJINI PERIKAMANA NARAYANAN

(Graduate Student Member, IEEE) received the B.Tech degree in electrical and electronics engineering from the Government Engineering College, Thrissur, Kerala, India, in 2022. She is currently pursuing the MS+PhD dual degree jointly with the Department of ICT, University of Agder, Grimstad, Norway, and with the Department of Electrical Engineering, Indian Institute of Technology Palakkad, Kerala, India.

Her research interests include biomedical signal processing, image processing, wearable healthcare monitoring, and machine learning.



M. SABARIMALAI MANIKANDAN (Senior Member, IEEE) received the B.E. degree in electronics and communication engineering from the Bharathiar University, Coimbatore, India, the M.E. degree in microwave and optical engineering from the Madurai Kamaraj University, Madurai, India, and the PhD degree in cardiovascular signal processing from the Department of Electronics and Communication Engineering, IIT Guwahati, Guwahati, India. He was an Assistant Professor

at Amrita Vishwa Vidyapeetham University, Ettimadai, India. He was a Chief Engineer with the Advanced Technology Group, Samsung India Electronic Pvt. Ltd., Noida, India. He was an Assistant Professor at the Biomedical System Laboratory, School of Electrical Sciences, IIT Bhubaneswar, India. He is currently an Associate Professor of Electrical Engineering at the Indian Institute of Technology Palakkad. He has published over 70 research papers in reputed journals and conference proceedings. His research interests include signal and image processing, adaptive machine learning, the Internet of Things, VLSI signal processing, machine learning architectures, application system development: health (human, machine, and structural) monitoring systems, audio and speech processing systems for human-machine interactions, biometric and data security for authentication and authorization, environmental monitoring systems for ambient assisted living, UAV-assisted IoT for smart surveillance systems, and context and quality-aware pattern learning networks for event recognition. He was a recipient of the 2012 Outstanding Performance Award during his tenure at Samsung India Electronic Pvt. Ltd. He served as a Reviewer for many reputed journals of the IEEE, IET, Springer, Hindawi, PLOS One, Frontiers, and Elsevier.



LINGA REDDY CENGERAMADDI (SENIOR MEMBER, IEEE) received the master's degree in electrical engineering from the Indian Institute of Technology Delhi (IIT Delhi), New Delhi, India, in 2004, and the Ph.D. degree in electrical engineering from the Norwegian University of Science and Technology (NTNU), Trondheim, Norway, in 2011.

He worked at Texas Instruments on mixed-signal circuit design before joining the Ph.D. Program at NTNU. After finishing his Ph.D. degree, he worked on radiation imaging for an atmosphere-space interaction monitor (ASIM mission to the International Space Station) at the University of Bergen, Bergen, Norway, from 2010 to 2012. He is currently the Leader of the Autonomous and Cyber-Physical Systems (ACPS) Research Group and a Professor at the University of Agder, Grimstad, Norway. He has co-authored over 250 research publications that have been published in prestigious international journals and standard conferences in the research areas of Internet of Things (IoT), Cyber-Physical systems, Autonomous systems, Robotics and Automation involving advanced sensor systems, Computer vision, Thermal imaging, LiDAR imaging, Radar imaging, wireless sensor networks, smart electronic systems, advanced machine learning techniques, connected autonomous systems including drones/unmanned aerial vehicles (UAVs), unmanned ground vehicles (UGVs), unmanned underwater systems (UUSs), 5G- (and beyond) enabled autonomous vehicles, and Socio-Technical Systems like urban transportation systems, Smart Agriculture and Smart Cities. He is also quite active in medical imaging. Several of his master's students won the best master thesis award in information and communication technology (ICT).

Dr. Cengeramaddi is also a senior member of IEEE, a member of ACM, and a member of the editorial boards of various international journals and the technical program committees of several IEEE conferences. He is the Principal Investigator and a Co-Principal Investigator of many research grants from the Norwegian Research Council.

...